UNIVERSITÄT KOBLENZ · LANDAU

IWVI

Malware detection using deep learning and machine learning with Keras Tuner

Masterarbeit

zur Erlangung des Grades eines Master of Science (M.Sc.) im Studiengang Computervisualistik

Julian Stadager

[213201351]

Koblenz, December 30, 2021

Erstgutachter:	Prof. Dr. Andreas Mauthe
	(Institut für Wirtschafts- und Verwaltungsinformatik, FG Mauthe)
Zweitgutachter:	Alexander Rosenbaum, M. Sc.
	(Institut für Wirtschafts- und Verwaltungsinformatik, FG Mauthe)

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe und dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe und von dieser als Teil einer Prüfungsleistung angenommen wurde.

Mit der Einstellung der Arbeit in die Bibliothek bin ich einverstanden. ja ☐ nein ☐

Der Veröffentlichung dieser Arbeit im Internet stimme ich zu. ja ☐ nein ☐

.....

(Ort, Datum) (Unterschrift)

Zusammenfassung

In der gegenwärtigen Zeit der Technologie nehmen Bedrohungen in Bezug auf Cybersicherheit zu. Insbesondere während der COVID-19-Pandemie missbrauchen Cyberkriminelle die aktuelle Situation, um Malware einzusetzen. Malware-Ersteller umgehen zunehmend traditionelle Malware-Erkennungsmethoden, was Cybersicherheitsforscher dazu antreibt, innovative Strategien zur Verbesserung der Cyberabwehr einzusetzen. Innovative Methoden bestehen aus intelligenten Algorithmen, die Schadsoftware mithilfe statischer oder dynamischer Analyse erkennen. Diese Arbeit verwendet Techniken des Machine-Learnings, um portable executable Malware basierend auf statischer Analyse zu erkennen. In dieser Arbeit findet ein Vergleich zwischen Deep-Learning, Machine-Learning und verschiedenen Datensätzen zur Erkennung von Malware statt. Dazu werden fünf Machine-Learning-Klassifikatoren, bestehend aus zwei Deep-Learning-Methoden, eingesetzt. Diese Lernmodelle sind Random Forest, KNN, SVM, MLP und ein DNN. Das Framework umfasst zwei Datensätze mit verschiedenen Merkmalen, den Dimensionsreduktionsalgorithmus PCA und die Hyperparameter Optimierungsbibliothek Keras Tuner. Die generierten Ergebnisse werden mit Keras Tuner optimiert und dann innerhalb der Datensätze und mit Ergebnissen verwandter Arbeiten, anhand vorgegebener Bewertungsmetriken, verglichen. Darüber hinaus wird die Aussagekraft der Merkmale berechnet und analysiert, welches wichtige Erkenntnisse für zukünftige Ansätze zur Merkmalsauswahl liefert. Die Ergebnisse zeigen, dass Random Forest (99,35%) und MLP (99,30%) die höchste Genauigkeit über beide Datensätze erreichen. In Anbetracht der Integration von Keras Tuner schneidet MLP mit dem genannten Framework am besten ab und kann die Genauigkeit um etwa 0,3% verbessern. Die Merkmalsanalyse ergibt, dass in beiden Datensätzen kritische Merkmale fehlen, was sich auf die Leistung der Modelle auswirkt.

Abstract

In the modern era of technology, threats concerning cybersecurity and safety are increasing. Especially during the COVID-19 pandemic, cybercriminals abuse the novel scenario to deploy malware. Malware creators increasingly bypass traditional malware detection methods, propelling cybersecurity researchers to utilize innovative strategies to improve cyber defenses. Innovative methods consist of intelligent algorithms that detect malware with static or dynamic analysis. This work uses machine learning techniques to detect portable executable malware based on static analysis. In this thesis, a comparison is made between deep learning, machine learning, and different datasets to detect malware. Therefore, five machine learning classifiers, consisting of two deep learning methods, are deployed. These machine learning models are Random Forest, KNN, SVM, MLP, and a DNN. The framework features two datasets containing various features, the dimensionality reduction algorithm PCA and the hyperparameter optimization library Keras Tuner. The obtained results are tuned with Keras Tuner and then compared within the datasets and other approaches based on specified evaluation metrics. Furthermore, the feature importance is computed and analyzed, giving critical insights for future feature selection approaches. The results show that Random Forest (99.35%) and MLP (99.30%) achieve the highest accuracy over both datasets. Considering the integration of Keras Tuner, MLP performs best with this framework, which enables Keras Tuner to improve accuracy by around 0.3%. The feature analysis reveals that critical features are missing in both datasets, impacting the performance of the models.

Contents

List of Figures

List of Tables

1. Introduction

Recent statistics show that cyber-attacks are on the rise, targeting companies, public facilities, or private individuals. Especially in the COVID-19 pandemic, cybercrime has increased up to 600% [1]. Statistics reports such as the consumer identity breach report [2] showcase an increased number of attacks on private sectors. Cybercriminals abuse the novel scenario, which originated from the pandemic, to tempt victims with false information [3]. For example, a vaccine appointment confirmation, which generally contains a link and a portable document format (PDF) file, can quickly be sent by a cybercriminal, resulting in data theft or loss of control of the system.

Malware is a broad term for malicious software and describes any code performing malicious actions against the victim's system [4]. The earliest documented malware appeared in 1971 and was known as the Creeper Worm, which could replicate itself [5]. The malicious code was the beginning of an industrial rise of malicious programs, resulting in a cyclical arms race. Current malware can be categorized based on the functionality and the goal of the cybercriminals [6]. For example, ransomware is traditionally aimed at larger companies, encrypting system files to generate a ransom for decryption.

Cybersecurity researchers need innovative methods to improve cyber defenses to keep up with the malware threat expansion. Enhancing better cyber defenses can consistently be complex due to technological advances and advanced obfuscation techniques [7, 8]. Additionally, the skill required to utilize and develop malware has decreased due to various available tools on the internet [9]. The endpoint protection provides a suite of software to detect and remove threats for the private user. For example, the endpoint software, Avira, is powered by an Anti Virus (AV) engine that receives periodic upgrades. Through a signature-based method, the AV can recognize malware and remove threats [10].

In some cases, the internet user can manually decide if the observed file is classified as malicious or clean. The signature-based method compares files on the endpoint, for example, on the android phone with signatures of known malicious code in its database [11]. The standard AV engine has several downsides. One of the downsides is time consumption. Additionally, the endpoint user has to manually update the software and utilize a real-time scanner. The major vulnerability arises from the new malware threats, which traditional AV scanners cannot detect due to attackers taking advantage of the weakness of the signature-based method [12]. The other AV engine detection method applies a heuristic-based approach. The AV engine scanner detects malware using rules and algorithms on the possible malicious command in the

code [13]. Current AV engines like Avast combine Heuristic-based methods with signature-based techniques [10].

Signature-based and heuristic-based methods presuppose that malware is analyzed to identify patterns. Cybersecurity researchers can then utilize the identified malware patterns to classify malware and develop better cyber defenses. The objective of malware analysis is to understand and study the behavior of malware [14]. Malware analysis consists of extracting information from malware samples, analyzing the characteristics and the functionalities. Malware analysis can be classified into four types: (1) static analysis, (2) dynamic analysis, (3) code analysis, and (4) memory analysis. Static analysis is the process of examining the binary of the file without execution. It is an initial analysis method to determine how to analyze and classify the concerned file [15].

On the other hand, dynamic analysis is the process of examining the file by performance. The execution is done in an isolated environment while being monitored. The analyzed activity provides information about the functionality and a possible pattern in the malware variant [16]. Code analysis consists of analyzing code with the prerequisites of having the skill to understand the coding language. Code analysis is a technique based on static and dynamic code analysis [17]. The final method, memory analysis, is commonly used in IT forensics. Memory analysis is used to determine malware's stealth and evasive functionalities by analyzing the user's endpoint RAM [18]. The method can be very relevant due to advanced obfuscation techniques.

The techniques mentioned above, signature-based, and heuristic-based are increasingly challenging to use in modern AV engines due to new malware variants and data storage advancements. Additionally, cybersecurity experts cannot keep pace with technological advances and the resulting vulnerabilities [19]. Machine learning techniques can diminish the issues identified here since they can handle large amounts of data and improve progressively with training data. Modern malware research is aimed towards an automated learning system that can detect unknown malware [20]. The ability for the automated learning systems to learn and improve makes them requisite for an intelligent malware protection system. Malware detection is an area of research with many artificial recognition techniques. For example, Gibert et al. [20] present a survey about current research methods and why malware detection research can be considered incomplete.

This thesis aims to deploy a malware detection framework to identify malware by extracting features from portable executable files. The objective is to utilize different machine learning techniques for malware detection. This process is executed with two datasets. The framework consists out of the following techniques:

1. Random Forest
2. k-Nearest Neighbors (KNN)
3. Support Vector Machine (SVM)
4. Multilayer Perceptron (MLP)
5. Deep Neural Network (DNN)

The two distinct datasets contain similar portable executable features, elaborated in the methodology section. The portable executable features of the datasets are reduced with a dimension reduction algorithm, PCA. Features of the datasets are then reduced to a set of components. After the models have been implemented, they are optimized with the Keras Tuner [21], a hyperparameter optimization library. During this process, each model is analyzed based on the prediction performance and computation time. This concludes the framework utilizing Keras Tuner with the proposed models. Based on the framework results, the features of each dataset are then analyzed and compared, presenting the importance of features.

This thesis is organized as follows. Section 2 describes the background of malware detection and explains the concepts of the proposed machine learning techniques. Additionally, the feature reduction algorithm and the format of the portable executable format are presented. Section 3 provides a summary of related work and current research insights. In section 4, a brief methodology of the used framework, the datasets, and the architecture of the models is illustrated.

Additionally, section 5 illustrates the classification results of the utilized models. The classification results are then evaluated and discussed in section 6. Finally, section 7 presents a summary and the possible future work of this thesis.

2. Fundamentals of Malware Detection with Machine Learning

2.1. Static Malware Analysis

Malware analysis is the process of observing the behavior and the properties of a malicious program file. The structure of an unknown sample can be decisive to classify it as malicious or benign. With the assistance of statistical tools, disassembled samples are evaluated and analyzed. This type of analysis is called static malware analysis. An example of static malware analysis is the extraction of strings from a portable executable file, giving insights into whether the file will access suspicious URLs [22].

On the other hand, dynamic analysis executes the malicious file in a controlled environment. By observing the behavior of the sample, it can be determined if it is malicious or benign. For example, for memory forensics, the malicious sample is run in a lab to analyze memory behavior [23].

The process of static analysis focuses on the resources that a sample contains. Although the static analysis has shortcomings, it provides a good understanding of how the sample functions. Understanding the structure of malware is the primary goal of static malware analysis. It clarifies how attackers obfuscate malicious content and which systems are vulnerable. The first step of static malware analysis is the disassembly of the executable, namely reverse-engineering. The next phase involves the analyst examining the resulting assembly language code and determining if the file is supposed to do its intent. Static malware analysis can be very effective and fast to determine whether a file is malicious. However, malicious files with sophisticated characteristics are challenging to classify. For example, the trojan Emotet is persistent malware designed to propagate fast [24]. Additionally, without the inclusion of dynamic analysis, critical information for a deeper understanding of the behavior of a malicious file is lost. Therefore, a hybrid approach is decisive to understand the malicious intent thoroughly [25]. A standard tool to analyze malware files is Virustotal [26]. The statistics of Virustotal show that the most analyzed file type is portable executable, followed by HTML and Android.

2.1.1. Portable Executable Format

The portable executable file format first appeared with Windows NT and is commonly used to execute a computer program in modern Windows operating systems. Furthermore, the format

describes program files with the endings .exe, .dll, and .sys. The structure of the portable executable file conforms to the Common Object File Format (COFF). Portable executable files consist of a series of structures, categories respectively. These structures contain the necessary information for the operating system, which is then loaded into the program's memory. The information consists of x86 instructions, data containing images and text, and metadata [6].

The semantics distinguish files with the EXE and Dynamic Link Library (DLL) endings. DLL files utilize the same format as the EXE file; however, a specific bit in the section characteristics determines whether the file is treated as EXE or DLL. This fundamental structure of the file format has several advantages. One advantage is that the data structure of the portable executable file is easily accessible after it has been loaded into the memory. When the portable executable file is loaded into memory, specific ranges of the file are mapped into the address space. Thus, information on the portable executable file is identical after being loaded into memory [27]. The mapping of the portable executable file into memory is dependant on the offset. For example, a higher offset in the file corresponds to a higher memory address.

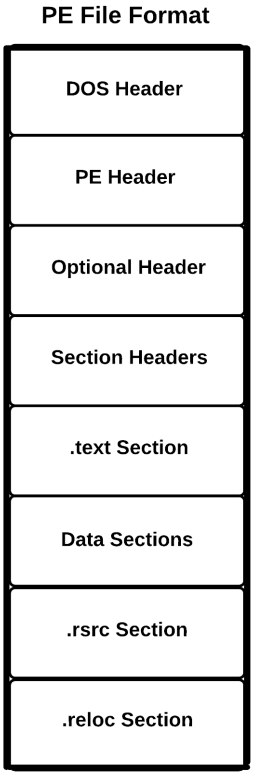


Figure 2.1.: Structure of a portable executable file [28]

Figure 2.1 presents a general structure of the portable executable file. The first component of the file format is the MS-DOS header. The MS-DOS real-mode stub program and the PE file

signature are summarized in this figure with the PE file header. The next section features the PE file optional header. Consequently, the section headers and section bodies are a fundamental component of the file structure. Finally, the file structure is closed by miscellaneous information concerning .text, .idata, .rsrc and .reloc sections. These sections are explained in more detail below.

The DOS header marks the beginning of the portable executable file. When loading the file into a DOS operating system, the system can read and determine if the file is compatible with the current version [29]. If the file is not executable, the linker returns the message: "This program cannot be run in DOS mode." The DOS header contains fields like *e_magic*, often referred to as magic number. The magic number defines the identification of the format of the file. For example, GIF files contain the ASCII-Code *GIF89a* at the beginning of the file. The last area, *e_lfanew*, locates the address of the new header, namely the PE file header. In principle, the DOS header is mainly utilized for compatibility.

The subsequent header, PE file header, defines the attributes of the portable executable. These attributes are *Machine*, *NumberOfSections*, *TimeDateStamp*, *PointerToSymbolTable*, *NumberOfSymbols*, *SizeOfOptionalHeader* and *Characteristics*. *Machine* determines which type of system, 32- or 64-bit, the portable executable is built for. The attribute *TimeDateStamp*, on the other hand, defines the time and date when the author compiled the file. Essential information contains the attribute *NumberOfSections*, which describes how many section headers and bodies are in the portable executable file.

The subsequent header in the portable executable file format is the optional header. Unlike the name suggests, this header contains crucial information about the executable file. For example, the field *AddressOfEntryPoint* defines the application's instructions after being loaded into the memory. In this case, it is the entry point for the program. Other attributes like *SizeOfImage* determine the size of the image, including all headers that are loaded into memory. The final attribute of the optional header is *NumberOfRvaAndSizes*. This attribute defines the number of data directories in the optional header and the size.

The section headers describe the data sections. A section can be mapped into memory when the portable executable file is loaded. However, some sections are not loaded directly but contain instructions on how an application should load the data into memory. The attribute *VirtualSize*, for example, defines the size of the section when loaded into memory.

The *.text* section contains information about the entry address and gets mapped into memory after the application has been executed.

The subsequent section, data sections, contains sections like *.rdata*, *.data* and *.idata*. These store media information such as mouse cursor images. *.idata* describe the import directory and address table. The table illustrates the library calls and functions of the application. Other data sections like *.rdata* contain strings and debug directory information.

.rsrc includes the resources and the structure information of the executable. The link between

old and new entries defines the network of entries and describes the application's functionality.

The last section, *.reloc*, defines how the binary code is relocated to a new memory location. The relocation is accomplished by adjusting the offset of the memory address.

The following figure 2.2 shows the complete structure of the portable executable file. A full description for each feature is explained on the official Microsoft website [29] or on the documentation from Plachy [28].

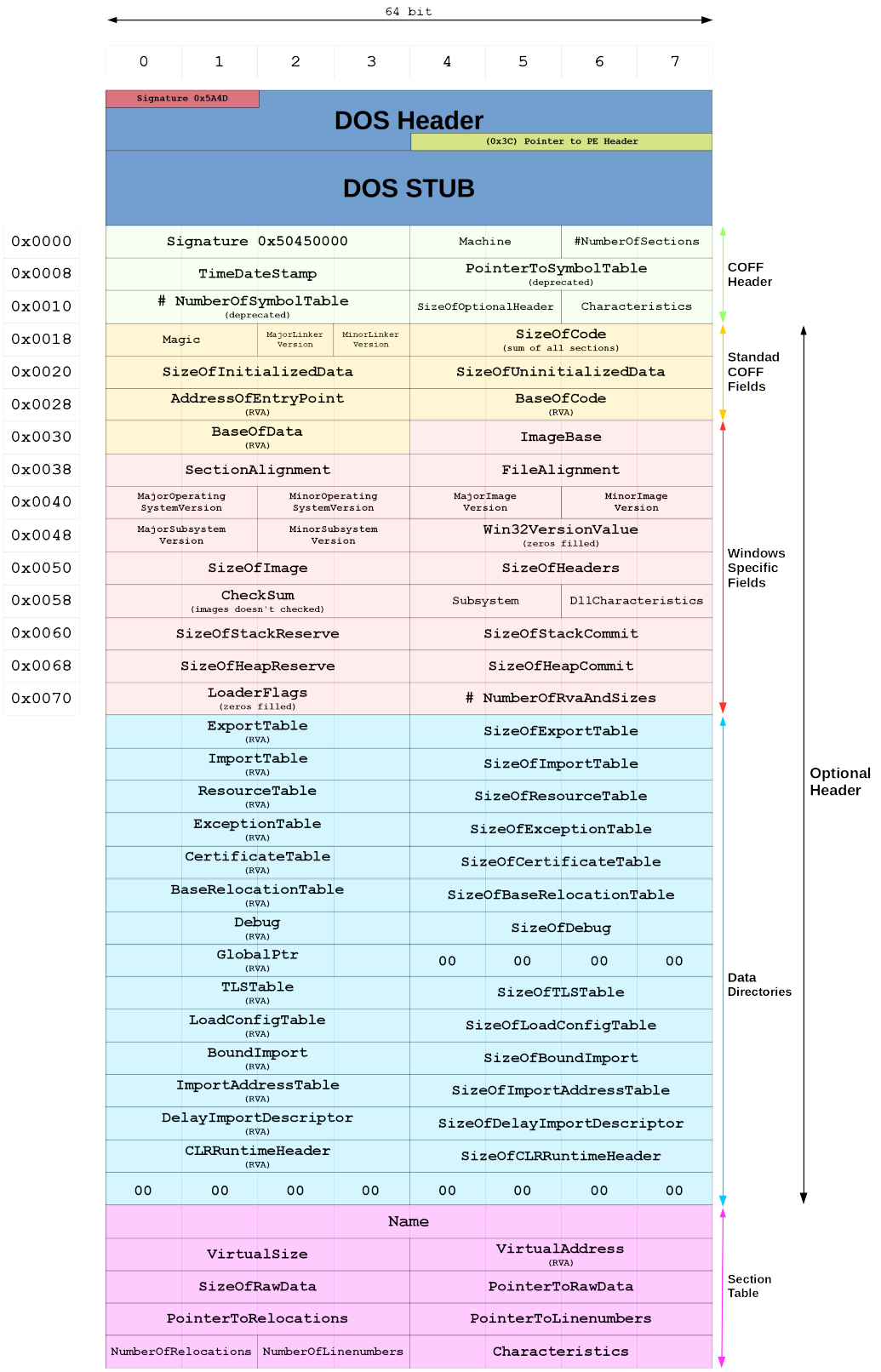


Figure 2.2.: Complete structure of a portable executable file [30]

2.2. Dimensionality Reduction

When working with machine learning algorithms, the performance is dependant on the properties of the dataset. A dataset with a high number of features requires a lengthy training process and might negatively affect classification performance. This phenomenon of high-dimensional data is called *curse of dimensionality* [31].

The problem of the *curse of dimensionality* can be bypassed by adjusting the dimensionality [32]. There are two principal approaches on how to deal with high-dimensional data.

- **Transformation of features**
  The first technique considers the transformation of features based on specified algorithms. The selection of features for the transformation depends on statistical results.

- **Selection of features**
  The selection of features method consists of the exclusion of not relevant features. The exclusion depends on observations and prior knowledge about the properties of the features. This technique does not change the characteristics of the features.

Dimensionality reduction can be advantageous when dealing with noisy data. The noisy data is then filtered and can positively influence the training process. Furthermore, the reduction affects overfitting, with a reduced dataset having a smaller chance to overfit. However, the reduction of dimensionality has some shortcomings that influence the classification results. The reduction process results similar to the lossy compression procedure with a JPEG image in a loss of information. Additionally, the framework structure is complex, and adjustments are more elaborate.

The subsequent section features two dimensionality reduction algorithms utilized in numerous data preprocessing steps.

2.2.1. High Correlation Filter

The first dimensionality reduction algorithm, the high correlation filter, utilizes the concept of feature selection. Filter methods work with statistical tools to rank features based on statistical measurements. The user then appoints a threshold, splitting the features into two subsets. The features which pass the threshold are then utilized for the future training process.

The high correlation filter computes the correlation between the input variables and identifies which features are similar. Features are highly correlated if they contain the same or comparable information. A threshold then decides if the variable is dropped or will be maintained. This threshold can be adjusted manually. Finding the optimal threshold can be obtained by observing the correlation matrix, hence the relations of the input variables. A correlation matrix is a table illustrating the correlation between the input variables. Therefore, a correlation matrix is usually computed before applying a high correlation filter.

2.2.2. PCA

A prevalent method to reduce dimensionality is PCA. This reduction algorithm is ranked in the transformation of features category. The goal of PCA is to create a new coordinate system where the features are depicted. Furthermore, PCA utilizes components that are not highly correlated. The depicted features on the new coordinate system are then called *principal components*. The selection of principle components is illustrated in figure 2.3.

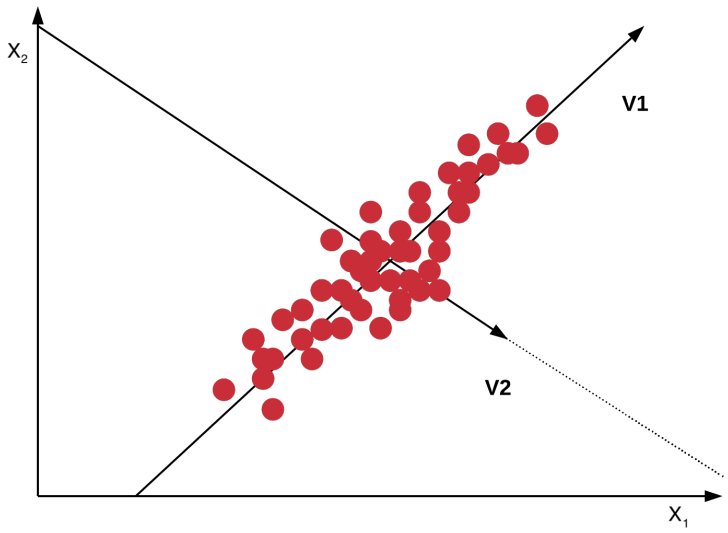


Figure 2.3.: Principal components

The first vector, *v1*, shows the first principal component in the two-dimensional space. The first principal component accounts for the most significant possible variance in the dataset [33]. The following principal component of the two-dimensional space is the vector *v2* that describes the second highest variance. One methodology of PCA is that the variance of all variables explained is as high as possible. This process is achieved by searching for the axis where the highest variance of the data sample lies. This axis is then a principal component of the data sample. The procedure is then repeated as often as the dataset contains dimensions.

2.3. Machine Learning

Machine learning is based on artificial intelligence. Algorithms utilizing machine learning learn by recognizing patterns and improve automatically by experience. The machine learning algorithm receives as input training data that contains multiple samples. After the training process, the machine learning algorithm can make predictions on a test dataset. In principle, it can be distinguished between four types of machine learning systems:

- **Supervised Learning**
  Supervised machine learning systems utilize labels in their training data to learn and predict a possible solution. These labels symbolize the target value that the algorithm is trying to predict [34]. Supervised machine learning systems are commonly used for classification but can also be utilized for regression. The advantages of supervised learning comprise predictions and the simplistic learning process. The most significant supervised machine learning algorithms are KNN, SVM, logistic regression, decision trees, and neural networks. A classic area of application for supervised learning is the detection of spam e-mails. The classifier learns with sample e-mails and classifies new e-mails either as spam or not.

- **Unsupervised Learning**
  The unsupervised machine learning system utilizes training data samples that are not labeled. The unsupervised algorithms try to recognize patterns and coherences exploratory and without guidance [35]. Unsupervised learning has some advantages in comparison to supervised learning. For example, unsupervised learning can be utilized in real-time and enables the discovery of new features for classification. Additionally, the input dataset requires less preprocessing; hence, acquiring unsupervised data is more uncomplicated. Practical unsupervised machine learning algorithms can be categorized into four categories: clustering, anomaly detection, dimensionality reduction, and association. K-Means and DB-Scan are algorithms located in the clustering category. Anomaly detection algorithms are one-class SVM and isolation forest. Algorithms for dimensionality reduction comprise, for example, PCA and locally-linear embedding. The last category features apriori and eclat utilized for association analysis. Applications of unsupervised learning can be found in the area of defense of network intrusion. Anomaly detection in intrusion detection systems, for example, detects anomalies in the traffic behavior of a network.

- **Semi-Supervised Learning**
  As the name suggests, semi-supervised learning is a hybrid machine learning system. The basic concept is the utilization of partially labeled data as input. Usually, the portion between labeled and unlabeled data is unevenly distributed, with labeled data representing the minority. Semi-supervised machine learning algorithms combine supervised and unsupervised techniques, solving, for example, the problem of identifying handwritten letters [36]. With the utilization of clustering methods, the letters are then segmented. The segments are then labeled and trained with a supervised machine learning algorithm.

- **Reinforcement Learning**
  Reinforcement learning utilizes a learning approach that is very different from the other systems explained above. The learning system is defined as an agent which interacts with the environment. The environment is a simulation of a scenario that gives feedback to

the agent. Beforehand, the agent observes the environment and takes action. The action is then either rewarded or punished. Through this process, the agent learns a strategy, which is defined as policy. The goal of the learning system is to find an optimal policy, which is refined with each action repetition of the agent. An application of reinforcement learning can be found in autonomous driving. The AWS DeepRacer, for example, is a self-driving car that utilizes cameras and reinforcement learning to guide itself on the runway [37].

2.3.1. Random Forest

Random forest is a supervised machine learning algorithm based on decision trees. The random forest algorithm utilizes a bagging method to train, which is a technique that combines individual predictions of the decision trees. These predictions are then passed on as weights into the classification result [38].

As the name suggests, decision trees consist out of a tree-like structure. The tree-like system has various components: the root node, the decision node, and the leaf node. The structure of a decision tree is illustrated in figure 2.4.

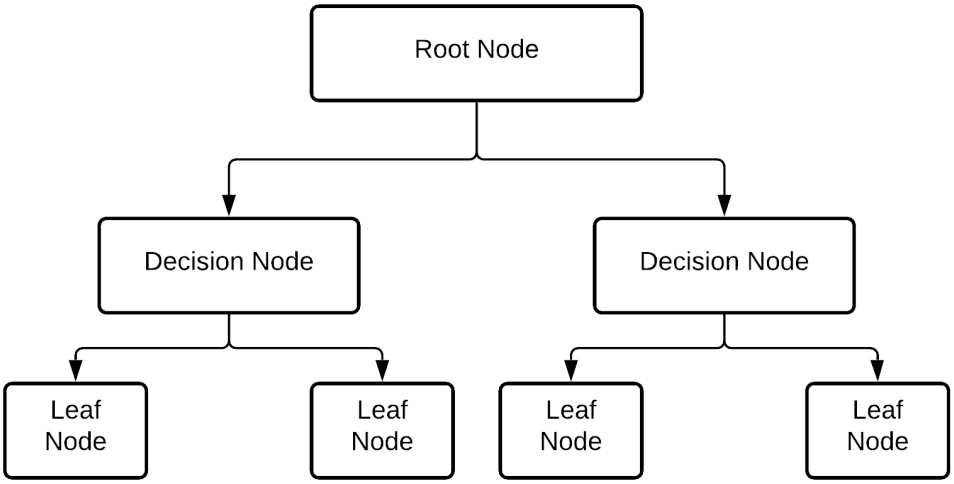


Figure 2.4.: Structure of a Decision Tree

Figure 2.4 shows that the root node is connected to the decision nodes, which are connected to the leaf nodes. The leaf nodes mark the lowest level of the tree structure and cannot be segregated. If a dataset is trained with a decision tree, the data is segregated and split up into branches. This process is repeated for so long as a leaf node is attained.

The random forest algorithm contains multiple decision trees, and the splitting process is executed randomly. In the next phase, the decision nodes determine the best outcome. There-

fore, the final output depends on the individual output of the decision trees in a majority-voting system. The root of the features computes the number of features selected for random forest. For example, considering the dataset has 16 features, *sqrt(16)* are chosen randomly. Then those four features are selected for the splitting process.

Random forest has many advantages compared to other classification algorithms. One advantage is the fast computation time of the training process. The fast training process is possible due to the parallelizability of the construction of decision trees. Another advantage is the functionality of random forest with large amounts of data and that it performs well with missing values [39]. On the other hand, random forest is very complex, especially in comparison with decision trees. Furthermore, the results of random forest are hard to interpret.

2.3.2. KNN

The KNN algorithm is a supervised machine learning algorithm that can be utilized for classification problems. The classifier is based on the idea that new data based on existing data can be classified. KNN observes the nearest neighbor of a data point [40]. It can then determine to which class it belongs. The criteria for choosing the suitable class is dependant on the distance between the data point and the nearest neighbor. If both neighboring classes have equal weight to the observed data point, the point is assigned to a random class.

The difficulty of implementing KNN is selecting the optimal value for K. One method is to choose different values for K. The dataset is then split up into training and test dataset. KNN is then computed for different K values, and the difference between the classification results of the test dataset is the number of errors encountered. The classification result that minimizes the error includes the optimal value for K.

KNN has several advantages, such as the low complexity of the implementation. Furthermore, KNN is very efficient and fast when dealing with small and low-dimensional datasets. However, when dealing with high dimensionality, KNN can have issues since it can be challenging to compute the distance to the nearest neighbor. Therefore, the training time can be unpredictable with high-dimensional data.

2.3.3. SVM

SVM has different areas of application. The two common approaches of SVM are linear and nonlinear classification. Furthermore, the application of SVM is optimal for small and medium datasets. SVM is a supervised machine learning algorithm similar to the algorithms explained above. The idea of SVM is to compute a hyperplane that separates the classes [41]. The criterion for positioning the hyperplane is based on the distance of the area between both classes. An optimal hyperplane lies there where the classes are separated the most. Figure 2.5 shows a hyperplane in a two-dimensional space between two classes.

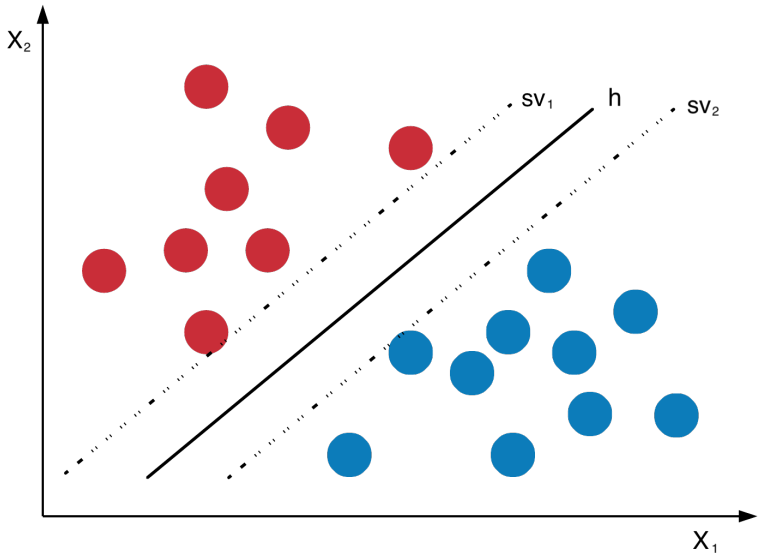


Figure 2.5.: Linear SVM

The figure shows two vectors beside the hyperplane *h*, called support vectors sv_1 and sv_2. The distance between the support vectors and the hyperplane is defined as margin. The figure illustrates that data points on the right side of the hyperplane are classified to the class blue, whereas points on the left side are classified as red. The higher the margin between the class and the hyperplane, the more accurate the prediction is. SVM, therefore, tries to achieve the maximum margin. The application of SVM in figure 2.5 describes a linear approach. For most classification problems, SVM has to solve a nonlinear problem.

A solution for dealing with nonlinear datasets is the utilization of polynomial features. The technique for this approach is called *Kernel-Trick*. This method aims to transform the low dimensional vector space into a higher-dimensional space. The data points are better separatable in this high-dimensional space, and finding an optimal hyperplane is more superficial. In summary, the nonlinear problem is transformed into a linear problem by changing dimensions.

The advantages of SVM are that SVM is very efficient when dealing with high-dimensional data and datasets with many features. Furthermore, SVM is very storage efficient. One disadvantage is the high training time when dealing with large datasets. Additionally, SVM in default settings provides no predictions.,

2.4. Neural Networks

Like the machine learning algorithms mentioned above, neural networks can learn from data and recognize patterns. Commonly, when dealing with complex tasks, neural networks can be used. The high flexibility of artificial neural networks (ANN) makes them beneficial in numerous fields of machine learning. The structure of an ANN is often in an analogy to the brain. A neuron

is a unit in a layer that receives weighted input signals and produces an output signal with an activation function. An artificial neuron resembles the behavior of a biological neuron which receives input from other neurons and performs processing, and passes it on to the output [31]. An ANN consists out of layers that have various tasks. Every ANN has at least an input and an output node. An ANN only composed of a single layer is described as a single-layer perceptron. An ANN has the following network structure:

- **Input layer**
  The first layer - the input layer receives the raw input of the data and passes it on to the next layer. The number of neurons of the input layer is dependant on the input variables of the dataset.

- **Hidden layer**
  Depending on the complexity of the ANN, an ANN can have multiple hidden layers. The hidden layer receives the data from the input or another hidden layer and applies weights to the inputs. Subsequently, the data is passed through an activation function, handing the result to the output layer [42]. The transformation of the data with the activation function is nonlinear.

- **Output layer**
  The output layer receives the output of the hidden layers and transforms the data into a format that solves the problem and makes a prediction.

The transformation of the data is dependant on the activation function, which has a significant impact on the performance of the neural network. Additionally, different activation functions can be chosen for each layer. Typically, hidden layers have the same activation function, while the output layer utilizes another activation function. The commonly used activation functions for hidden layers are:

- **Rectified Linear Activation (ReLU)**
  The ReLU is a nonlinear function that transforms input values. Input values greater than zero are returned directly, whereas values below zero are returned as zero. The activation function is, therefore, a piecewise linear function. The simplicity of the implementation of the ReLU makes the activation function very advantageous compared to other functions.

- **Logistic (Sigmoid)**
  The sigmoid activation function transforms the input value into a range between zero and one. Values below zero are returned as zero, and values above one are returned as one. The shape of the sigmoid function has an S-form and is likewise a nonlinear function.

- **Hyperbolic Tangent (Tanh)**
  The Tanh activation function behaves similarly to the sigmoid function. Instead of transforming values into an interval between zero and one, tanh transforms values into an interval between minus one and one.

The most common activation functions for the output layer are linear, sigmoid, and softmax. As its name suggests, a linear function returns the input value directly. In other words, it is described as *no activation*. The softmax activation function outputs one value for each neuron, then summed up to one, representing the probability, respectively [43].

The decision to choose the proper activation function depends on the prediction problem. The sigmoid activation function has proven very beneficial in binary classification.

2.4.1. Multilayer Perceptron and Deep Neural Network

The MLP is a neural network consisting of input, one or more hidden, and output layers. It belongs to the category of feedforward algorithms. All layers except the output layers are connected to the next layer and contain a bias neuron. Additionally, the mapping between the input and output layer is nonlinear [44]. The activation function of the multilayer perceptron can be any activation function explained above. Figure 2.6 shows the architecture of an MLP.

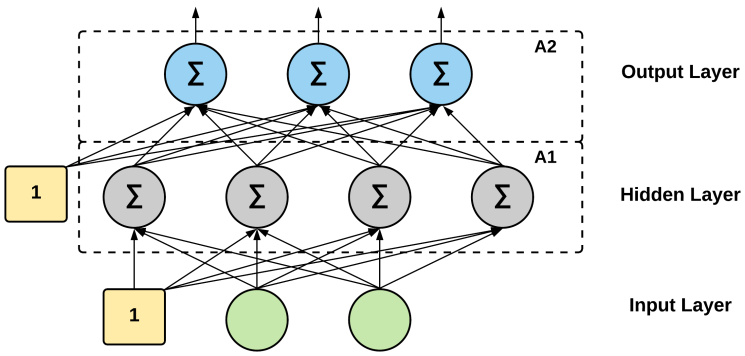


Figure 2.6.: The architecture of an MLP

The yellow nodes of figure 2.6 symbolize the bias neurons of the MLP. Furthermore, the activation functions are labeled as *A1* and *A2*. The input layer contains the initial weights computed to a weighted sum. These are then passed on to the activation function. In summary, the linear combination is transmitted to the next layer. For the MLP to learn, the networks' weights have to be adapted. This so-called gradient descent technique computes the gradient after each iteration. The process of learning in an MLP is defined as backpropagation to minimize a cost function. The error gradient can be calculated for each model parameter with each iteration [31]. In other words, the mean squared error of the input and output are computed. The weights and bias are then adapted with the value of the gradient to reduce the

error. This step is repeated until the neural network has approached a solution, meaning the computed gradient has passed a convergence threshold.

The MLP is a deep learning algorithm and can be categorized as a subset of a deep neural network. The MLP is beneficial for solving classification problems concerning supervised learning. To summarize it up, the MLP is a feedforward algorithm and a finite acyclic graph. Despite this, different deep learning architectures differentiate from the traditional MLP. The convolutional neural network (CNN) is a DNN commonly used in computer vision. The main difference between CNN and the MLP is the local connectivity [45]. CNN has smaller weights and is easier to train than MLPs. Additionally, the connection between the layers is sparse instead of fully connected.

2.5. Hyperparameter Optimization

Depending on the dataset, machine learning models have specified hyperparameters that influence the learning process [46]. The configuration of hyperparameters for known classification problems is often straightforward. However, in the opposite case, selecting optimal hyperparameters is challenging. The optimization of hyperparameters ranges from the configuration of the model (number of neighbors in KNN) to the selection of the activation function (ReLU, Sigmoid) [47]. Scikit-learn offers predefined values for the hyperparameters that are based on an average composition of a machine learning model. Changing these values can have a significant effect on the model's performance.

The main goal of hyperparameter optimization is to achieve the best results of a model with the given dataset. Past research and experiments can be very decisive in determining which hyperparameter is relevant for the tuning process. Often the optimization of a hyperparameter has no significant effect on performance and can be left out. After selecting the hyperparameters for optimization, a search space must be specified. This space consists of dimensions for which hyperparameters can attain an integer- or a category value. The hyperparameter optimization algorithms with search space can be categorized into two classes.

- **Grid search**
  Grid search defines a finite set of values of each hyperparameter and computes the cartesian product. Grid search is easy to implement; however, it might never reach the desired optimum.

- **Random search**
  Random search defines a distribution over the input space and ends after a specific criterion has been reached. In most cases, this is a time criterion defined in trials.

Bayesian optimization is a more advanced algorithm commonly used for well-performing machine learning models. Bayes optimization is based on the Bayes theorem, finding the

extrema of an objective function [48]. The first step of Bayesian optimization consists of selecting a sample by optimizing the acquisition function. The acquisition function decides which sample is next in the query, depending on the *probability of improvement* [49]. The next phase is the evaluation of the sample with the objective function. The sample is then converted into a surrogate function, which is a Bayesian approximation of the objective function. These steps are then repeated, optimizing the acquisition functions.

2.5.1. Keras Tuner

Keras Tuner is a library for hyperparameter optimization that the Keras team released at the end of 2019 [21]. The goal of the Keras team was to create a library that can efficiently perform hyperparameter tuning and is applicable for different machine learning implementations.

The architecture of the Keras Tuner is structured as follows. First, the hyperparameters are selected that are going to be tuned. The machine learning model is trained and evaluated based on a validation set [50]. The best evaluation results concerning validation accuracy are the optimal hyperparameters and are then tested on the test set. The first three steps, defining the combination of different hyperparameter settings, training the model with these settings, and evaluating the model, are repeated until the max trial is achieved. The user defines the max trials, depending on how many hyperparameters are selected and the search space.

Before tuning hyperparameters, it has to be decided which hyperparameters are optimal for tuning. Then a search space consisting of the to-be-tuned hyperparameters and the tuning process's range must be defined. The following syntax is possible to tune potential hyperparameters.

- **Boolean**
  Defines a choice between true and false. First, the parameter's name is appointed, then both choices are listed. For example, to tune the hyperparameter of the probability of SVM, the syntax has the following input:
  probability=hp.Boolean('probability', True, False).

- **Choice**
  The choice method chooses one value out of a predefined set of values. The hyperparameter name to-be-tuned is listed first, then the set of values is defined. The tuner then selects values from the set and evaluates the optimal value. If different activation functions, such as tanh and relu, for MLP are to be tested, this can be defined in the Keras Tuner by the following code:
  activation=hp.Choice('activation', ('tanh', 'relu')).

- **Float**
  The float method defines a range of values to be considered for tuning. Unlike the int method, the float method can obtain decimal numbers and is more precise. Additionally, the float method contains a step argument that defines the distance between two values. For example, if the hyperparameter *gamma* of SVM is tuned from 0.01 to 1 in steps of 0.01. This example is defined by the following code:
  gamma=hp.Float('gamma', 0.01, 1, step=0.01).

- **Int**
  The int method has a similar structure to the float method. The main difference is that it only operates with ints and that the default value for the step attribute has the value one. For example, if the number of units in a DNN are tuned:
  units=hp.Int('units', 64, 128, 16, default=16)

In the section above, possible hyperparameter tuning methods were discussed. Keras Tuner supports different tuning methods, including bayesian optimization. This has to be set in the tuner search method to define Keras Tuner utilizing Bayesian optimization. For example, the following code snippet shows the setting: *tuner_hb = kt.BayesianOptimization(...)*.

Keras Tuner is applicable for various machine learning models that are not implemented with the Keras API. For example, scikit-learn models can be tuned with a built-in tuner optimized for scikit-learn. Keras Tuner with scikit-learn, however, partially works due to various issues and bugs that still need to be addressed. These issues are then managed in the Keras Tuner management project with diverse contributors.

3. Related Work based on Malware Detection

Recent publications based on malware detection and feature extraction are summarized and presented in the following section. Additionally, related work on android devices is demonstrated since the approaches are similar in principle. This chapter aims to illustrate current state-of-the-art techniques and proceedings on different approaches.

3.1. Malware Detection based on Portable Executables

Raff et al. [51] introduced a technique to detect malware with a one-dimensional convolutional network. Utilizing a static analysis approach, the portable executable is viewed and extracted as a file with raw bytes. The dataset obtained from Virusshare is divided into two groups and then used as input for training the one-dimensional convolutional neural network. With this input and a sigmoid activation function, the model's performance achieves an accuracy of 90%. The paper compares previous malware detection publications with the obtained results, for example, an approach that gained features with n-grams.

Souri et al. [52] presented a survey showing malware detection mechanisms using data mining techniques. The illustrated approach divided the methods into two categories: (1) signature-based methods and (2) behavior-based methods. In the publication, Souri et al. compare different malware detection approaches based on evaluation and proficiency. Although the paper does not include current state-of-the-art models, the survey gives insight into the feature taxonomy used in standard machine learning algorithms.

A survey of machine learning techniques for malware analysis by Ucci et al. [53] categories existing methods along three dimensions: (1) what is the goal of the analyzed paper, (2) which feature types are extracted for machine learning approaches, and (3) which technique is used to detect malware. Similar to Souri et al., the survey does not consider current publications and multimodal approaches. Although this paper does not feature novel malware detection techniques, the approach demonstrated valuable topical directions for feature extraction algorithms.

Zhong et al. [54] proposes a multi-level deep learning system for malware detection. The presented multi-level deep learning model is trained on a unique data distribution utilizing a tree structure. Due to the organizational structure, the models cooperate, delivering a reasonable solution. The presented technique yields reliable results, outperforming many malware detection

approaches.

Vinayakumar et al. [55] analyzes traditional machine learning algorithms and deep learning methods for malware detection. The classification and evaluation process is done with private and public datasets. The publication illustrates an image processing technique for a zero-day malware detection model subsequently. The detection model, called ScaleModelNet, can handle extensive data, making it scalable in real-time. Through deep learning, the malware is detected and then, in the second phase, classified by its corresponding category.

Comparing machine learning and deep learning techniques for malware detection is rarely done in recent publications. One approach is presented by Rathore et al. [56]. The paper describes malware analysis as a machine learning and deep learning problem. The report demonstrates the utilization of Variance Threshold and Auto-Encoders to reduce the dimensionality of the features in the dataset. The deep learning models consist of a multilayer approach and are compared with random forest. In conclusion, random forest outperforms deep neural networks with an accuracy of 99.78%. Nonetheless, Reather et al. append that the dataset is not optimal for deep learning, and the deep learning model results can occur due to overfitting.

Azeez et al. [57] deploy an ensemble of deep learning neural networks to detect and classify malware. The framework features neural networks in the first stage and various machine learning models in the final stage as a classifier. The dense ANN and One-Dimensional CNN models achieve the best results with this approach. Azeez et al. indicate that the framework is limited to supervised learning and that future work focuses on validating the framework by utilizing a larger dataset.

3.2. Feature Extraction and Selection

Raman [58] proposes a technique to select features for malware classification. In this approach, Raman uses data mining to identify critical features within the portable executable file format. Key features for malware classification are important since irrelevant features have adverse effects on machine learning approaches. The algorithm utilized six classifiers. One classifier, J48, is an algorithm used to create a decision tree, provides the best results with an accuracy of 98%. From the initial 645 features, the feature selection algorithm can reduce the features to 13. The proposed feature selection algorithm can be used as input for future malware detection algorithms to increase the detection rate.

Jureček et al. [59] propose a distance metric learning (MDL) algorithm for multiclass classification. The features for the algorithm are extracted from the portable executable file format and reduced to 25. The best results achieved are with the random forest model and an average accuracy of 98.90%. This approach aims to improve multiclass classification performance by training the models with the Mahalanobis distance metric.

Kim et al. [60] propose a classification system to classify various malware types. The portable

executable file features are extracted based on an automatic feature selection and a combination of machine learning and deep learning. With various machine learning algorithms, consisting of Random Forest, Gradient Boosting, Decision Tree, and CNN, the experiment achieved an accuracy of approximately 98%. The novel method provides an intuitive approach to static analysis to detect malicious code. At the same time, the file itself is visualized and analyzed with a CNN model. The proposed technique is not bound to the portable executable file format and can classify different types of malware from shellcode.

Alkasassbeh et al. [61] provide a feature selection algorithm similar to Raman et al.. Proposed in 2020, Alkasassbeh et al. illustrate a classification technique for feature selection within the portable executable file format using machine learning algorithms. The objective of the proposed feature selection method is to identify critical features to enhance the detection rate of malware detection algorithms. The feature selection algorithm receives an initial input of 13 portable executable features and turns the features into a minimal feature set. The machine learning algorithms implemented can minimize the feature set into seven key features utilized as input for a malware detection algorithm. The machine learning algorithms are IBK, Random Forest, J48, J48 Graft, Ridor, and PART. Compared to related research, the minimal feature set receives good results, with J48 Graft achieving an accuracy of 98.55%.

3.3. Malware Detection on Android devices

MLDroid is a malware detection framework for android devices presented by Mahindru [62]. Through dynamic analysis, the proposed framework detects malware from real-world apps. A traditional android app file has the format android package (APK), which generally is a ZIP-Archive file. The workflow initially starts with the collection of features from the APK. In the second phase, the class of the given file is identified, then features from the API calls are extracted. The features are subsequently trained with a least square support vector machine (LSSVM). After the training, the model is validated based on existing techniques. The proposed algorithm achieves a detection rate of 98.7% with deep learning algorithms, fastest first clustering, Y-MLP, and a nonlinear ensemble decision tree forest algorithm. Compared to other traditional frameworks, the algorithm achieves a 3% higher detection rate.

Wang et al. [63] present a framework to detect malware on android devices. In the proposed framework, the kernel task structures are observed to identify malware on the android device. The proposed detection framework features a weight-based detection (WBD) system, identifying and categorizing benign and malware applications. Additionally, the framework observes the memory and the external storage for malicious applications. The approach achieves accuracies up to 98%, detecting malicious applications with reduced-dimensional features.

Gao et al. [64] introduce a novel approach to detect and classify malware on android devices. Utilizing a Graph Convolutional Network (GCN), apps and android APIs are mapped into a

heterogeneous graph. The graph is then used as input for the GCN model, generating nodes. In conclusion, the fundamental idea is the conversion of the classification problem into a node classification task. The proposed framework achieved an accuracy of 98,99%, outperforming current state-of-the-art approaches.

3.4. Literature Analysis

The literature review shows that portable executable malware detection is increasingly addressed in recent related works. Most of the proposed methods are based on static analysis and barely perform dynamic analysis.

However, when performing static analysis on portable executable files, it is essential to utilize relatively up-to-date datasets, to make the algorithms applicable for real-time detectors. Using present data is difficult since only a few datasets are publicly available. Several approaches [56, 60] use datasets that contain samples from old operating systems. Hence results are challenging to transfer to current detection problems considering advanced obfuscation methods. Other published results [51] use private datasets for malware detection. These are, however, limited in size, and results are difficult to verify. The limited number of publicly available datasets complicates the comparison since the dataset's creation span varies. Therefore, most approaches [56, 57, 59, 60] use exclusively one dataset. However, most of these datasets are limited in sample size. Several approaches [58, 61] compute the feature importance mainly with one dataset, disregarding datasets with other features. Features that are contained in one dataset but not in the utilized dataset are therefore not analyzed. Hence research on the feature importance of portable executable malware detection is insufficient.

Besides the limitations of datasets, research focuses mainly on traditional machine learning [61] or deep learning [51, 54] methods. Implementing deep learning and standard machine learning algorithms in a framework [55, 56] is occasionally done. However, a profound analysis of the comparison between the classification results of the models is limited. Additionally, the optimization of hyperparameters for the detection models is scarcely discussed [55] in recent work. A method for optimally tuning the hyperparameters for the malware detection problem has not yet been adopted.

4. Structure of the Implementation

The following section illustrates the datasets, the workflow, and the architecture of the proposed framework. Additionally, the research questions are expressed, and the evaluation metrics are defined. Furthermore, the experimental setup is presented, namely the hardware specifications. The objective of this section is to demonstrate the structure of this malware recognition approach.

4.1. Datasets

In the first part of this section, the datasets are described. Both datasets contain portable executable files with labels that define a sample as benign or malware.

4.1.1. Kaggle Dataset

The Kaggle dataset [65] contains 19.611 samples with 79 features. The number of malicious and benign data can be acquired from figure 4.1.

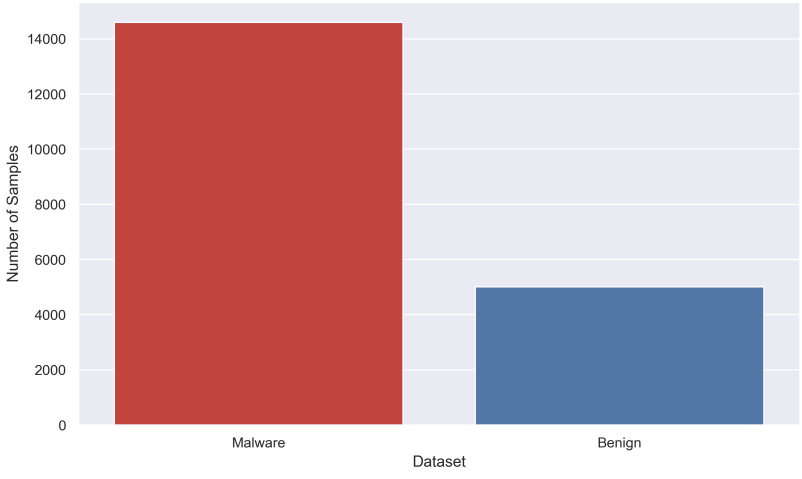


Figure 4.1.: Number of benign and malicious files in the Kaggle dataset. Malware samples are colored red. Benign samples are colored blue.

The plot shows a skewed class distribution ratio between malware and benign samples. The balance between benign and malware samples is 1:2.9. Malware samples are labeled malware if they have the numerical value one at the feature *malware*. The full list of features is illustrated in the appendix in table A.1.

4.1.2. Virusshare Dataset

The Virusshare dataset contains 138.047 samples of portable executable files obtained from Virusshare [66]. Compared to the Kaggle dataset, the Virusshare dataset is approximately seven times bigger. Additionally, the dataset contains 57 features, with 22 less features than the Kaggle dataset. Some of the 22 missing features are listed in table 4.1. A complete list of the 57 features is presented in the appendix in table B.1. Malware samples are labeled malware if they have the numerical value zero at the feature *legitimate*. The number of malware and benign samples can be obtained from figure 4.2.

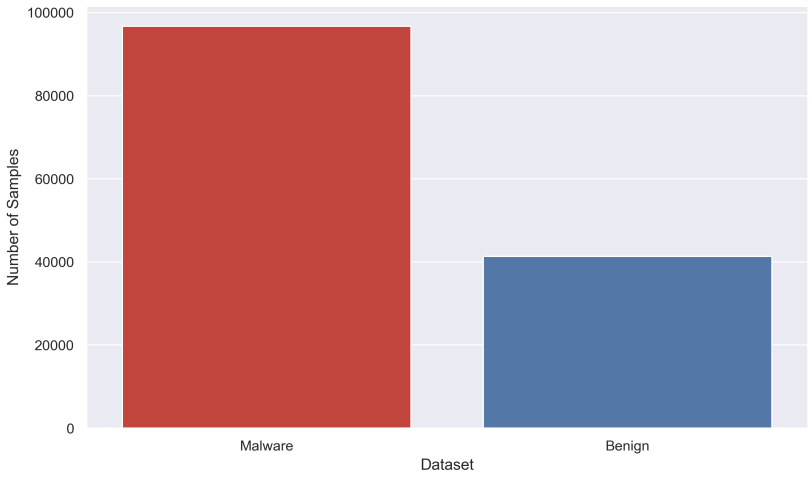


Figure 4.2.: Number of benign and malicious files in the Virusshare dataset.

The following plot shows a similar class distribution as the Kaggle dataset. Most samples belong to the malware class; the distribution ratio is 1:2.34. Past research has shown that an imbalanced dataset can sometimes lead to prediction issues for the trained model [67]. However, experiments showed that a balance of data in the preparation process was unnecessary.

Feature	Description
PointerToSymbolTable	Pointer to the symbol table or zero if no table exists
NumberOfSymbols	Number of symbols in the entry table
e_cp	Pages in the PE file
TimeDateStamp	Time when the file was created
e_minalloc	The minimum of extra paragraphs needed
e_magic	Magic number
e_cblp	Bytes in the last page of the file
NumberOfSections	Number of section headers and bodies in the file
e_cparhdr	The size of the header in the paragraph

Table 4.1.: Missing features of the Virusshare dataset [28] [29].

4.2. Framework

This section describes the structure of the framework. First, the workflow of this approach is described, then the research questions are defined. Finally, the deep learning architecture is illustrated.

This machine learning-based malware detection framework employs a traditional detection framework divided into three phases: (1) Data preprocessing, (2) Model generation, and (3) Detection and Classification.

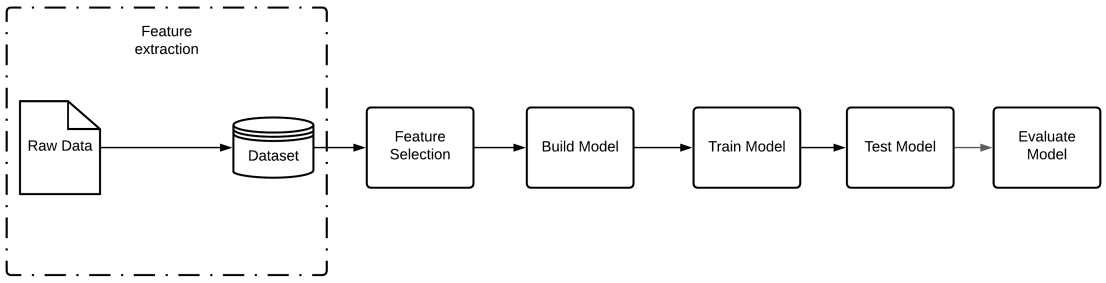


Figure 4.3.: Workflow of the malware detection system. Phases: (1) feature extraction, (2) feature selection, (3) model construction, (4) training the model, (5) testing model, and (6) evaluation

The workflow of this framework utilizes standard machine learning and neural network workflow models. Figure 4.3 demonstrates the workflow of the phases involved in the system methodology.

The initial phase consists of a feature extraction algorithm that obtains features from a portable executable file. A feature selection process decides which features are essential and prepares them for the artificial model. The model is then built and trained on the provided data. Afterward, the model can be tested and based on the provided results evaluated.

With the consideration of the presented workflow in figure 4.3, an in-depth overview of the proposed detection system is illustrated in figure 4.4.

The in-depth overview illustrates the initial state with a dataset as input, such as the Kaggle dataset presented 4.1. In this proposed framework, the features are extracted from the dataset and then reduced to a minimal feature set based on principle component analysis (PCA). For comparison reasons, the features of both datasets are reduced to the same subset size. The proposed models are constructed and then trained with the associated features. The models are then optimized with Keras Tuner to achieve the best possible results. Optimization is done by tuning the hyperparameters of the learning algorithms to detect the optimal parameters. Towards the end of this workflow, the outcome of each algorithm is evaluated and tested on unknown data. The best possible outcome defines the optimal model for a malware detection system. A detailed explanation of the architecture of the neural networks is explained in more detail in the following subsections. Additionally, the selected hyperparameters for optimization are described in the course of this section.

In summary, the framework receives two different datasets with similar features, reduced to a minimal subset with PCA. Additionally, the framework contains three machine learning algorithms and two artificial neural networks to train and detect malware. The trained models optimized with Keras Tuner are evaluated and sorted on performance. With the methodology framework presented in figure 4.4, the following research questions are illustrated and listed according to priority:

- **How do the implemented machine learning algorithms compare to the results from deep learning models?**
  In recent research publications, either machine learning or neural networks are used to detect and classify malware. A combination of both learning algorithms is rarely implemented and compared. The comparison concludes the evaluation of each model based on the evaluation metrics presented in the evaluation section of this thesis. Additionally, the classifiers are ranked based on their performance.

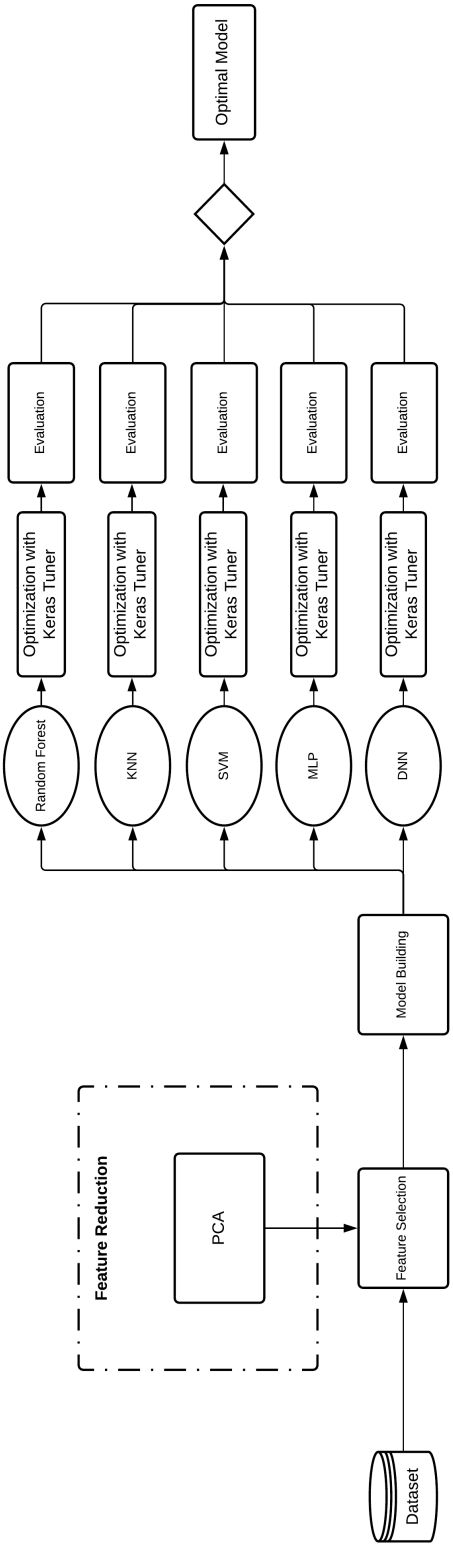


Figure 4.4.: In-depth overview of the malware detection approach

- **How do results compare, based on the input of the Virusshare dataset and the Kaggle dataset?**
  The Virusshare dataset has more samples but fewer features than the Kaggle dataset. Which dataset is superior for malware detection based on the given framework. The evaluation of classification results is decisive to answer this question. Additionally, the analysis based on the selected features for the models can be informative to determine why a result is superior. The research question presupposes a deeper analysis and understanding of features.

- **How efficient is the framework based on recent publications?**
  Recent publications achieved 98% accuracy with different machine learning techniques (Rathore et al. [56] and Azeez et al. [57]). The proposed framework in this thesis could achieve better results, particularly with Keras Tuner.

- **How effective is Keras Tuner with the proposed framework?**
  Keras Tuner can be very effective in optimizing machine learning algorithms. Which hyperparameters are optimal in the proposed framework, and which parameters are unfavorable for future classification processes? Classification results of classifiers are analyzed with and without Keras Tuner to compute the efficiency. The resulting evaluation results for each model are then utilized to determine the efficiency of Keras Tuner.

- **How advantageous is PCA in this framework?**
  PCA can be very favorable when dealing with high-dimensional data. Reducing features to a minimal subset can significantly influence performance, especially computation time. Classification results with and without PCA are compared and discussed to evaluate the performance of PCA with this framework.

4.3. Deep Learning Architecture

The following section presents a brief outline of the proposed deep learning architecture. Each overview shows a visual presentation and defines parameters for the classification process.

4.3.1. MLP

The first proposed deep learning model, MLP, contains two hidden layers. Figure 4.5 illustrates the architecture of an MLP and the approximate structure of the model in this framework.

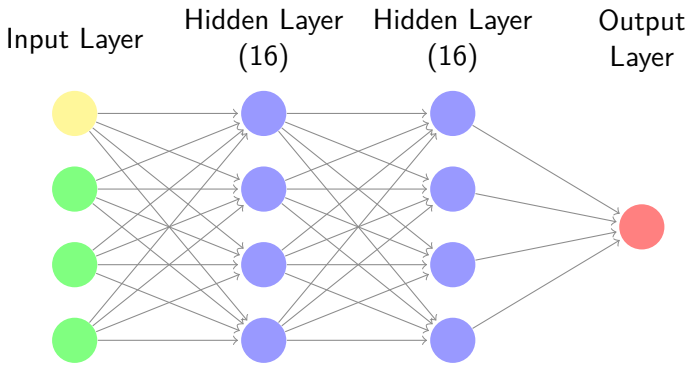


Figure 4.5.: Architecture of an MLP, created with a neural network tool from Cown [68]

The MLP in this figure is just a simple representation of the proposed model due to the visualization volume. The MLP presented in this thesis contains two hidden layers with 16 nodes each. The activation function for the hidden layers of the MLP is ReLU. Additionally, the solver for weight optimization in this MLP is adaptive moment estimation (ADAM). Finally, the output layer uses the logistic activation function.

4.3.2. DNN

The DNN, compared to the MLP approach presented above, has three hidden layers. The architecture for this model is demonstrated in figure 4.6.

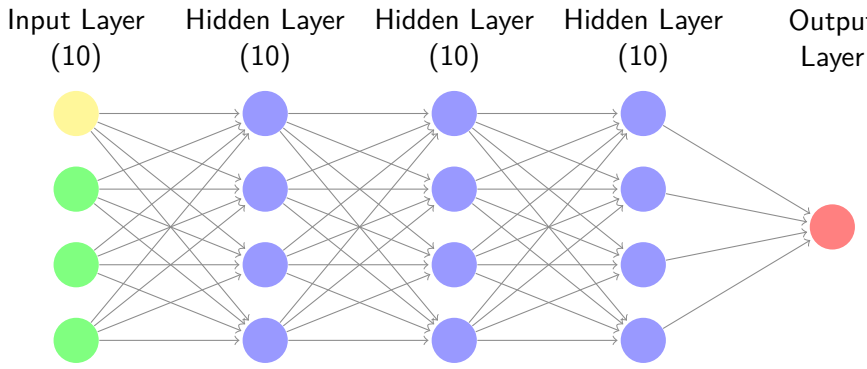


Figure 4.6.: Architecture of the proposed DNN.

This figure is a simplistic demonstration of the proposed model, similar to the representation above. The input node, first hidden layer, second hidden layer, and third hidden layer have ten nodes each. Additionally, all hidden layers utilize ReLU as an activation function. On the other hand, the output layer uses the sigmoid activation function. The model is optimized with ADAM, an efficient gradient descent algorithm. ADAM is advantageous because it automatically tunes itself and is well suited for problems with large amounts of data [69]. The loss function specified for this approach is binary cross-entropy.

4.4. Hyperparameter Optimization with Keras Tuner

In the following section, the hyperparameters for optimizing the framework are appointed. The selection of parameters is based on past research and experimental results. For each classifier, the associated parameter for optimization is listed and described in a table.

Random Forest: Parameters of Random Forest can define the number of decision trees in a forest or the criteria when a node is split. Scitlearn utilizes default parameters that can achieve good results but are possibly not optimal for the classification process with this framework. The selected parameters for optimizing Random Forest are based on Probst et al. [70] and Koehrsen [71]. The hyperparameters are displayed in table 4.2.

Parameter	Description	Optimization	Default Value
n_estimators	Determines the number of trees in a forest	100-180 (Step=10)	100
max_features	Number of features that are considered for the best split	1-50 (Step=5)	sqrt(n_features)

Table 4.2.: Optimized parameters for Random Forest

KNN: The selection of parameters for tuning KNN is based on the article from Bhandari [72] and Inan [73]. Additionally, own experiments determined which parameters are utilized in Keras Tuner. KNN possesses parameters that appoint the size of neighbors or the size of the leaf. The hyperparameters chosen for the optimization are illustrated in table 4.3.

Parameter	Description	Optimization	Default Value
n_neighbors	Determines the number of neighbors that are considered	1-20 (Step=2)	5
weights	Appoints the weight function that is utilized for the prediction	uniform or distance	uniform
metric	Describes the distance to the nearest neighbor	minkowski, euclidean or manhattan	minkowski

Table 4.3.: Optimized parameters for KNN

SVM: The supervised machine learning algorithm SVM contains three significant parameters that are suitable for optimization. Optimizing the parameter *kernel* was not substantial and harmed performance throughout experiments. Therefore, this parameter is not considered for optimization. The fundamental selection of parameters derives from the work of Liu [74]. The selected parameters for optimization are displayed in table 4.4.

Parameter	Description	Optimization	Default Value
C	Regularization parameter that determines how much error is acceptable	1-20 (Step=2)	1
gamma	Appoints the influence of points to the line of seperation	0.1-1 (Step=0.1)	scale

Table 4.4.: Optimized parameters for SVM

MLP: The MLP classifier has various parameters that are suitable for optimization. Research has shown that *activation*, *hidden_lay_sizes* and *batch_size* are common parameters, utilized in most tuning frameworks. Experiments with these parameters have confirmed this assumption. Therefore the parameters presented in table 4.5 are selected for tuning.

Parameter	Description	Optimization	Default Value
hidden_layer_sizes	Determines the number of neurons in the hidden layer	10,100 (Step=10)	100
activation	Appoints the activation function for the hidden layer	tanh or relu	relu
batch_size	Describes the size of the minibatch for the optimizers	10-500 (Step=10)	min(200, n_samples)

Table 4.5.: Optimized parameters for MLP

DNN: The selection of parameters for tuning is based on the work of [75] and own experiments. For comparison reasons, the number of tuned parameters is minimal. Furthermore, the default values are based on the values defined in section 4.3.2. Table 4.6 shows the selected parameters for the optimization process with Keras Tuner.

Parameter	Description	Optimization	Default Value
units	Determines the dimensionality of the output space	16, 64, 4	10
activation	Appoints the activation function for the layer	relu, tanh or sigmoid	relu

Table 4.6.: Optimized parameters for DNN

4.5. Evaluation

The evaluation of this framework is divided into four parts.

1. **Prediction performance comparison of machine learning and deep learning models**
   The models of the proposed framework are analyzed based on the evaluation metrics mentioned below. The evaluation results of the models are then compared to determine which model performs superior.

2. **Analysis of the datasets**
   The portable executable features of the datasets are analyzed and compared. The importance of features is determined by the framework's accuracy, recent work in the research field, behavior and distribution of the features, and possible anomalies.

3. **Evaluation of the Keras Tuner**
   Keras Tuner is applied and measured for each model based on the evaluation metrics. The evaluation results with and without Keras Tuner are then utilized to provide a comparison.

4. **Evaluation of PCA**
   The performance of the models with PCA and without PCA are compared. The results indicate which models are not optimal with the utilization of PCA.

4.5.1. Evaluation Metrics

The following measures and metrics for comparing and measuring the proposed models will be assessed:

Accuracy: The Accuracy describes the fraction of correct predictions to the total number of input samples [76]. The accuracy of a classifier can be calculated with equation 4.1.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.1}$$

Precision: The Precision describes the fraction of correct positive predictions to the total number of correctly positive samples. The precision of a classifier can be calculated with equation 4.2.

$$Precision = \frac{TP}{TP + FP} \tag{4.2}$$

Recall: The Recall describes the fraction of correct positive predictions to the total number of correct samples and false negatives. The recall of a classifier can be calculated with equation 4.3.

$$Recall = \frac{TP}{TP + FN} \tag{4.3}$$

F1 Score: The F1 Score describes the ratio between precision and recall. The lowest value is zero, and the highest value is one, which at the same time is a perfect classification. The F1 Score of a classifier can be calculated with equation 4.4.

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{4.4}$$

ROC: The receiver operating characteristic curve (ROC) shows the performance of a classifier based on recall and false positive rate (FPR) [77]. The false positive rate is defined in equation 4.5.

$$FPR = \frac{FP}{FP + TN} \tag{4.5}$$

The curve then plots the recall and the FPR at different classification thresholds.

AUC: The area under the ROC curve (AUC) measures the complete data underneath the ROC curve. Figure 4.7 shows the AUC and the ROC Curve.

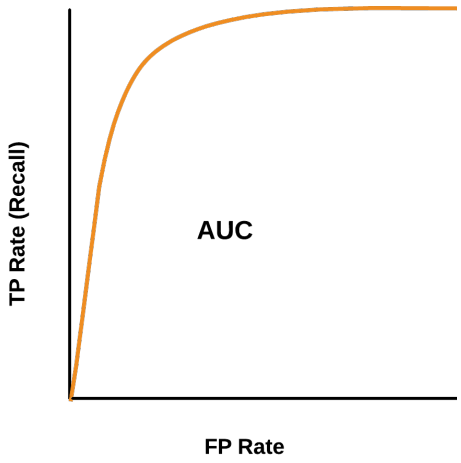


Figure 4.7.: AUC and ROC curve

The curve then plots the recall and the FPR at different classification thresholds.

Confusion Matrix: The Confusion Matrix shows the complete performance of the model. The Confusion Matrix is illustrated in 4.8

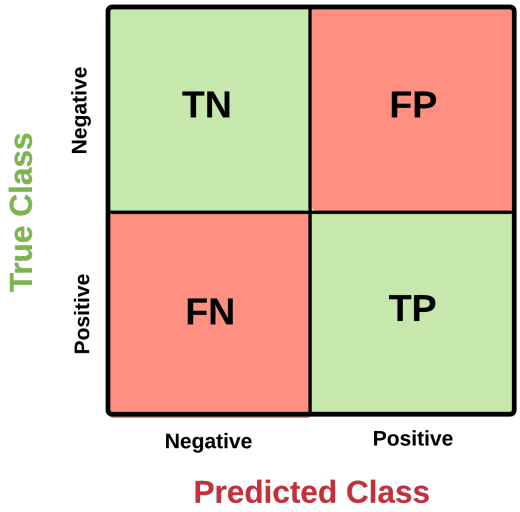


Figure 4.8.: Confusion matrix

Execution Time: The execution time of the classifiers is measured with the Python module *time*.

4.6. Experimental Setup

The implementation of the proposed framework requires high computational resources. The experiments were conducted in a web-based environment in JupyterLab. Further specifications for the machine utilized in the implementation are shown in Table 4.7.

Element	Value
CPU	Intel(R) Core(TM) i7-9700K CPU @ 3.60GHz
RAM	16,0 GB
OS	Microsoft Windows 10 Education
GPU	NVIDIA GeForce RTX 2080 SUPER
Drive	Samsung SSD 500 GB

Table 4.7.: Hardware specifications

5. Classification Results of the Framework

This chapter presents the results of the introduced framework from section 4. Additionally, the statistical analysis conducted on the datasets is featured and represented at the beginning of each dataset chapter. The implementation results are displayed for each dataset individually, while the comparison is highlighted in the discussion section. The classifiers are evaluated based on the evaluation metrics illustrated in section 4. The presentation of the results is displayed in the following structure. First, the model results with PCA are displayed. Then the classification results with Keras Tuner are illustrated, characterized by the symbol *KT*. Finally, the last presentation of results covers the model's outcomes without the integration of PCA. The results for this process are marked in the table with the description *WPCA*. The results in the table are the macro average of the classification results. Subsequently, the confusion matrices of the classification results are presented. The confusion matrices are then evaluated based on the classification results. However, the evaluation of the classification results, illustrated in the tables, will be discussed in the next chapter.

5.1. Kaggle Dataset

The dataset contains 75 features that were previously listed in the Methodology section. The ROC and AUC of the classification results are illustrated in the appendix in figure C.1. With the utilization of the pandas.DataFrame library the data was imported and transformed into a data frame, providing an outline of the given samples.

Afterward, features with no statistical meaning, for example, *Name* and *Malware*, were filtered out. Consequently, the dataset only contained numerical values with whom the correlation matrix was plotted.

The correlation matrix was plotted using the seaborn library and is presented in figure 5.1. The matrix shows the relationship of all features to each other based on the color. The values range between -0,6 and 0,6. Positive values are interpreted as positive correlations and displayed as red. Values with a blue color, negative respectively, are interpreted as negative correlations. Values that are close to 0 indicate that there is no correlation between these features. The correlation matrix will be analyzed in more detail in the significance of features section 6.2.1.

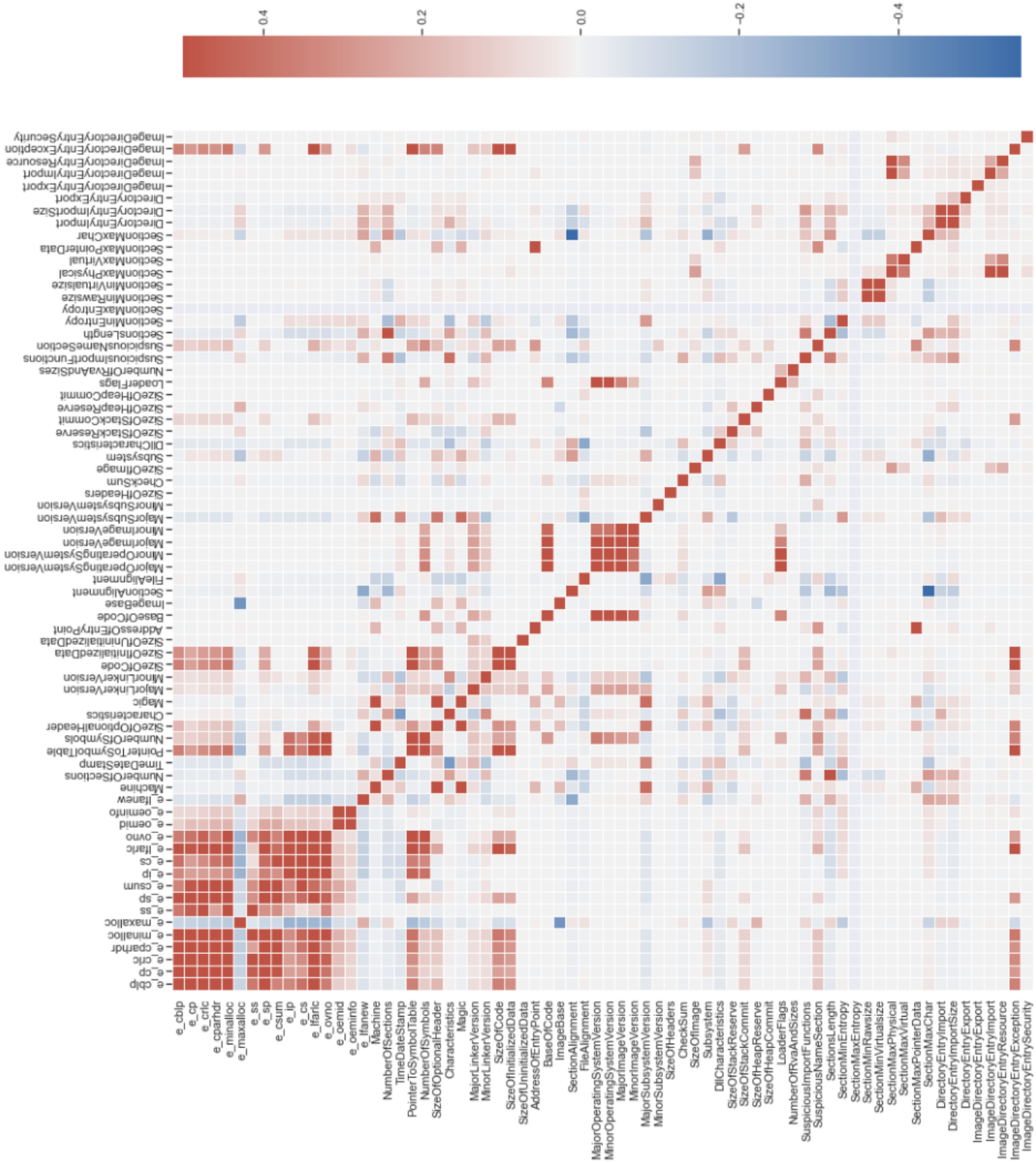
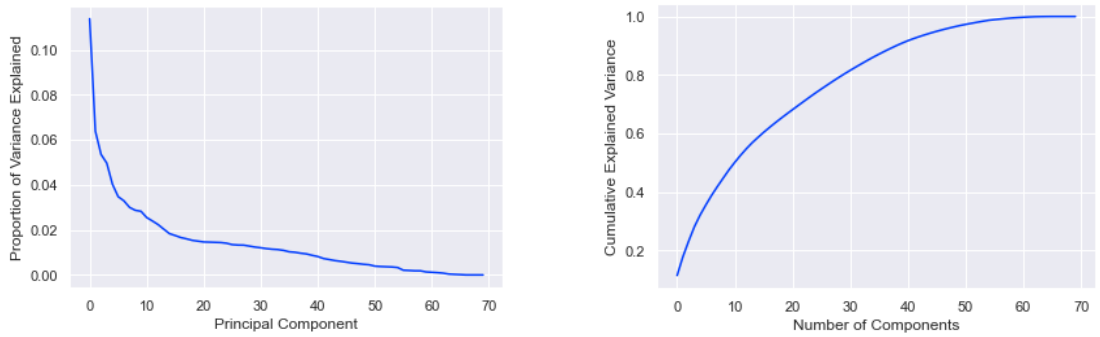


Figure 5.1.: Correlation matrix of the Kaggle dataset

5.1.1. PCA

A fundamental part of the presented framework was the implementation of PCA to reduce dimensionality. The left figure 5.2 shows the percentage of variance each feature describes, plotted with Matplotlib.



(a) Proportion of variance explained

(b) Cumulative explained variance

Figure 5.2.: Variance explained

The right side of figure 5.2 presents the cumulative sum over all variances. On the one hand, the horizontal axis illustrates the number of features in the dataset. On the other hand, the vertical axis describes the proportion of variance explained. The proportion of explained variance for implementing this dataset was 96.9%. This means that approximately 3% of variance is lost.

5.1.2. Random Forest

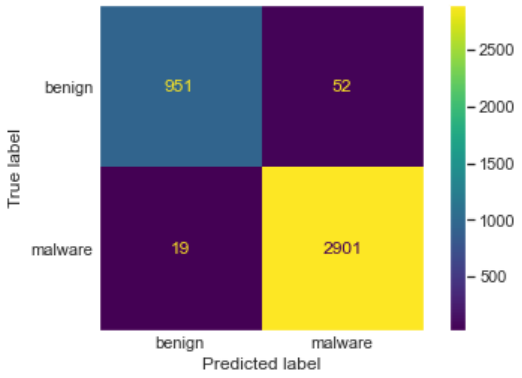
The first evaluated model was Random Forest. Table 5.1 summarizes the classification results for Random Forest.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9819	0.9814	0.9708	0.9760	0.9963	6.15
RF KT	0.9834	0.9831	0.9732	0.9780	0.9957	24.39
RF WPCA	0.9903	0.9925	0.9820	0.9871	0.9985	2.02

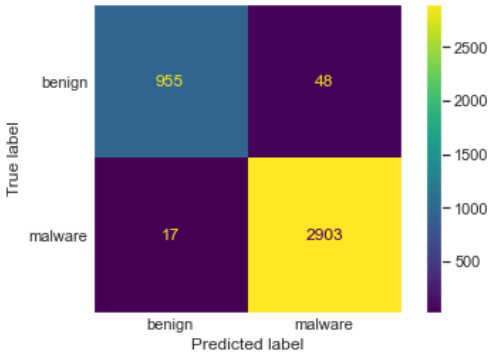
Table 5.1.: Classification results for Random Forest with the Kaggle dataset

Figure 5.3 shows the confusion matrices for Random Forest with the Kaggle dataset. The confusion matrix in figure 5.3b displays the classification results with the integration of Keras Tuner. In this figure, the algorithm detected 2903 malware samples correctly. Random Forest falsely identified only 17 samples as benign. On the other hand, 48 samples were predicted to

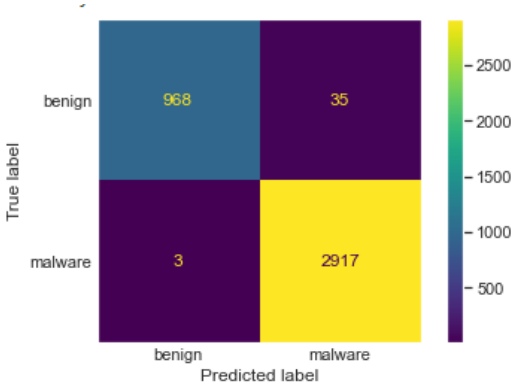
be malicious but were, in verity, benign. In the confusion matrix without PCA (5.3c), Random Forest falsely predicted three malware samples.



(a) Confusion matrix of Random Forest



(b) Confusion matrix of Random Forest with Keras Tuner



(c) Confusion matrix of Random Forest without PCA

Figure 5.3.: Confusion matrices of Random Forest with the Virusshare dataset

The confusion matrices for the classification results with Random Forest and the Kaggle dataset have similarities. The difference between the confusion matrix from Random Forest and Random Forest with Keras Tuner is marginal. However, figure 5.3c illustrating Random Forest without PCA deviates from the other matrices.

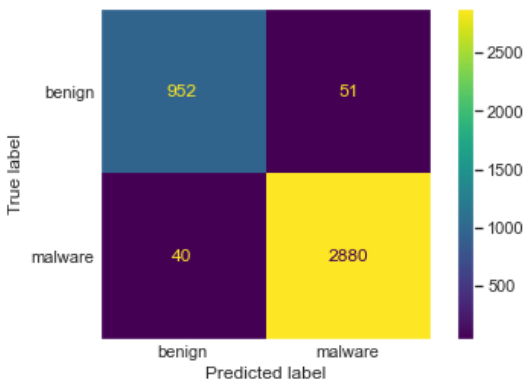
5.1.3. KNN

The second evaluated Model was KNN. The classification results of this classifier are presented likewise to the representation of Random Forest. The classification results of KNN are displayed in table 5.2.

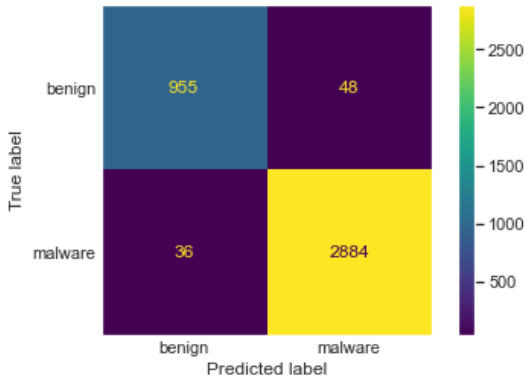
Name	Accuracy	Precision	Recall	F1-score	AUC	Time
KNN	0.9768	0.9711	0.9677	0.9694	0.9849	0.02
KNN KT	0.9786	0.9737	0.9699	0.9718	0.9866	0.01
KNN WPCA	0.9781	0.9730	0.9692	0.9711	0.9851	0.02

Table 5.2.: Classification results for KNN with the Kaggle dataset

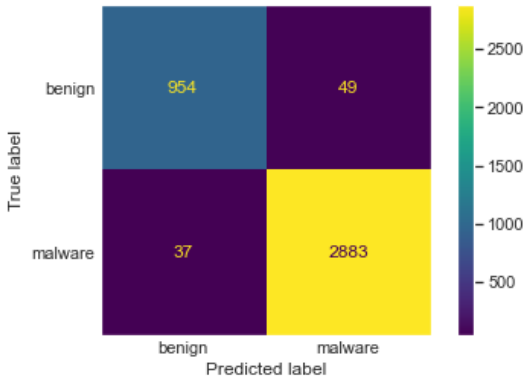
The confusion matrices for the classification results of KNN are illustrated in figure 5.4.



(a) Confusion Matrix of KNN



(b) Confusion Matrix of KNN with Keras Tuner



(c) Confusion Matrix of KNN without PCA

Figure 5.4.: Confusion matrices of KNN with the Kaggle dataset

Figure 5.4a shows the confusion matrix of KNN with PCA. This matrix has the highest false positives and false negatives compared to the other matrices. The confusion matrix of KNN with Keras Tuner, illustrated in figure5.4b, has the lowest false-negative rate out of all matrices, with 36 samples mispredicted as benign. The confusion matrix in figure 5.4c shows a similar distribution of true positives and true negatives like the other matrices. In summary, the confusion matrices for KNN with different approaches are mainly identical.

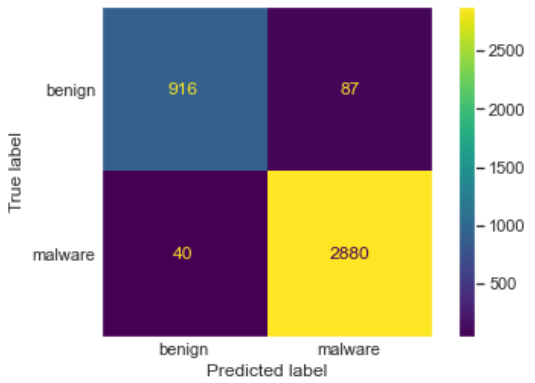
5.1.4. SVM

The third classifier was SVM. The classification results for SVM are displayed in table 5.3.

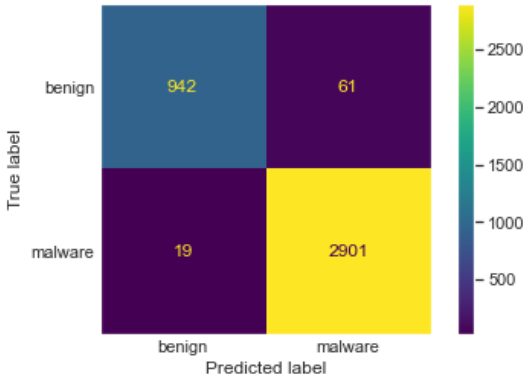
Name	Accuracy	Precision	Recall	F1-score	AUC	Time
SVM	0.9676	0.9644	0.9498	0.9568	0.9798	8.74
SVM KT	0.9796	0.9798	0.9663	0.9728	0.9888	7.91
SVM WPCA	0.9679	0.9646	0.9503	0.9572	0.9799	16.86

Table 5.3.: Classification results for SVM with the Kaggle dataset

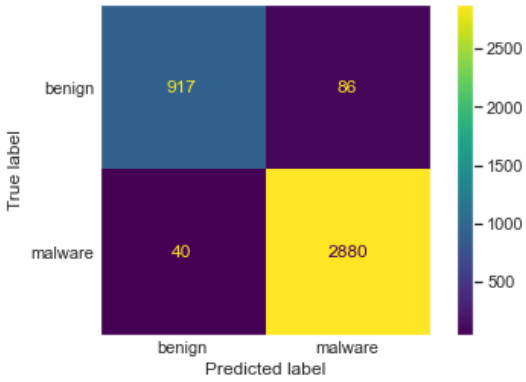
Similar to the structure of the presentation of results with Random Forest and KNN, the confusion matrices of SVM are illustrated in figure 5.5. The first confusion matrix, presented in figure 5.5a, mispredicted 87 benign samples. However, SVM with Keras Tuner in figure 5.5b only mispredicted 61 benign samples. Additionally, SVM with Keras Tuner correctly identified 2901 malware samples, almost 30 more samples than SVM with and without PCA.



(a) Confusion matrix of SVM



(b) Confusion matrix of SVM with Keras Tuner



(c) Confusion matrix of SVM without PCA

Figure 5.5.: Confusion matrices of SVM with the Kaggle dataset

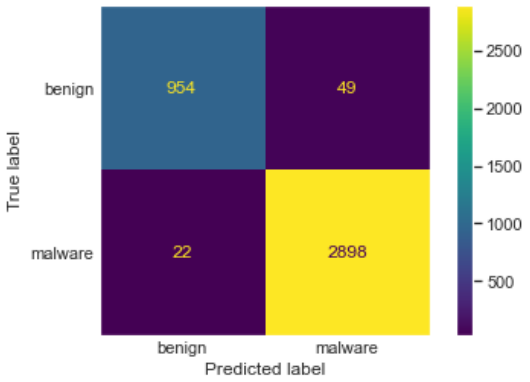
The last confusion matrix presented in figure 5.5c has a nearly identical distribution of true positives and true negatives like the matrix of SVM with PCA.

5.1.5. MLP

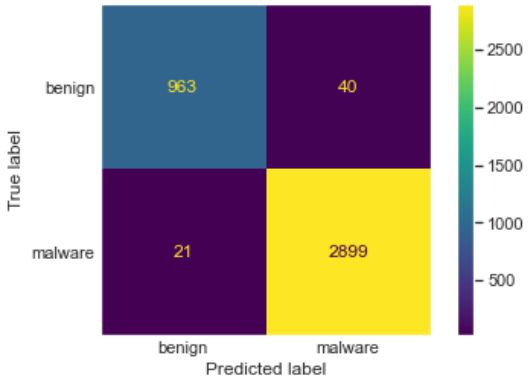
The first deep neural network of this implementation was MLP which contained 16 hidden layers. The classification results of this model are structured equally as the structure of the machine learning models. The results for the implementation of MLP are obtainable from table 5.4, and the classification results, namely the confusion matrix are presented in figure 5.6.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
MLP	0.9819	0.9804	0.9718	0.9760	0.9927	9.13
MLP KT	0.9845	0.9825	0.9765	0.9794	0.9938	23.06
MLP WPCA	0.9816	0.9806	0.9710	0.9756	0.9907	11.97

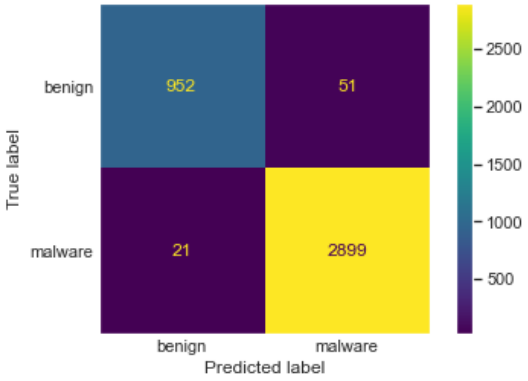
Table 5.4.: Classification results for MLP with the Kaggle dataset



(a) Confusion matrix of MLP



(b) Confusion matrix of MLP with Keras Tuner



(c) Confusion matrix of MLP without PCA

Figure 5.6.: Confusion matrices of MLP with the Kaggle dataset

The confusion matrix illustrated in figure 5.6b shows the results of MLP with Keras Tuner. The confusion matrix also has the lowest false positives and false negatives. Especially the number of false positives deviate from the other illustrations. The number of false negatives is almost identical to the confusion matrix of MLP with PCA, illustrated in figure,5.6a and MLP without PCA, presented in figure 5.6c. In comparison, the difference between results from MLP with and without PCA is marginal.

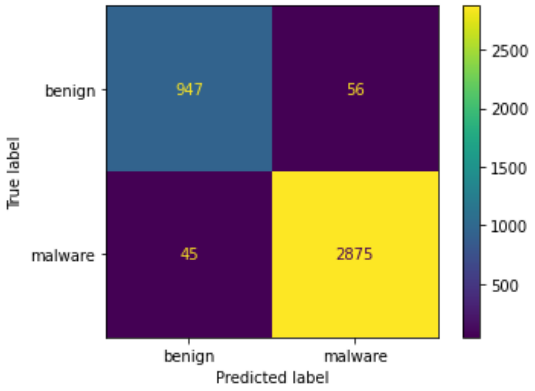
5.1.6. DNN

The final classifier was a deep neural network with four hidden layers. The following table 5.5 shows the classification results for the deep neural network. Furthermore, the confusion matrices of the results are illustrated in figure 5.7.

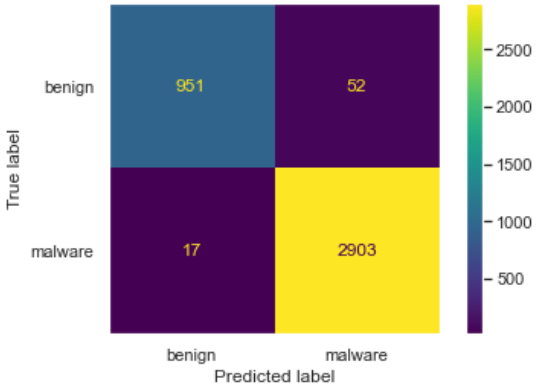
Name	Accuracy	Precision	Recall	F1-score	AUC	Time
DNN	0.9758	0.9791	0.9884	0.9836	0.9868	7.41
DNN KT	0.9773	0.9790	0.9907	0.9847	0.9784	7.75
DNN WPCA	0.9776	0.9782	0.9917	0.9848	0.9873	8.42

Table 5.5.: Classification results for DNN with the Kaggle dataset

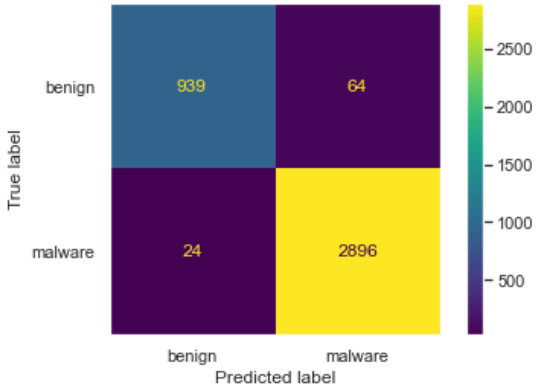
Figure 5.7a illustrates the confusion matrix based on the results of the DNN with PCA. The figure demonstrates that the DNN could correctly classify 947 benign samples. Furthermore, 2875 malware samples were correctly classified. However, the confusion matrix of the DNN with Keras Tuner, illustrated in figure 5.7b, delivers a higher detection rate. With only 17 samples mispredicted as benign, the DNN with Keras Tuner has the lowest false-negative rate out of all confusion matrices with the DNN. Figure 5.7c presenting the results of the DNN without PCA has a low false-negative rate but obtained the highest number of false positives.



(a) Confusion matrix of DNN



(b) Confusion matrix of DNN with Keras Tuner



(c) Confusion matrix of DNN without PCA

Figure 5.7.: Confusion matrices of DNN with the Kaggle dataset

5.2. Virusshare Dataset

The following section contains the classification results for the Virusshare dataset. The results of each classifier are presented in the subsection of this chapter. The Virusshare dataset contains 57 features and 138.047 samples. The ROC and AUC of the classification results of each classifier are illustrated in the appendix in figure D.1. An adequate description of the dataset is shown in the methodology section 4.1.2. Similar to the framework implementation with the Kaggle dataset, the Virusshare dataset was transformed into a data frame with pandas.DataFrame. Then features with no statistical value were eliminated, and then the Correlation Matrix was computed.

The corresponding correlation matrix is displayed in figure 5.8. The correlation matrix was plotted with the seaborn library. The interpretation of positive and negative correlations based on the correlation matrix in figure 5.8 similarizes the description in the Kaggle dataset section 5.1.

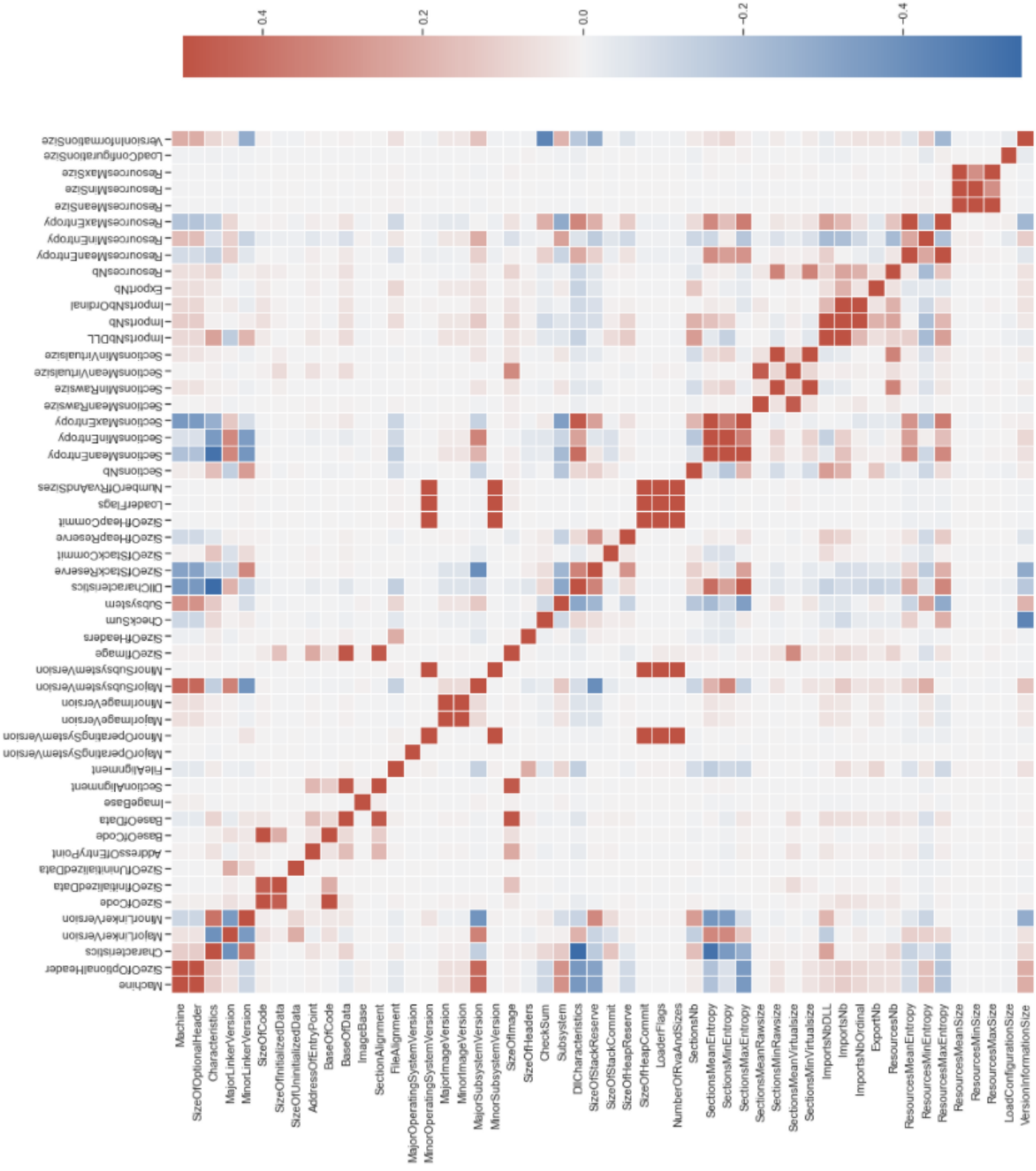
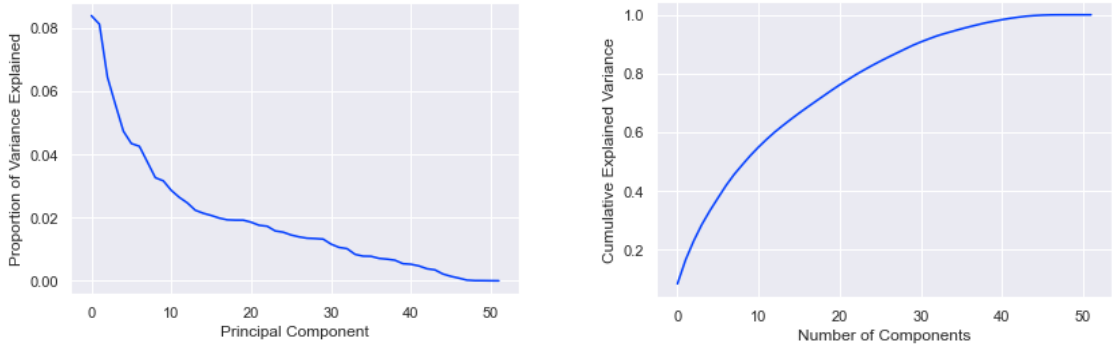


Figure 5.8.: Correlation matrix of the Virusshare dataset

5.2.1. PCA

The results for PCA are illustrated in figure 5.9. The left figure shows the percentage of variance each feature describes. The right figure displays the cumulative sum over all variances.



(a) Proportion of variance explained

(b) Cumulative explained variance

Figure 5.9.: Variance explained for the Virusshare dataset

The proportion of explained variance for implementing this dataset was 99.99%.

5.2.2. Random Forest

Similar to the structure of the result section from the Kaggle dataset, Random Forest was the first implemented model. Table 5.6 summarizes the classification results of Random Forest with the Virusshare dataset.

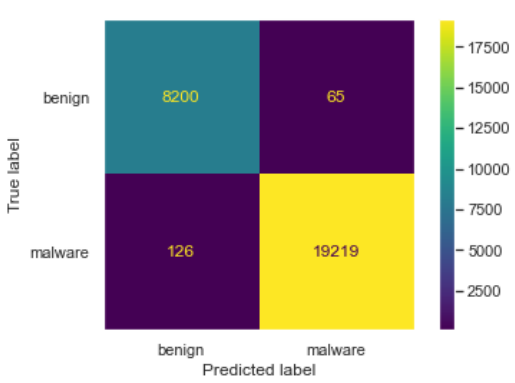
Name	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9931	0.9907	0.9928	0.9918	0.9996	54.64
RF KT	0.9935	0.9912	0.9933	0.9922	0.9995	50.51
RF WPCA	0.9944	0.9932	0.9934	0.9933	0.9997	16.99

Table 5.6.: Classification results for Random Forest with the Virusshare dataset

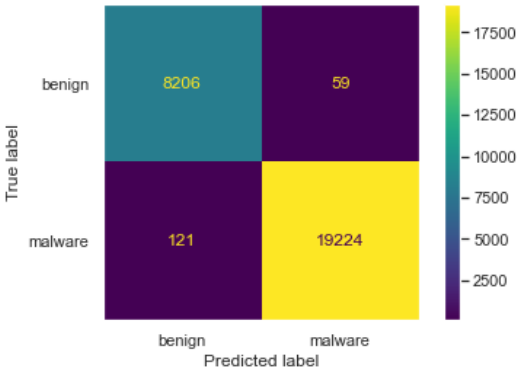
The confusion matrices of Random Forest with the Virusshare dataset are presented in figure 5.10.

The confusion matrices have a similar false and negative ratios distribution like the matrices with the Kaggle dataset. Although the Virusshare dataset has a much higher sample size, the number of false positives illustrated in figure 5.10b is meager. Random Forest with Keras Tuner obtained the lowest false positives compared to the other Random Forest implementations. However, Random Forest without PCA, presented in figure 5.10c has the lowest number of false negatives. The algorithm, featuring Random Forest with PCA, shown in figure 5.10a,

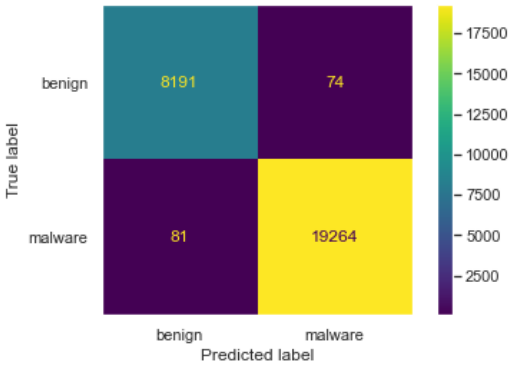
identified 8200 benign samples correctly. Compared to the other Random Forest matrices, the detection ratio is average.



(a) Confusion matrix of Random Forest



(b) Confusion matrix of Random Forest with Keras Tuner



(c) Confusion matrix of Random Forest without PCA

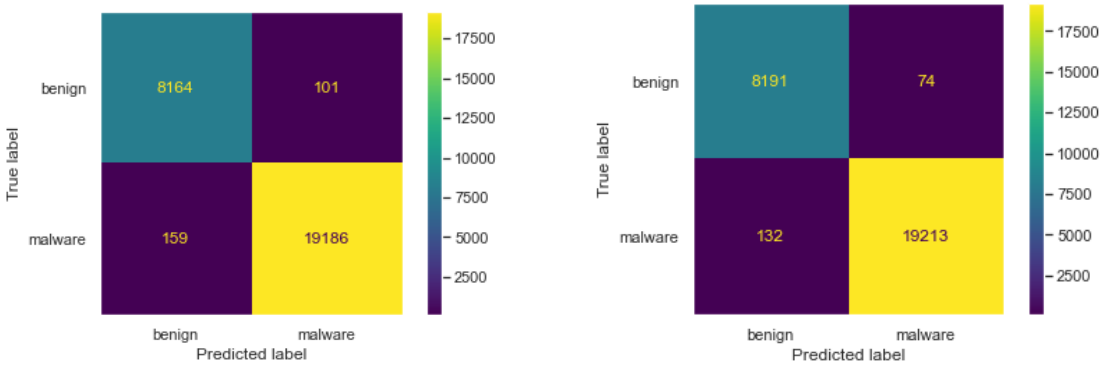
Figure 5.10.: Confusion matrices of Random Forest with the Virusshare dataset

5.2.3. KNN

For the second model, KNN, classification results are displayed in table 5.7.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
KNN	0.9906	0.9878	0.9898	0.9888	0.9973	0.02
KNN KT	0.9925	0.9902	0.9921	0.9911	0.9980	0.03
KNN WPCA	0.9906	0.9878	0.9898	0.9888	0.9973	0.02

Table 5.7.: Classification results for KNN with the Virusshare dataset



(a) Confusion matrix of KNN

(b) Confusion matrix of KNN with Keras Tuner

Figure 5.11.: Confusion matrices of KNN with the Virusshare dataset

The confusion matrices for KNN with the Virusshare dataset are illustrated in figure 5.11. The confusion matrix of KNN with PCA, presented in figure 5.11a, detected 19186 malware samples correctly. However, with 159 samples falsely identified as benign, KNN has a relatively high number of false negatives. The results of KNN with Keras Tuner, illustrated in figure5.11b, show that the classifier could predict 8191 benign samples correctly. The false-negative samples were also fewer than the algorithm, featuring KNN without PCA. Both approaches, KNN with and without PCA, have an identical positive and negative samples distribution. Therefore, the confusion matrix featuring KNN without PCA is presented in the appendix in figure D.2a.

5.2.4. SVM

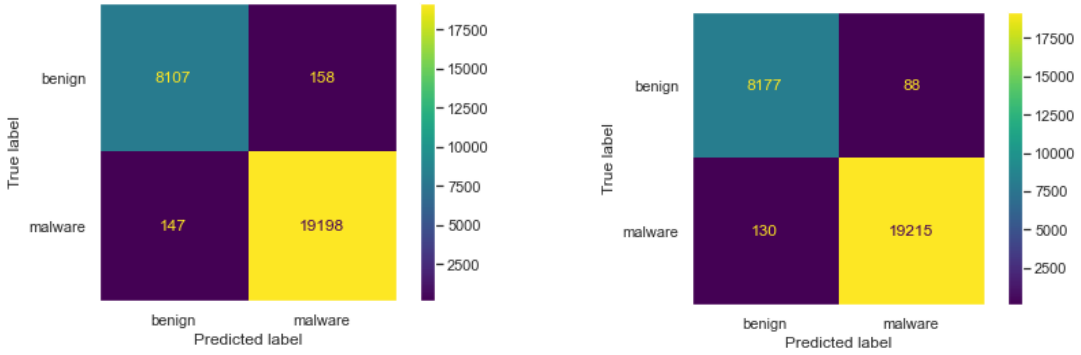
The third classifier, SVM, was computed with the Virusshare dataset, and the classification results are displayed in table 5.8.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
SVM	0.9890	0.9870	0.9866	0.9868	0.9974	472.93
SVM KT	0.9921	0.9899	0.9913	0.9906	0.9981	933.23
SVM WPCA	0.9890	0.9870	0.9866	0.9868	0.9974	483.36

Table 5.8.: Classification results for SVM with the Virusshare dataset

The confusion matrices of SVM with the Virusshare dataset are illustrated in figure 5.12. The first figure 5.12a shows the confusion matrix of SVM with PCA. The classifier falsely predicted 305 samples. An identical distribution of false predictions achieved SVM without PCA. The confusion matrix of SVM without PCA can be obtained from the appendix in figure D.2b. The confusion matrix of SVM with Keras Tuner illustrated in figure 5.12b, shows a lower number

of false negatives. The classifier mispredicted only 88 benign samples. The number of false negatives was almost similar to the result of SVM in figure 5.12a.



(a) Confusion matrix of SVM

(b) Confusion matrix of SVM with Keras Tuner

Figure 5.12.: Confusion matrices of SVM with the Virusshare dataset

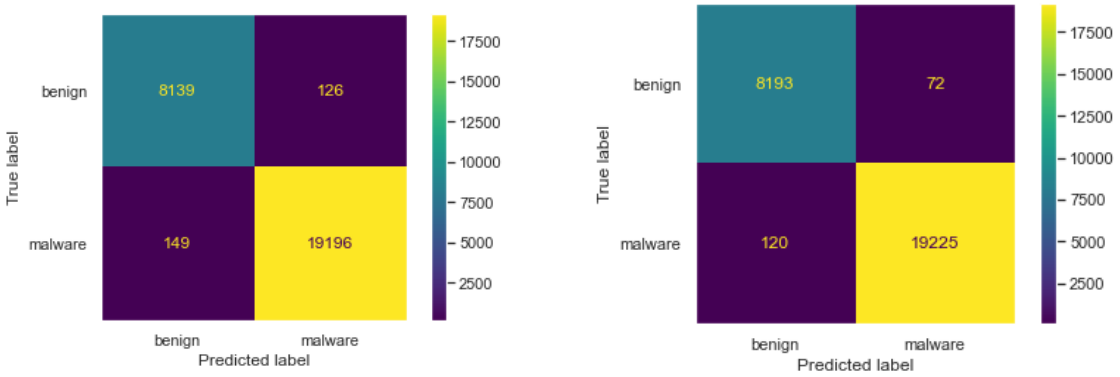
5.2.5. MLP

The MLP model, containing 16 hidden layers, was computed next. The classification results can be obtained from table 5.9.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
MLP	0.9900	0.9878	0.9885	0.9881	0.9990	39.38
MLP KT	0.9930	0.9909	0.9925	0.9917	0.9992	117.96
MLP WPCA	0.9905	0.9889	0.9886	0.9887	0.9990	58.91

Table 5.9.: Classification results for MLP with the Virusshare dataset

Next, the confusion matrices are illustrated in figure 5.13. The confusion matrix of MLP with Keras Tuner, illustrated in figure 5.14b, correctly classified 19225 malware samples. Furthermore, 8193 benign samples were accurately identified. Figure 5.14a shows the confusion matrix of MLP with PCA. The classifier mispredicted 126 benign samples and 149 malware samples. The confusion matrix of MLP without PCA is illustrated in the appendix in figure D.2c. The difference between the confusion matrix of the MLP with and without PCA is marginal.



(a) Confusion matrix of MLP

(b) Confusion matrix of MLP with Keras Tuner

Figure 5.13.: Confusion matrices of MLP with the Virusshare dataset

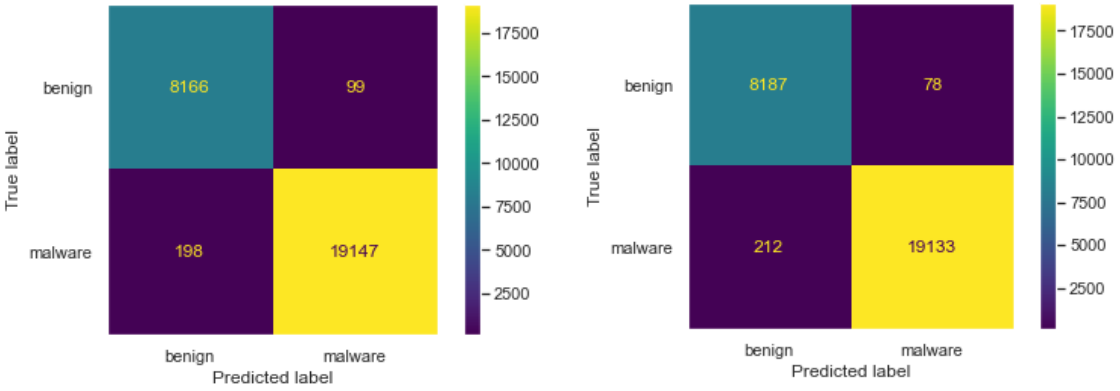
5.2.6. DNN

The final classifier, DNN, contained four hidden layers. The following table 5.10 shows the classification results for the DNN.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
DNN	0.9885	0.9732	0.9882	0.9804	0.9985	48.32
DNN KT	0.9898	0.9760	0.9901	0.9828	0.9987	48.75
DNN WPCA	0.9899	0.9784	0.9877	0.9828	0.9986	50.49

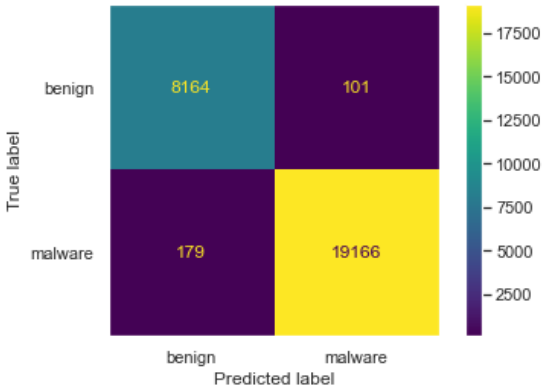
Table 5.10.: Classification results for DNN with the Virusshare dataset

The confusion matrices for the DNN are illustrated in figure 5.14. The confusion matrix of the DNN with Keras Tuner, presented in figure 5.14b, detected 19133 malware samples correctly. However, 212 malware and 78 benign samples were mispredicted. A similar distribution can be observed in the confusion matrices in figure 5.14a and 5.14c. The number of false positives is higher than the approach with Keras Tuner. However, both algorithms achieved a lower amount of false negatives.



(a) Confusion matrix of DNN

(b) Confusion matrix of DNN with Keras Tuner



(c) Confusion matrix of DNN without PCA

Figure 5.14.: Confusion matrices of DNN with the Virusshare dataset

6. Analysis of the Detection Results

In this section, the results of the proposed framework are discussed. The analysis is based on the research questions defined in section 4. First, the results of the framework are summarized with selected performance measures. Then, the results are compared with recent publications. Then, the research question addressing the difference between both datasets is elaborated. The subsequent section establishes an analysis of the evaluation results obtained with Keras Tuner. Finally, the results achieved with PCA are discussed, highlighting critical elements for feature selection.

6.1. Summary of Results of the Framework

The summary of results achieved using the framework with the Kaggle Dataset is presented in table 6.1.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9834	**0.9831**	0.9732	0.9780	**0.9957**	24.39
KNN	0.9786	0.9737	0.9699	0.9718	0.9866	**0.01**
SVM	0.9796	0.9798	0.9663	0.9728	0.9888	7.91
MLP	**0.9845**	0.9825	0.9765	0.9794	0.9938	23.06
DNN	0.9773	0.9790	**0.9907**	**0.9847**	0.9784	7.75

Table 6.1.: Summary of results of the Kaggle dataset

The table shows the classification models paired with accuracy, precision, recall, F1- score, AUC, and time. Evaluation metrics with the highest performance are marked in bold. The classification results illustrate that all classification models were vastly able to classify the correct label with accuracy, precision, recall, F1-score, and AUC ranging between 0.9663 and 0.9957.

Random Forest achieved excellent results with an accuracy of 0.9834 and a recall of 0.9732. The classification model obtained the highest performance based on precision and AUC, measuring 0.9831 for precision and 0.9957 for AUC. However, the computation time was high, compared to the other classification models.

KNN achieved the lowest performance with 0.9737 precision and 0.9718 F1-score. Mainly the precision was low compared to the other algorithms. Nevertheless, KNN accomplished good results for the AUC and the computation time. With a computation time of 0.01 seconds, the model had the lowest computation time out of all classification models.

The scores of the SVM classifier resemble the results of the KNN model. The accuracy, precision, and F1-score results are higher than KNN but lower than most classifiers. With an 0,9663 recall, the model achieved the most deficient recall performance out of all models. However, like KNN and DNN, SVM achieved a reasonable computation time of 7.91 seconds.

MLP achieved the highest accuracy value with 0.9845. The results of the MLP classifier resemble the results of the Random Forest model, with a slightly better performance in recall and F1-score. Additionally, a good AUC score of 0.9938 demonstrates that the MLP classifier is one of the best performing models of this framework. Compared to the other classifiers, MLP has a high computation time that marginally differs from the computation time of Random Forest.

The execution of the DNN model with Keras accomplished comparable results to the classifiers implemented with Scikit-learn. Furthermore, the classification results presented that the DNN achieved the best scores with 0.9907 recall and 0.9847 F1-score. The computation time was likewise relatively low compared to Random Forest and MLP. However, the low accuracy with 0.9773 and the precision with 0.9790 are averagely low compared to the other classifiers.

An unambiguous ranking of all classifiers based on the results presented in table 6.1 is challenging since no classifier achieved the best results in all evaluation metrics. Additionally, all classifiers achieved comparable results while considerable differences are primarily apparent in the computation time. If the computation time is not considered, knowing that the metric is dependant on technical resources and the time is on an overall basis tolerably low, MLP and Random Forest are the best performing classification models of this framework with the Kaggle Dataset.

Name	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	**0.9935**	**0.9912**	**0.9933**	**0.9922**	**0.9995**	50.51
KNN	0.9925	0.9902	0.9921	0.9911	0.9980	**0.03**
SVM	0.9921	0.9899	0.9913	0.9906	0.9981	933.23
MLP	0.9930	0.9909	0.9925	0.9917	0.9992	117.96
DNN	0.9898	0.9760	0.9901	0.9828	0.9987	48.75

Table 6.2.: Summary of results of the Virusshare dataset

In the following segment, the classification results for the Virusshare Dataset are discussed. The results are summarized and displayed in table 6.2. Initial insights obtained from the classifi-

cation results illustrate that all classifiers performed very well. The lowest score of an evaluation metric without computation time was 0.9760, while the average score of all classifiers was above 0.99. Evaluation metrics with the highest score are once again marked in bold.

The Random Forest classifier achieved the best measures with 0.9935 accuracy, 0.9912 precision, 0.9933 recall, 0.9922 F1-score, and 0.9995 AUC compared to all classifiers. Random Forest is, therefore, the classifier with the highest performance. The computation time of 50.51 seconds was average in comparison to other classification results.

The classifier with the lowest computation time was KNN. The excellent computation time is supported by a good accuracy, recall, and F1 score. KNN outperforms SVM and DNN in almost every evaluation metric. However, the KNN model achieved the lowest AUC score out of all classifiers.

A relatively low performance achieved SVM, primarily by a computation time of 933.23 seconds. Additionally, the computation time is almost nine times higher than the classifier with the second-highest computation time, MLP. With a accuracy of 0.9921, the classifier achieved the second-lowest accuracy, precision, recall, and F1 score compared to all classification results. The classification results of SVM demonstrate that the classifier has deficiencies obtaining comparable results to other classifiers. It can be considered that the classifier achieves the lowest performance out of all classifiers in this framework.

MLP, on the other hand, resembles the results of the Random Forest model. Achieving the second highest accuracy, precision, recall, F1 score, and AUC measures, the classifier is one of this framework's best performing classification models. The main difference between Random Forest and MLP is the computation time, with MLP obtaining the second-highest computation time, more than twice as high as Random Forest.

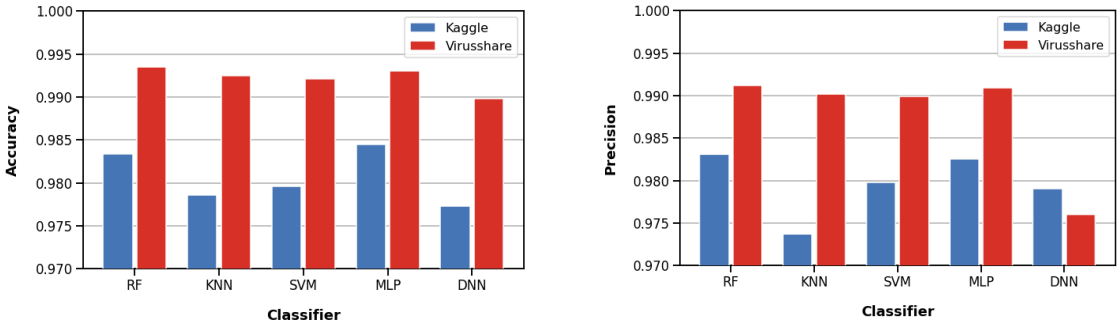
The DNN achieved the lowest accuracy, precision, recall, and F1-score. The classification results are well comparable to the outcomes of SVM. While SVM achieved better measurement scores in the metrics mentioned above, the DNN model obtained the second-best computation time, outperforming, for example, Random Forest in this specific metric. Nonetheless, the relatively low evaluation metric scores rank the DNN model in the lower area of performance.

A ranking of classifiers based on the results in table 6.2 is more straightforward. The results of Random Forest demonstrate that the classifier is, without uncertainty, the best performing model. Ranking behind Random Forest, MLP can be considered a reliable classifier based on the classification performance. Although the measurements of KNN are lower than scores of Random Forest and MLP, the classifier obtained related results. Finally, the SVM and DNN classifiers received good results but fell short concerning other classifiers.

6.2. Comparison between Datasets

The following section compares the results obtained from the Kaggle and Virusshare dataset. First, the comparison is based on the evaluation metrics illustrated in the methodology section. Then, critical reasons for the performance differences between datasets and classifiers are elaborated.

The first comparison between evaluation results from the Kaggle and Virusshare dataset is presented in table 6.1.



(a) Comparison of Accuracy

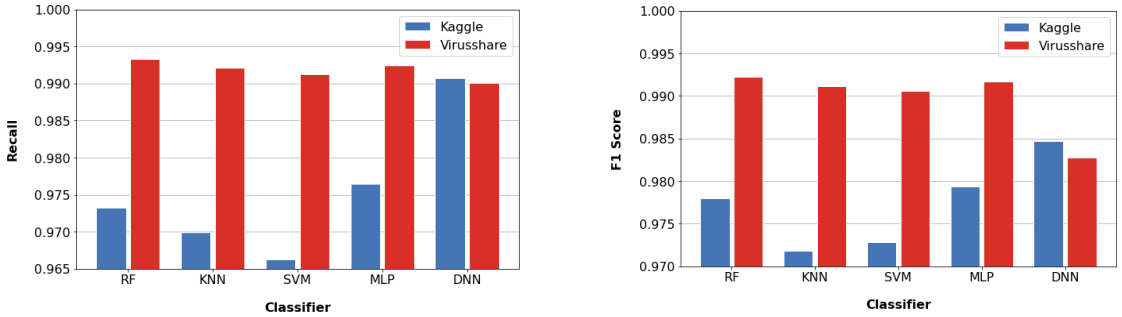
(b) Comparison of Precision

Figure 6.1.: Comparison of Accuracy and Precision

Figure 6.1a shows the comparison of accuracy between the Kaggle and Virusshare datasets. Performance results of the Kaggle dataset are color-coded in blue. Evaluation results from the Virusshare dataset, on the contrary, are color-coded in red. First observations indicate that accuracy evaluation scores from the Virusshare dataset are superior for each classifier. The difference between accuracy values in the Virusshare dataset and the Kaggle dataset is approximate, on average, 0,011. The most significant difference between accuracy scores is 0,014, resulting from the KNN classifier. From the above figure 6.1a, it can be observed that results from the Kaggle dataset have an uneven distribution. The values vary between 0,9786 and 0,9845, with a standard deviation of 0,0028. Classification results based on accuracy with the Virusshare dataset have a more balanced distribution with a standard deviation of 0,0013.

The comparison of precision between the Kaggle and Virusshare datasets is displayed in figure 6.1b. Similar to the accuracy comparison described in the above segment, most evaluation results from the Virusshare dataset outperform results obtained from the Kaggle dataset. Again, the most significant difference between precision values results from the KNN classifier. The difference in the precision value is approximately 0,017. A similarity of the comparisons between the accuracy and the precision are the Random Forest and MLP classifiers. Both classifiers are proportional to the opposing dataset in the comparison. The precision results of the DNN illustrate that results from the Kaggle dataset outperform results from the Virusshare dataset. Therefore, the assumption that results obtained from the Virusshare dataset are superior to

the Kaggle dataset can be disproved. Noteworthy is the distribution of the precision results of the Virusshare dataset. While Random Forest, KNN, SVM, and MLP contribute to an evenly balanced distribution, the results based on the DNN classifier highly deviate.



(a) Comparison of Recall

(b) Comparison of F1 score

Figure 6.2.: Comparison of Recall and F1 Score

Comparisons of evaluation results based on accuracy and precision frequently have a distinct tendency. Classification results from Random Forest, KNN, SVM, and MLP, utilizing the Kaggle dataset, perform worse than results obtained by the Virusshare dataset. A continuation of this trend can be observed in figure 6.2.

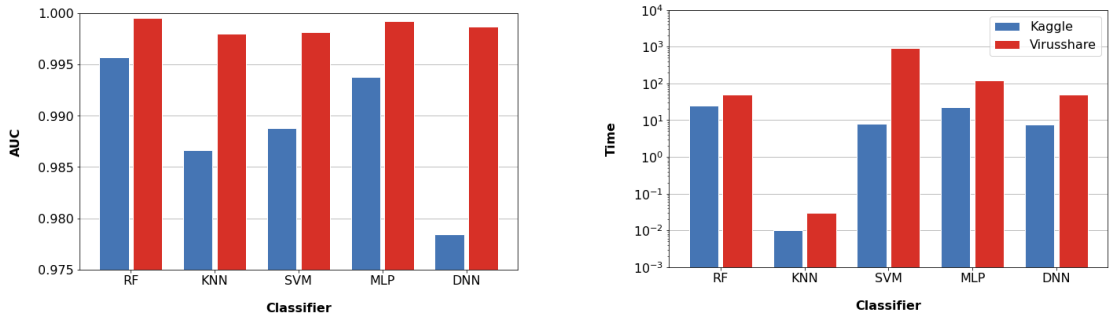
A comparison of recall between results of the Kaggle dataset and Virusshare dataset is demonstrated in figure 6.2a. In contrast to all classifiers, DNN had the lowest difference between results from both datasets. Noteworthy is the observation that the DNN results of the Kaggle dataset are almost equal to the results of other classifiers with the Virusshare dataset. The most significant difference between recall values resulted from the SVM classifier. The approximate difference of recall measurements of the SVM results was 0,025. Similar to the observations made in figure 6.1a classification results based on recall from the Virusshare dataset have a balanced distribution. The standard deviation of these recall measurements is 0,0011, which is very low compared to similar statistical measures.

Figure 6.2b illustrates the comparison of F1 score values between the Kaggle and Virusshare datasets. The comparison resembles the discussion conducted in the last segment. The most significant difference between F1 score values was approximately 0,019, based on the KNN classifier. The F1 score results of the DNN with the Virusshare dataset were very low, similar to the outcomes observed in the precision comparison. In both cases, the results of the Kaggle dataset outperformed these measurements. Based on the past observations, a tendency to compare the DNN classification results is developing.

Following the scheme directed in the above segments, the evaluation metrics AUC and Computation Time are now discussed. The comparison is presented in figure 6.3.

A comparison of AUC measurements can be examined in figure 6.3a. The first conspicuities are that nearly both classification results of the datasets are similar to each other. Random For-

est achieved the highest AUC score in both datasets with a difference of 0,004. Random Forest is closely followed by MLP, which performed well in both classification results and achieved in both datasets the second-highest scores. However, the results of the DNN classifier with the Kaggle dataset do not compare to the relation between the datasets utilizing Random Forest, KNN, SVM, or MLP. The AUC measurements of the DNN classifier show that the results are similar to the comparison of accuracy. Furthermore, the difference between results from each dataset related to the DNN classifier is approximately 0,02. This statistical value is the most significant distance between the Kaggle and Virusshare dataset measurements.



(a) Comparison of AUC

(b) Comparison of Computation Time

Figure 6.3.: Comparison of AUC and Comutation Time

Since the computation time of the classifiers had a high range with very high and low values, the measurements are displayed in a scaled bar plot. The distribution of data can be obtained from figure 6.3b. As previously discussed in the Summary of the Results section, KNN had a short computation time. Noteworthy is that the difference in the computation time between both datasets with KNN is likewise low, considering the Virusshare dataset is 14 times larger than the Kaggle dataset. This observation supports KNN as a good classifier for this framework and possible implementations with more extensive datasets. While KNN had a low difference in computation time between datasets, results with SVM measured the highest difference. The computation time SVM needed for the Virusshare dataset was 117 times larger than the computation with the Kaggle dataset. These results support the complexity problem of SVM, considering SVM has a complexity between $O(n^2)$ and $O(n^3)$ [78]. The computation time results of SVM make the classifier unfitting or limited for some instances in this framework. While the computation time is relatively low with a small dataset, a larger dataset is unsuitable for this classifier utilizing this framework. The plots symbolizing Random Forest illustrate a marginal difference between computational time results. Observations indicate that Random Forest has no difficulties with larger datasets in this framework and is a suitable classifier for future work. While Random Forest took twice as long to compute results with the Virusshare dataset, MLP needed five times longer. The computation time remains relatively low, considering the size of the dataset. A similar formulation can be applied to the computation results of

the DNN classifier. DNN needed six times longer to compute the classification results with the Virusshare dataset; 48,75 seconds is an adequate computation time compared to other results.

In this section, both datasets we compared based on the evaluation metrics of the classification results. There is no detailed assessment that results from the Virusshare dataset are generally better than results from the Kaggle dataset. However, the comparison established that classification results of the Virusshare dataset mostly outperform results from the Kaggle dataset. There are many possible reasons why measurements from the Virusshare dataset are superior. These reasons will be shortly addressed and later the most significant will be discussed in the subsequent chapter. The following possible reasons for deviant measurements between the datasets are illustrated:

- **Sample Size**
  The Kaggle dataset has a sample size of 19.611 samples, whereas the Virusshare dataset has 138.047 samples. The classifiers from the Virusshare dataset receive more data as input, therefore, more training data, likely improving the classification process.

- **Distribution Ratio**
  Although the distribution ratio between malware and benign samples was reasonable in both datasets; the Kaggle dataset had a higher imbalanced distribution ratio. Combined with a smaller sample size, the training data of the Kaggle dataset contained less benign samples and more malware files.

- **Significance of Features**
  The Kaggle dataset has 79 features, whereas the Virusshare dataset has 57 features. A large number of features can lead to better classification results. However, if these features are redundant and irrelevant, this can lead to a decreased detection rate.

- **Feature Selection**
  This analysis is possibly the compelling reason why classifiers with the Kaggle dataset perform worse. The feature selection process consists of PCA, reducing the dimension of features to a defined minimal subset. Through the utilization of this method, information is being lost.

The comparison of datasets has also illustrated that specific models generally perform well in this framework, independent of the dataset. With Random Forest and MLP, two classifiers present excellent classification results and can be considered to extend this framework. Considering Random Forest is an ensemble learning algorithm, and MLP is a deep learning model, it can be determined that machine learning and deep learning models are well suited for this framework based on the results provided. The prior discussion clarified that deep learning and machine learning models could obtain reliable results with this framework. Random Forest outperforms all classifiers if specified conditions are not regarded, for instance, computation time.

However, deep learning models implemented in this framework can achieve similar results and are consequently excellent classifiers. KNN had an average performance in both datasets, with classification results mostly resembling results from the SVM classifier. However, KNN has proven to be a decent classifier for this framework, with an extremely low computation time. KNN can provide fast, reliable results for future classification processes with extensive datasets. Still, KNN is not a classifier that can achieve the most favorable results for this and future frameworks. SVM achieved good classification results with both datasets. Nevertheless, the complexity of SVM is an immense obstacle that can be hard to overcome, depending on the size of the dataset. These limitations, paired with an average performance of accuracy, precision, recall, and F1 score, make the classifier practically unreliable for extending this framework. The DNN classifier provided great results concerning measurements with the Kaggle dataset, while results with the Virusshare dataset were, in comparison, poor. Additionally, the comparison between both datasets with the DNN revealed that the results are relatively unrelated, which diverges from the observations made with different classifiers. Since this classifier is highly adjustable, other parameter settings could have led to better classification results concerning the Virusshare dataset. Additionally, more epochs could have increased the performance, mainly as the Virusshare dataset contains more samples. However, the model had the same settings for both datasets for better comparability. A modification of parameters for the DNN model can be an assignment for a future extension of this framework.

6.2.1. Significance of Features

In this section, the features of each dataset are analyzed and compared. The examination of features serves to understand the performance of the classification process. Additionally, the results of this chapter can function as an essential input for future studies.

The feature analysis starts with the examination of correlations. The correlation matrix described in figure 5.1 shows the correlation coefficients of all features to each other based on the Kaggle dataset. The values are computed with a linear correlation approach, the Pearson correlation. These values are always between -1 and 1, with a value above 0 describing a positive correlation. Therefore, a negative correlation has values below 0.

Figure 5.1 displays numerous positive correlating features located in the top left corner of the matrix. These highly correlating variables indicate that one of these two correlating features can be dropped. Additional positive correlations are found in features based on operating versions, for example, *MajorOperatingSystem*. Features like *SizeOfStackReserve*, *SectionMaxEntropy*, and *ImageDirectoryEntryExport* have almost no correlation to other features. In addition, there are hardly any negative correlations. A perfect negative correlation exists between elements of *SectionMaxChar* and *SectionAlignment*. In short, the correlation matrix illustrates that several features can be dropped and may not be relevant for classification processes.

A similar visualization of the correlation matrix is presented in figure 5.8. The corresponding

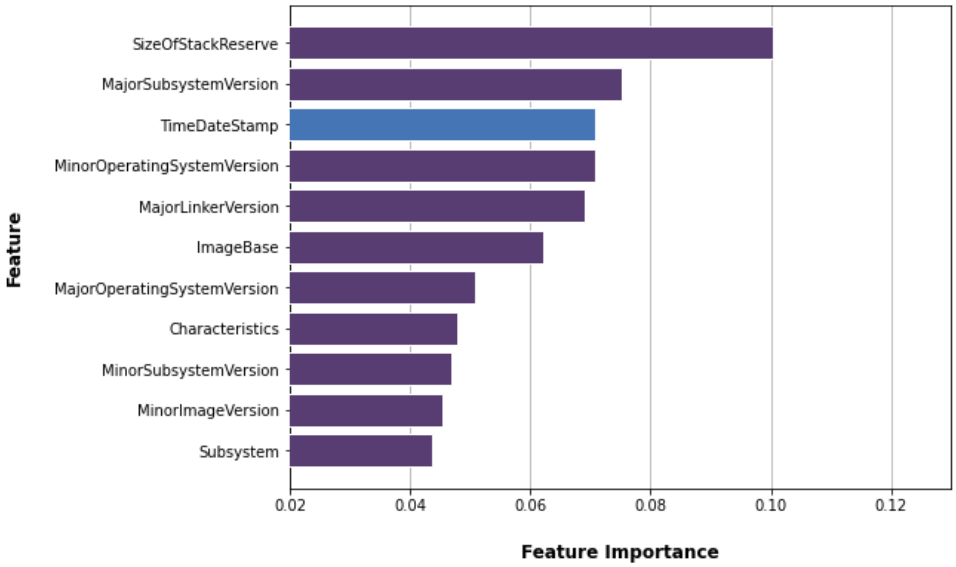


Figure 6.4.: Top critical features of the Kaggle dataset. Jointly features with the Virusshare dataset are colored-coded in purple. A complete list of the Feature Importance is illustrated in the appendix in figure E.1.

values are computed based on features from the Virusshare dataset. Unlike results from the Kaggle dataset, the matrix has more scattered correlating features. Positive correlating features can be found between *SectionMaxEntropy*, *SectionMeanEntropy*, *MinorOperatingSystem*, and *LoaderFlags*. Almost no correlations exist between features such as *MinorImageVersion* or *ResourcesMinSize*. *DLLCharacteristics* and *SectionsMeanEntropy*, on the other hand, have primarily negative correlations with *Characteristics* or *SubSystem*. The visualization of the Pearson correlation of the Virusshare dataset indicates that there are a few linear dependant features. However, in comparison to the Kaggle dataset, there are significantly fewer correlating features.

The visualization of the correlation between features demonstrated that several features of the Kaggle dataset might not be relevant for the classification process, which would prove the assumption that a large amount of features, in this case, has no advantage for an increased detection rate. This hypothesis is supported by the results obtained from the Feature Importance algorithm. Feature Importance is a built-in Random Forest algorithm, which determines which features are relevant for the classification process. The results of this algorithm are illustrated in figure 6.4 and 6.5.

The first figure 6.4 shows the top relevant features for the Random Forest classification process with the Kaggle dataset. The most significant feature was *SizeOfStackReserve*, with an importance of approximately 0.1. Furthermore, the feature *MajorSubsystemVersion* achieved an importance of 0.075. Both features are also present in the Virusshare dataset, with the most critical features presented in figure 6.5. The most significant feature for the classification

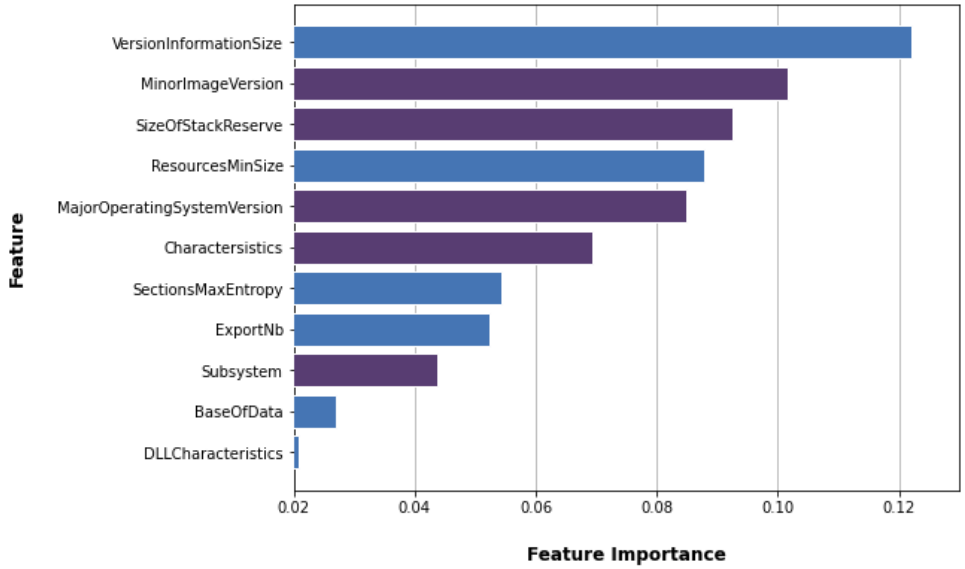


Figure 6.5.: Top critical features of the Virusshare dataset. Jointly features with the Kaggle dataset are colored-coded in purple. A complete list of the Feature Importance is illustrated in the appendix in figure F.1.

process with the Virusshare dataset was *VersionInformationSize*, followed by *MinorImageVersion*. With a value of 0.12, *VersionInformationSize* had higher importance than the top feature of the Kaggle dataset. However, the feature *SizeOfStackReserve* achieved in both datasets similar results. A review of features confirms that *VersionInformationSize* is not a feature included in the Kaggle dataset. Therefore, the dataset is likely to miss a critical feature for the classification process, relying more on unfavorable features. In addition, other essential features like *ResourcesMinSize*, *SectionsMaxEntropy*, *ExportNb*, *BaseOfData*, and *DLLCharacteristics* do not exist in the Kaggle dataset. Whereas, *TimeDateStamp* is not included in the Virusshare dataset.

The evaluation of Feature Importance has shown that the Kaggle dataset is missing critical features, even though it contains more features than the Virusshare dataset. Furthermore, the Virusshare had more meaningful features, with the top five features ranging between 0.08 and 0.12. With the results obtained from the classification process with the Virusshare dataset and the results from the Feature Importance evaluation, it must be considered that the missing features from the Kaggle dataset can be associated with a more inaccurate detection rate.

6.2.2. Feature Analysis

The previous section presented critical features for the classification process with both datasets. It is conspicuous that many of the top essential features are represented with similar weights in each dataset. Therefore, some of those significant features are analyzed and compared.

The first observed feature is *VersionInformationSize* which is the most significant feature in the Virusshare dataset. The distribution of values for the feature is illustrated in figure 6.6.

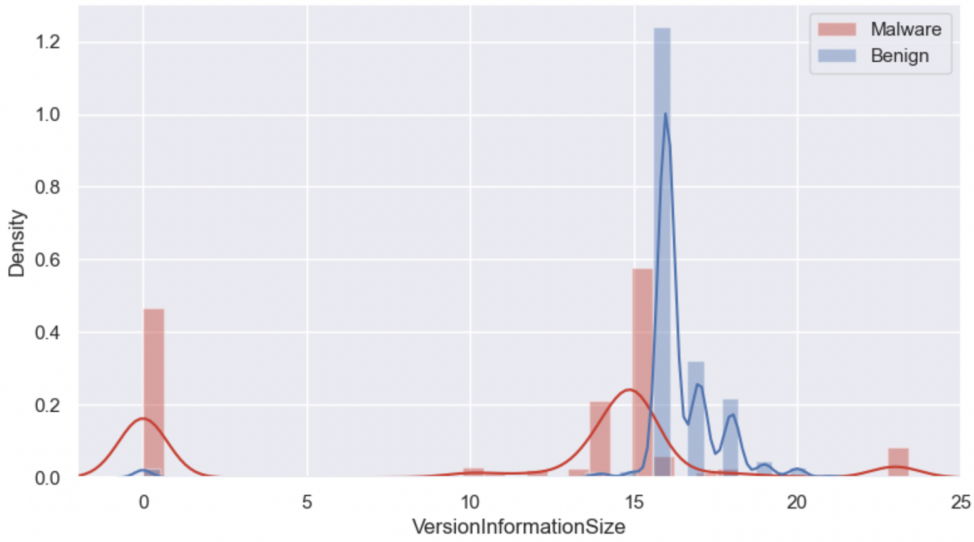
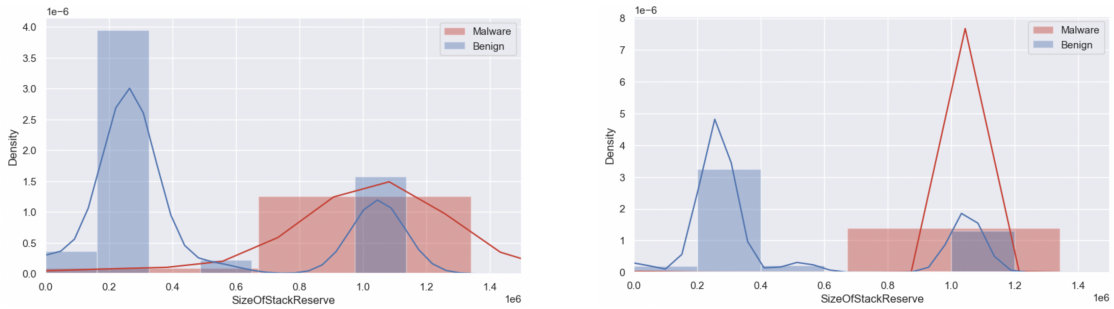


Figure 6.6.: Distribution of *VersionInformationSize*

The figure shows the density of values for malware and benign samples. Malware samples are color-coded in red and benign in blue, respectively. The plot indicates that malware samples with the feature *VersionInformationSize* have values in the range between 0 and 24. The density of malware samples illustrates that most values are approximately either 1 or 15. The span for benign samples with the feature *VersionInformationSize* is without consideration of outliers between 15 and 20. The highest density for benign samples is at approximately 16. The evaluation of the feature *VersionInformationSize* presented an excellent contrast between values from malicious and benign samples. This feature contains information such as the intended operating system and file version. Therefore, it can be assumed that most malware samples don't specify or obfuscate the version or operating system. Benign samples, on the other hand, define this resource.

A similar distribution is presented in figure 6.7. The figure shows the distribution of *SizeOfStackReserve*, which determines how much memory is reserved for the image.

Figure 6.7a and 6.7b show a similar distribution. Benign samples have the highest density at approximately $0.3 \cdot 10^6$. On the other hand, malware samples range between $0.7 \cdot 10^6$ and $1.3 \cdot 10^6$. However, the figures illustrate that some benign values are also in the region between $1 \cdot 10^6$ and $1.2 \cdot 10^6$. This observation indicates that no assertion can be made for samples with values in that range, whether the sample is benign or malware. Nevertheless, samples with values below $0.7 \cdot 10^6$ can indicate that the given sample is benign, with a high possibility when the value is approximately $0.3 \cdot 10^6$.



(a) Distribution of *SizeOfStackReserve* of the Kaggle dataset

(b) Distribution of *SizeOfStackReserve* of the Virusshare dataset

Figure 6.7.: Distribution of *SizeOfStackReserve*

The evaluation of the distribution of *SizeOfStackReserve* displays that it can mainly be distinguished between values from malware and benign samples. Therefore, an unknown sample can principally be associated with the correct label. However, the feature is irrelevant for values in the range of the span mentioned above. The distribution of malware samples concerning the feature *SizeOfStackReserve* illustrates that malware samples mostly reserve more memory for the image.

The final evaluated feature is *TimeDateStamp*. This feature represents the time and date the developer created the file. Based on the Feature Importance algorithm, the feature is critical for the classification process with the Kaggle dataset. The distribution of *TimeDateStamp* is presented in figure 6.8.

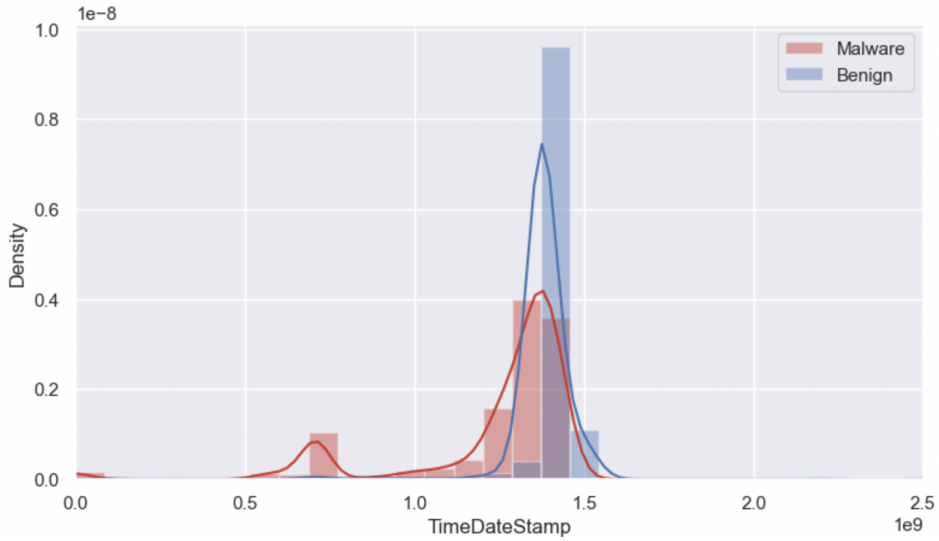


Figure 6.8.: Distribution of *TimeDateStamp*

The figure shows that the distribution of malware and benign samples for this feature are relatively similar. Values of malware samples range between $0.5 \cdot 10^9$ and $1.4 \cdot 10^9$, whereas

benign samples have a shorter span of $1.3 \cdot 10^9$ and $1.6 \cdot 10^9$. With a density of nearly $1 \cdot 10^{-8}$, the distribution for benign samples is very low. On the other hand, malware samples achieve the highest density of $0.4 \cdot 10^{-8}$ at $1.3 \cdot 10^9$. This evaluation illustrates that benign samples were relatively more current than malware samples.

A similar comparison was made by observing the distribution of the feature *VersionInformationSize*. Hence it can be assumed that malware samples are more likely to have outliers, or in other words, unknown samples with values that do not fall in the span of benign samples are probably labeled as malware. The evaluation of the three features has shown that classifiers should differentiate well between malware and benign samples. There were overlaps in values for both samples; however, these are minimal, and considering the datasets contain at least 50 features, it can be assumed that false positives and negatives are marginal. The analysis of the features has also shown that each feature can contribute an essential part to the classification process. This clarification underlines the assumption that a classification process with a dataset not containing these features is more likely to perform worse. Hence, it was provided why classification results from the Kaggle dataset were relatively worse than results from the Virusshare dataset.

Recent PE malware detection research marginally eliminates features based on feature selection analysis but utilizes dimensionality reduction algorithms to eliminate variables based on the importance of explained variance. The results of this section show that essential features are necessary for excellent classification results. Therefore, these features should be an integral part of the dataset before reducing dimensionality. Damasevicius et al. [79] for example, used PCA to reduce the dimensionality but was missing several features described in section 6.2.1. Therefore, the classification results were, for the most part, relatively worse.

Results of this section indicate potential weaknesses of detection methods with these datasets. If the evaluation of the feature *TimeDateStamp* is inspected, it has to be acknowledged that the dataset is limited in time. The dataset must permanently receive new samples to train the classifiers for the current application since malware is constantly being developed. New malware contains characteristics that differ from old malware. This assessment is also applicable to benign samples. For example, a benign sample from 2014 has different values of *TimeDateStamp* and *MajorOperatingSystemVersion* than a sample from 2020. Furthermore, advanced obfuscation techniques are reasons for updating these datasets. Additionally, adding new samples to old samples requires a higher computation time, assuming older samples are not deleted. Hence, classifiers like Random Forest, MLP, and KNN are essential for the future training process with this framework.

6.3. Comparison with other Approaches

The following section focuses on the comparison of the performance of the classification results with related approaches. Each approach is based on the classification of benign and malware files, utilizing various datasets. The work of Karbab, for example, uses API method sequences to detect and classify malware. The comparison of performance is presented in table 6.3.

Reference	Accuracy	Precision	Recall	F1-score	AUC
This thesis	**0.9935**	**0.9912**	**0.9933**	**0.9922**	**0.9995**
Azmee et al.(2020) [80]	0.986	0.98	0.99	0.98	0.996
Vinayakuma et al.(2019) [55]	0.963	0.963	0.962	0.962	N/A
Raff et al.(2018) [51]	0.940	N/A	N/A	N/A	0.981
Hou et al.(2017) [81]	0.9666	0.9655	0.9667	0.9666	N/A
Karbab et al.(2017) [82]	N/A	0.9629	0.9629	0.9629	N/A

Table 6.3.: Comparison of performance with related work

The table shows that classification results from this thesis outperformed other existing approaches. Classification results from Azmee et al. with the Kaggle dataset achieved comparable results. Utilizing the XGBoost classifier, Azmee et al. obtained an accuracy of 0.986. A similar performance reached Random Forest with 0.981. Considering that the classification results from XGBoost are very high and the classifier has been used in recent works, it can be established that this classifier is utilized for future experiments.

The comparison of performance with related work illustrates that classification results continuously improve. Additionally, the analysis of previous approaches reveals that the latest work utilizes multiple classifiers consisting of machine learning and neural network models. In contrast, older methods used one or a few classifiers, for example, Raff et al.. With the implementation of multiple classifiers to detect malware in the latest studies, it can be examined which classifiers are most favorable for future processes. A pattern of classification performance based on malware detection can already be recognized. For example, Random Forest achieved excellent results in this thesis, whereas Random Forest obtained the second-best results in the approach from Azmee et al.. Another example is Vinayakuma et al., where Random Forest achieved excellent from all machine learning classifiers, and CNN + LSTM obtained the best results.

6.4. Evaluation of Keras Tuner

In the following section, the performance of Keras Tuner is evaluated. The evaluation process compares classification results obtained with and without Keras Tuner. Additionally, the

computation time for all trials is analyzed and compared.

The classification results with and without Keras Tuner and the Kaggle dataset are illustrated in table 6.4. Evaluation metrics with the highest performance are marked in bold.

Classifier	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9819	0.9814	0.9708	0.9760	**0.9963**	**6.15**
RF KT	**0.9834**	**0.9831**	**0.9732**	**0.9780**	0.9957	24.39
KNN	0.9768	0.9711	0.9677	0.9694	0.9849	0.02
KNN KT	**0.9786**	**0.9737**	**0.9699**	**0.9718**	**0.9866**	**0.01**
SVM	0.9676	0.9644	0.9498	0.9568	0.9798	8.74
SVM KT	**0.9796**	**0.9798**	**0.9663**	**0.9728**	**0.9888**	**7.19**
MLP	0.9819	0.9804	0.9718	0.9760	0.9927	**9.31**
MLP KT	**0.9845**	**0.9824**	**0.9765**	**0.9794**	**0.9938**	23.06
DNN	0.9758	**0.9791**	0.9884	0.9836	**0.9868**	**7.41**
DNN KT	**0.9773**	0.9790	**0.9907**	**0.9847**	0.9784	7.75

Table 6.4.: Comparison of performance with Keras Tuner and the Kaggle dataset

A first observation indicates that most models utilizing Keras Tuner outperform models without Keras Tuner. Random Forest, for example, has higher accuracy, precision, recall, and F1-score with Keras Tuner. However, the computation time has quadrupled from 6.15 to 24.39. The higher computation time is due to the hyperparameter *n_estimators*, which is has been set to 170. The default value for this parameter is 100. The comparison of classification results from Random Forest with Keras Tuner supports the assessment made by Probst et al. [70]. The evaluation of Tuning Strategies from Probst et al. specifies that Random Forest is less tunable than other classifiers. With a difference of 0.0015 in accuracy, the performance gained with Keras Tuner is minimal but still reasonable.

The highest performance gained with Keras Tuner achieved SVM. The difference between accuracy results is 0.0120, which is a remarkable improvement in performance. The regularization parameter *C*, which has a default value of 1.0, was calculated with Keras Tuner to be set to 11.0. Additionally, the *gamma* parameter was set to 0.05. The hyperparameter calculation results resemble the results of Clare Liu [74], which were based on the Iris dataset.

The comparison of the DNN shows that hyperparameter tuning only partially increased the performance. Keras Tuner calculated that *ReLU* is the optimal activation function for the classifier. Additionally, the optimal units for the first three layers are 60.

Overall the improvements of performance with Keras Tuner are excellent. Every model had at least three evaluation metrics that increased in performance. It has to be remarked that the optimization of parameters is sometimes combined with a higher computation time,

which can cause issues with large datasets. Additionally, the results of the DNN show that it can be challenging to find optimal parameters for tuning, especially when the classifier has numerous parameters. Since previous work based on PE malware detection barely focused on hyperparameter tuning, this work relies on different datasets' tuning results. Table 6.5 shows a similar comparison of performance with the Virusshare dataset.

Classifier	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9931	0.9907	0.9928	0.9918	**0.9996**	54.64
RF KT	**0.9935**	**0.9912**	**0.9933**	**0.9922**	0.9995	**50.51**
KNN	0.9906	0.9878	0.9898	0.9888	0.9973	**0.02**
KNN KT	**0.9925**	**0.9902**	**0.9921**	**0.9911**	**0.9980**	0.03
SVM	0.9890	0.9870	0.9866	0.9868	0.9974	**472.93**
SVM KT	**0.9921**	**0.9899**	**0.9913**	**0.9906**	**0.9981**	933.23
MLP	0.9900	0.9878	0.9885	0.9881	0.9990	**39.38**
MLP KT	**0.9930**	**0.9909**	**0.9925**	**0.9917**	**0.9992**	117.96
DNN	0.9884	0.9732	0.9882	0.9804	0.9985	**48.32**
DNN KT	**0.9898**	**0.9760**	**0.9901**	**0.9828**	**0.9987**	48.75

Table 6.5.: Comparison of performance with Keras Tuner and the Virusshare dataset

The table shows that almost all classifiers with tuned hyperparameters improved in performance. The assertion that Random Forest is less tunable than other classifiers can be confirmed based on the results. Additionally, the results from Random Forest without Keras Tuner are already very close to a perfect classification, hence tuning hyperparameters for this classifier are marginally beneficial. An intriguing remark is the comparison of computation time for Random Forest. In the comparison illustrated above, Random Forest with Keras Tuner needed quadruple computation time. However, Random Forest with Keras Tuner was faster by approximately four seconds in this comparison. The fast computation time is based on the calculated parameter *n_estimator* and *max_features*, set to 110 and 6, respectively.

The hyperparameters for SVM *C* and *gamma* were calculated to be set to 20 and 0.1, respectively. These settings led to an increase in performance. However, the tuned hyperparameters boosted the already high computation time to 933.23 seconds.

Excellent results were achieved by tuning the hyperparameters of MLP. The classifier was highly tunable, with a difference of 0.003 in accuracy and 0.004 in recall. The hyperparameters tuned and responsible for the increase of performance for MLP were *activation* and *batch_size*. Keras Tuner calculated that the optimal activation function was the hyperbolic tan functioning. The default activation function for MLP is the rectified linear unit function. Additionally, the optimal *batch_size* was computed to be set to 310.

The assertions based on the performance comparison with the Kaggle dataset are reinforced with the same evaluation with the Virusshare dataset. Hyperparameters of Random Forest can be well tuneable. However, the evaluation has shown that the increase in performance is marginal. Furthermore, if parameters like *n_estimator* and *max_features* are set to be very high, the computation of Keras Tuner can take very long. Tuning Random Forest is therefore limited to an extend. Like KNN, SVM, and MLP, other classifiers seem more tunable. MLP, for example, provided excellent results with three tuned hyperparameters. Assuming the computation time is not considered, SVM improves very well in performance with the application of Keras Tuner.

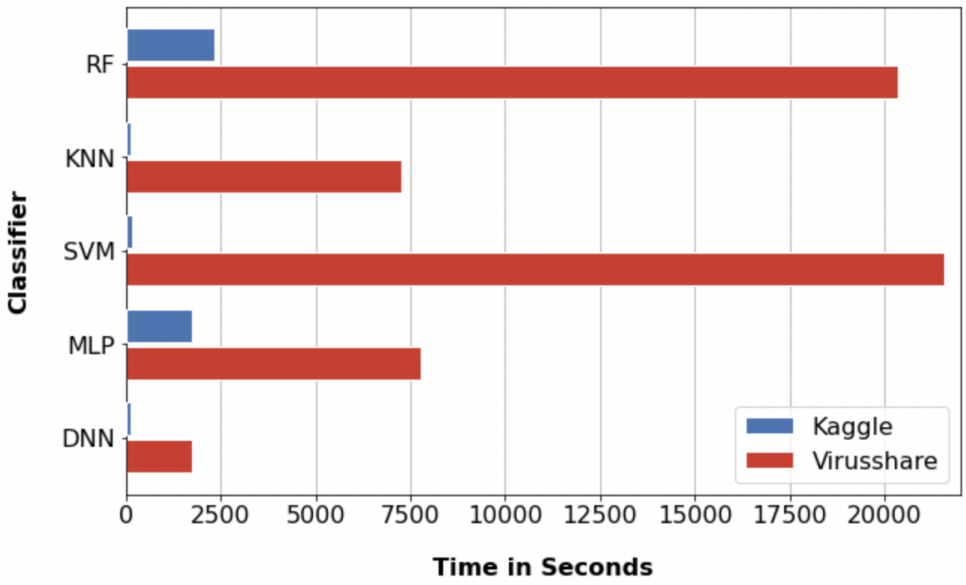


Figure 6.9.: Total elapsed time for the tuning process

However, the computation time of Keras Tuner can be a decisive factor to assert if a classifier needs to be tuned with the selected parameters. Since Keras Tuner is applied on both datasets, it can be derived how this tuning process behaves with datasets of different sizes. The hyperparameters of the classifiers are configured for each dataset the same. The computation time of the tuning process is illustrated in figure 6.9.

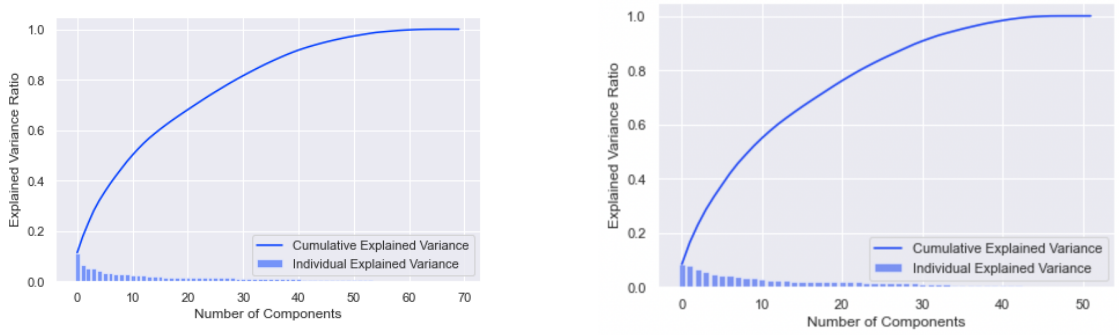
The figure shows that the computation time of Keras Tuner was very high for the classifiers Random Forest and SVM with the Virusshare dataset. The tuning process with both classifiers required nearly six hours. Like KNN and MLP, other classifiers had a comparatively low tuning time of two hours. DNN had the lowest computation time of approximately 50 seconds.

The evaluation of the comparison of performance and the tuning computation time illustrates that Keras Tuner is an excellent hyperparameter tuning framework. Significantly MLP and KNN improved in performance. Tuning hyperparameters with Random Forest was partially successful, the performance improvement was limited, and the computation time was simultaneously high. The optimization of the DNN worked very well, however with the utilization of more tunable

parameters such as *batch_size*, the results could have improved. Tuning hyperparameters of SVM with Keras Tuner can lead to excellent results. Nevertheless, the tuning process is dependant on the size of the dataset. The results of this section recommend Keras Tuner as a good tuning framework, which can be utilized for future work. However, the framework's execution can be challenging when dealing with large datasets and specific parameters.

6.5. Evaluation of PCA

In this section, the feature reduction algorithm PCA is evaluated based on the framework's performance. Figure 6.10 shows the cumulative and individual explained variance of both datasets.



(a) Cumulative and individual explained variance of the Kaggle dataset

(b) Cumulative and individual explained variance of the Virusshare dataset

Figure 6.10.: PCA of the Kaggle and Virusshare dataset

Figure 6.10a illustrates that approximately 50 components explain 100% of the variance. Additionally, ten components describe 50% of the dataset. From initially 79 features, the number of components for the classification process were reduced to 50. For future work, the number of features can be reduced to 30, explaining approximately 80% of the variance.

A similar approach was chosen for the Virusshare dataset, with cumulative and individual explained variance presented in figure 6.10b. The dataset initially had 57 features, with PCA reduced to 50. The number of components could have been reduced to 40, describing approximately 100% of the variance. However, for better comparability between datasets and classifiers, both datasets had equal quantities of components.

Observing figure 6.10b, it can be determined that 20 components describe approximately 80% of the variance. On the other hand, PCA with the Kaggle dataset required 30 components to explain the same variance. This evaluation underlines the conclusion that the Kaggle dataset has more irrelevant features, whereas features from the Virusshare dataset are more suitable for classification.

Classifiers with and without PCA were trained and tested to evaluate PCA with this framework. The results of those experiments are presented in table 6.6. Classifiers trained without PCA are labeled with WPCA.

Classifier	Accuracy	Precision	Recall	F1-score	AUC	Time
RF	0.9819	0.9814	0.9708	0.9760	0.9963	6.15
RF WPCA	**0.9903**	**0.9925**	**0.9820**	**0.9871**	**0.9985**	**2.02**
KNN	0.9768	0.9711	0.9677	0.9694	0.9849	0.02
KNN WPCA	**0.9781**	**0.9730**	**0.9692**	**0.9711**	**0.9851**	**0.02**
SVM	0.9676	0.9644	0.9498	0.9568	0.9798	**8.74**
SVM WPCA	**0.9679**	**0.9646**	**0.9503**	**0.9572**	**0.9799**	16.86
MLP	**0.9819**	0.9804	**0.9718**	**0.9760**	**0.9927**	**9.31**
MLP WPCA	0.9816	**0.9806**	0.9710	0.9756	0.9907	11.98
DNN	0.9758	**0.9791**	0.9884	0.9836	0.9868	**7.41**
DNN WPCA	**0.9776**	0.9782	**0.9917**	**0.9848**	**0.9873**	8.42

Table 6.6.: Comparison of performance with PCA and the Kaggle dataset

The table shows that some classifiers improved in performance with dimensionality reduction. A few classifiers, namely Random Forest and KNN, performed better without PCA. The main goal of the implementation of PCA was to improve the computation time of the models and transform correlated features into a smaller set of uncorrelated components. This goal is only partially achieved, as the computation time for Random Forest was noticeably higher with PCA. There are two main reasons why Random Forest performed worse with PCA:

- ***criterion***
  The parameter measures the quality of the split [83]. During the execution of Random Forest within each iteration, the best split is picked based on the criteria. The classifier might have more issues finding the right splits by reducing the number of components. The algorithm, therefore, needs more iterations, hence a longer computation time.

- ***max_features***
  Random Forests built-in parameter reduces the number of features when looking for the best split. However, the number of features were already reduced with PCA. Since the default value for this parameter is $sqrt$, the features are reduced by $sqrt(50)$ instead of $sqrt(79)$.

The evaluation of PCA indicates that the dimensionality reduction algorithm can be beneficial for a future extension of the framework. It is, however, sensible to some classifiers; hence PCA

is not always necessary. MLP, SVM, and DNN primarily benefited from the implementation of PCA. Therefore, the framework structure with the feature reduction component was principally successful. Nevertheless, some framework components will be modified and subsequently illustrated in the future work section.

In the summary of the results section, the assertion was made that Random Forest is the best classifier for this framework. However, the evaluation of PCA shows that Random Forest is not optimal with this framework pipeline. Considering the discussion in this section and the review of Keras Tuner 6.4, it has to be reconsidered if Random Forest is the best classifier for this detection framework. Since MLP ranks closely behind Random Forest and performs equally well with the Kaggle dataset, MLP can be considered the optimal classifier for this framework. This assessment is supported by the performance of MLP with Keras Tuner and PCA. Keras Tuner optimized the Accuracy of MLP by approximately 0.003, whereas Random Forest was marginally tunable. Furthermore, the integration of PCA negatively impacted the training time of Random Forest.

Determining which classifier is best for detecting malware depends on the condition. Considering the synergy with this framework, MLP is the optimal model for this detection problem using PCA and Keras Tuner. Although it performs worse in some evaluation metrics than Random Forest, the adaptability concerning principal components of PCA and hyperparameter optimization with Keras Tuner characterize it as an excellent classifier. Furthermore, it is flexible, has an acceptable computation time, is well customizable, and is uncomplicated to implement with the scikit-learn library. One primary reason why MLP is the optimal classifier for this framework is that it provides fast predictions after training, making it an ideal model for real-time application. Characteristics of MLP's and recent literature support this assertion [84].

In a situation where PCA and Keras Tuner are not implemented, Random Forest would be the best malware detector. However, it is not as adaptable as MLP. Hence when dealing with a large number of trees in a Random Forest, the prediction is remarkably slow. This is a negative factor for a real-time detection application.

7. Reviewing the Malware Detection Framework

This section summarizes the research work based on the evaluated and discussed findings. After concluding the discoveries of this work, potential improvements of this framework are highlighted in the subsequent section.

7.1. Conclusion

In previous sections, it was illustrated that current AV engines use signature-based and heuristic-based methods to detect malware. However, signature-based and heuristic-based methods are limited due to increasingly new malware variants and data storage advancements. Therefore, modern research relies on automated learning to detect possible threats. The statistics of Virustotal support this process [26]. The most submitted file type on Virustotal is portable executable, with approximately seven million submissions in the last seven days. This observation indicates that portable executable malware detection is a current problem and an intelligent detection algorithm is essential for future anti-malware systems. A prototype can be obtained and integrated to detect other file types after successfully implementing a portable executable malware detection framework. Therefore, the results of this framework are not exclusively bound to portable executable files.

This research work presents an intelligent malware detection approach that classifies samples based on static features. The algorithm features three machine learning and two deep learning models trained and evaluated with two datasets. These datasets have distinct properties. The most significant difference is the size of the samples. Additionally, the features are reduced to a minimal subset with PCA, and the model's hyperparameters are tuned with Keras Tuner. The selection of parameters for tuning is based on own experiments and optimization research. This research work aims to determine optimal classification models for practical usability. Thus, machine learning models are compared to deep learning classifiers concerning datasets with different properties. The results are then compared to related approaches, and the performance of Keras Tuner is evaluated. Furthermore, the performance of PCA with the framework is analyzed, and an optimized integration is proposed.

The results obtained from the framework indicate that Random Forest and MLP achieve the best performance concerning both datasets. Random Forest reached an accuracy of 0.9935 and

MLP an accuracy of 0.9930, outperforming all other classifiers. Without the integration of PCA, Random Forest is the best model, achieving an accuracy of 0.9944. With the integration of PCA and Keras Tuner, MLP is the best performing model, especially considering the adaptability and flexibility. Therefore, machine learning and deep learning models achieve comparable results.

Additionally, evaluation results provide that a deep learning solution, namely DNN for malware detection, is well realizable but requires more fine-tuning in future work. The framework is, in principle, very successful, with all classifiers achieving an accuracy above 95%. The comparison with past work has shown that other classifiers could be considered for the framework in the future, namely XGBoost. Furthermore, the implementation of Keras Tuner is primarily successful, optimizing the performance of every classifier in almost every evaluation metric. The MLP and SVM classifiers benefited the most from the Keras Tuner library, improving accuracy by approximately 0.003.

By comparing feature importance between the Kaggle and Virusshare datasets, future feature engineering processes are implied, improving the classification of malicious portable executables. The results show that essential features, for example, *VersionInformationSize* and *TimeDateStamp*, are not included in both datasets, impacting classification performance. The advantage of this framework is that it is well customizable, reducing the computation time with PCA, thus, if desired, providing a real-time detection system. Additionally, the results of Keras Tuner with this framework were evaluated and discussed, revealing the shortcomings and strengths of this library. An optimized framework for future work is provided, which will be adapted based on the obtained results and illustrated in the next section.

7.2. Future Work

This section presents possible future work. This thesis offers many extensions in combination with the utilized framework and tools. Additionally, this chapter will suggest possible alternatives for tools and datasets. This section is structured as follows. First, future work with other hyperparameter optimization libraries is illustrated. Then, a potential framework extension based on feature selection is presented. The last section covers the possible future work on mobile devices.

7.2.1. Hyperparameter Optimization Libraries

The hyperparameter optimization library utilized in this thesis, Keras Tuner, is one out of multiple hyperparameter optimization tools. Optuna is an open-source framework introduced in 2019 by Akiba et al. [85] to automate hyperparameter search. The framework offers easy parallelization and quick visualization and could be a good alternative for the optimization framework used in this thesis [86]. Furthermore, Ray Tune is a library using distributed computing power for advanced hyperparameter optimization [87]. The Ray Tune python library is a good alter-

native since it offers parallelization across multiple GPUs and cloud support. Possible future work could involve implementing various hyperparameter optimization frameworks and testing them on the proposed malware detection framework. Since the tools offer different features, resulting in advantages and disadvantages when used on specific frameworks, the differentiation results are fundamental for future malware detection approaches utilizing hyperparameter tuning frameworks. The tools mentioned above are merely examples; Scikit-Optimize and Hyperopt are furthermore good frameworks for hyperparameter optimization.

7.2.2. Framework Extension

Recent publications based on malware detection utilize various frameworks to achieve the optimal detector. The framework demonstrated in this thesis uses two deep learning models to train and test data for malware detection. A considered expansion of this framework would be a one-dimensional CNN model. Although a CNN model is mainly designed for image processing, the approach from Raff et al. [51] demonstrated that a one-dimensional CNN could retrieve good results for malware recognition. This approach can be considerably challenging for the featured framework because the preprocessing step differentiates from the preprocessing step used in this framework.

Another extension of this framework would be the supplementation of a deep neural network with additional hidden layers. The research focused on this approach is sparse and would provide interesting results. Furthermore, future researchers can expand the framework with multiple deep neural networks containing various amounts of neurons and hidden layers. Analyzing why the specific model performed better than other deep learning approaches could extend this topic.

Furthermore, the results of this thesis have shown that specific machine learning models perform better for this classification problem. The integration of XGBootst to extend this framework could provide numerous beneficial results. Additionally, the number of classifiers used for this framework can be reduced to a subset of models that provide the best performance. Hence, the focus can be on a few well-performing models.

The final extension of the framework could include implementing another deep learning model, recurrent neural networks. During this thesis's research, Brownlee [88] discusses the advantages and disadvantages of RNNs and for which data they are optimal. Since RNNs are designed for sequence prediction problems, they might not be optimal for the framework proposed in this thesis. Nevertheless, they could be helpful for malware recognition, provided that the data is encoded or transformed into a sequential form. An extension could introduce an RNN in this framework and adapt the data preprocessing step. Additionally, the RNN would be analyzed, evaluated, and compared to the already implemented approaches.

7.2.3. Feature Selection

Feature selection is an essential step in the proposed malware detection framework. The main objective of the feature selection and reduction algorithm is to select a minimal set of features. The proposed models require fewer resources to analyze the given file with the reduced features. Furthermore, feature reduction can optimize the training process and eliminate outliers. A minimal set of features can improve the proposed models' accuracy, training, and detection.

The presented framework features the feature reduction algorithm PCA. PCA is a prevalent technique commonly used for feature reduction based on dense data [89]. Future work can consist of experimenting with different dimensionality reduction algorithms. Singular Value Decomposition (SVD) is a common technique optimal for sparse data. Furthermore, future research can utilize Linear Discriminant Analysis (LDA) for feature reduction. Using the proposed framework, the algorithms present a subset of dimensionality reduction algorithms that can achieve better results than PCA. Comparing the different dimensionality reduction algorithms based on the framework results can gain important insights.

Based on the analysis of the particular feature, manual feature reduction is possible with the publication from Raman et al. [58]. The paper presents a minimal set of features that can be used as input for the malware detection framework. Similar publications can be analyzed and utilized in future work to experiment with the given framework. A discussion comparing results from a minimal feature set, using a feature reduction algorithm, and outcomes based on features from recent publications can supply important aspects for optimizing the proposed framework.

The feature analysis of the Virusshare dataset has shown that it contained meaningful features for the classification process. Some of these features were not in the Kaggle dataset; hence performance was worse. Future work can extract data for a new dataset based on the features evaluated in this thesis. This dataset can then be utilized for the extended framework, improving classification results.

7.2.4. Mobile Application

Malware detection on mobile devices has been discussed in the recent work section. The advantage of this framework is that it is not bound to portable executable malware. Hence, this framework can apply to android or IOS malware. The number of features extracted from the mobile Android Package (APK) resembles the number utilized in this classification problem. An adjustment of this framework to detect malware samples is therefore minimal.

Additionally, the results from Kouliaridis et al. [90] can support and optimize the implementation of this framework with android malware. Furthermore, the framework can be extended to detect different types of android malware.

Bibliography

[1] PurpleSecLLC, "2021 cyber security statistics the ultimate list of stats, data and trends," 2020, last accessed: 02.05.2021. [Online]. Available: https://purplesec.us/resources/cyber-security-statistics/

[2] ForgeRock, "2020 forgerock consumer identity breach report," 2020, last accessed: 13.05.2021. [Online]. Available: https://www.forgerock.com/resources/2020-consumer-identity-breach-report

[3] J. Firch, "10 cyber security trends you cannot ignore in 2021," 2020, last accessed: 02.05.2021. [Online]. Available: https://purplesec.us/cyber-security-trends-2021/

[4] K. A. Monnappa, *Learning Malware Analysis*. Packt, 2018. [Online]. Available: https://www.packtpub.com/product/learning-malware-analysis/9781788392501

[5] Kaspersky, "A brief history of computer viruses and what the future holds," 2020, last accessed: 15.05.2021. [Online]. Available: https://www.kaspersky.com/resource-center/threats/a-brief-history-of-computer-viruses-and-what-the-future-holds

[6] H. S. Joshua Saxe, *Malware Data Science*. No Starch Press, 2018. [Online]. Available: https://www.buecher.de/shop/netzwerksicherheit/malware-data-science/saxe-joshua-sanders-hillary/products_products/detail/prod_id/50442583/

[7] K. Iliev, "Top 6 advanced obfuscation techniques hiding malware on your device," 2017, last accessed: 15.05.2021. [Online]. Available: https://sensorstechforum.com/advanced-obfuscation-techniques-malware/

[8] J. Olekss, "An introduction to code obfuscation," 2020, last accessed: 14.05.2021. [Online]. Available: https://www.intertrust.com/blog/an-introduction-to-code-obfuscation/

[9] P. Putman, "Script kiddie: Unskilled amateur or dangerous hackers?" 2019, last accessed: 14.05.2021. [Online]. Available: https://www.uscybersecurity.net/script-kiddie/

[10] T. Bottesch, "Evolutionary antivirus," 2014, last accessed: 17.05.2021. [Online]. Available: https://www.avira.com/en/blog/evolutionary-antivirus

[11] Bricata, "Layers of cybersecurity: Signature detection vs. network behavioral analysis," 2017, last accessed: 17.05.2021. [Online]. Available: https://bricata.com/blog/signature-detection-vs-network-behavior/#:~:text=Signature%2Dbased%20detection%20is%20a,of%20a%20known%20bad%20file.

[12] Y. Hollander, "Behavioral rules vs. signatures: Which should you use?" 2003, last accessed: 20.05.2021. [Online]. Available: https://www.computerworld.com/article/2581345/behavioral-rules-vs--signatures--which-should-you-use-.html

[13] Kaspersky, "What is heuristic analysis?" 2017, last accessed: 14.05.2021. [Online]. Available: https://usa.kaspersky.com/resource-center/definitions/heuristic-analysis

[14] R. Bellairs, "What is static analysis (static code analysis)?" 2020, last accessed: 15.05.2021. [Online]. Available: https://www.perforce.com/blog/sca/what-static-analysis

[15] Kaspersky, "https://usa.kaspersky.com/resource-center/definitions/heuristic-analysis," 2017, last accessed: 13.05.2021. [Online]. Available: https://usa.kaspersky.com/resource-center/definitions/heuristic-analysis

[16] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, "Dynamic malware analysis in the modern era a state of the art survey," *ACM Comput. Surv.*, vol. 52, no. 5, sep 2019. [Online]. Available: https://doi.org/10.1145/3329786

[17] R. A. Grimes, *Malware Code Analysis*. Berkeley, CA: Apress, 2005, pp. 337–361. [Online]. Available: https://doi.org/10.1007/978-1-4302-0007-9_12

[18] R. Shoemake, "Malware analysis using memory forensics," 2018, last accessed: 15.05.2021. [Online]. Available: https://www.secjuice.com/malware-analysis-memory-forensics/

[19] A. A. Mawgoud, H. M. Rady, and B. S. Tawfik, *A Malware Obfuscation AI Technique to Evade Antivirus Detection in Counter Forensic Domain*. Cham: Springer International Publishing, 2021, pp. 597–615. [Online]. Available: https://doi.org/10.1007/978-3-030-52067-0_27

[20] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *Journal of Network and Computer Applications*, vol. 153, p. 102526, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804519303868

[21] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, L. Invernizzi *et al.*, "Kerastuner," 2019, last accessed: 21.12.2021. [Online]. Available: https://github.com/keras-team/keras-tuner

[22] Infosec, "Static malware analysis," 2015, last accessed: 29.12.2021. [Online]. Available: https://resources.infosecinstitute.com/topic/malware-analysis-basics-static-analysis/

[23] K. Baker, "Malware analysis," 2020, last accessed: 29.12.2021. [Online]. Available: https://www.crowdstrike.com/cybersecurity-101/malware/malware-analysis/

[24] ——, "The 11 most common types of malware," 2021, last accessed: 29.12.2021. [Online]. Available: https://www.crowdstrike.com/cybersecurity-101/malware/types-of-malware/

[25] S. Khillar, "Difference between static malware analysis and dynamic malware analysis," 2018, last accessed: 13.12.2021. [Online]. Available: http://www.differencebetween.net/technology/difference-between-static-malware-analysis-and-dynamic-malware-analysis/

[26] G. Sood, *virustotal: R Client for the virustotal API*, 2021, last accessed: 27.12.2021.

[27] DelphiBasics, "An in-depth look into the win32 portable executable file format - part 1," 2010, last accessed: 15.05.2021. [Online]. Available: https://www.delphibasics.info/home/delphibasicsarticles/anin-depthlookintothewin32portableexecutablefileformat-part1

[28] J. Plachy, "Portable executable file format," 2018, last accessed: 15.05.2021. [Online]. Available: https://blog.kowalczyk.info/articles/pefileformat.html

[29] K. Bridge, "Pe-format," 2021, last accessed: 13.12.2021. [Online]. Available: https://docs.microsoft.com/de-de/windows/win32/debug/pe-format#references

[30] Wikipedia contributors, "File:portable executable 32 bit structure.png — wikimedia commons, the free media repository," 2020, last accessed: 15.05.2021. [Online]. Available: https://commons.wikimedia.org/w/index.php?title=File:Portable_Executable_32_bit_Structure.png&oldid=443505829

[31] A. Géron, K. Rother, and T. Demmig, *Praxiseinstieg Machine Learning mit Scikit-Learn, Keras und TensorFlow: Konzepte, Tools und Techniken für intelligente Systeme (Aktuell zu TensorFlow 2)*. Dpunkt.Verlag GmbH, 2020. [Online]. Available: https://books.google.de/books?id=hUZqzQEACAAJ

[32] J. Steinwendner and R. Schwaiger, *Neuronale Netze programmieren mit Python*. Rheinwerk Computing, 2020. [Online]. Available: https://www.rheinwerk-verlag.de/neuronale-netze-programmieren-mit-python/

[33] Z. Jaadi, "A step-by-step explanation of principal component analysis (pca)," 2021, last accessed: 21.12.2021. [Online]. Available: https://builtin.com/data-science/step-step-explanation-principal-component-analysis

[34] L. Wuttke, "Was ist supervised learning (überwachtes lernen)?" 2020, last accessed: 16.12.2021. [Online]. Available: https://datasolut.com/wiki/supervised-learning/

[35] ——, "Was ist unsupervised learning (unüberwachtes lernen)?" 2020, last accessed: 16.12.2021. [Online]. Available: https://datasolut.com/wiki/unsupervised-learning/

[36] A. Bharath and S. Madhvanath, "A framework based on semi-supervised clustering for discovering unique writing styles," in *2009 10th International Conference on Document Analysis and Recognition*, 2009, pp. 891–895.

[37] D. Mwiti, "10 real-life applications of reinforcement learning," 2021, last accessed: 16.12.2021. [Online]. Available: https://neptune.ai/blog/reinforcement-learning-applications

[38] O. Mbaabu, "Introduction to random forest in machine learning," 2020, last accessed: 16.12.2021. [Online]. Available: https://www.section.io/engineering-education/introduction-to-random-forest-in-machine-learning/

[39] E. Sruthi, "Understanding random forest," 2021, last accessed: 16.12.2021. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/

[40] O. Harrison, "Machine learning basics with the k-nearest neighbors algorithm," 2018, last accessed: 16.12.2021. [Online]. Available: https://towardsdatascience.com/machine-learning-basics-with-the-k-nearest-neighbors-algorithm-6a6e71d01761

[41] S. Luber and N. Litzel, "Was ist eine support vector machine?" 2019, last accessed: 16.12.2021. [Online]. Available: https://www.bigdata-insider.de/was-ist-eine-support-vector-machine-a-880134/

[42] H. Singh, "Deep learning 101: Beginners guide to neural network," 2021, last accessed: 18.12.2021. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/03/basics-of-neural-network/

[43] J. Brownlee, "Softmax activation function with python," 2020, last accessed: 18.12.2021. [Online]. Available: https://machinelearningmastery.com/softmax-activation-function-with-python/

[44] C. Bento, "Multilayer perceptron explained with a real-life example and python code: Sentiment analysis," 2021, last accessed: 19.12.2021. [Online]. Available: https://towardsdatascience.com/multilayer-perceptron-explained-with-a-real-life-example-and-python-code-sentiment-/analysis-cb408ee93141

[45] Uniqtech, "Multilayer perceptron (mlp) vs convolutional neural network in deep learning," 2018, last accessed: 19.12.2021. [Online]. Available: https://medium.com/data-science-bootcamp/multilayer-perceptron-mlp-vs-convolutional-neural-network-in-deep-learning-c890f487a8f1

[46] J. Brownlee, "Hyperparameter optimization with random search and grid search," 2020, last accessed: 20.12.2021. [Online]. Available: https://machinelearningmastery.com/hyperparameter-optimization-with-random-search-and-grid-search/

[47] A. Bissuel, "Hyper-parameter optimization algorithms: a short review," 2019, last accessed: 20.12.2021. [Online]. Available: https://medium.com/criteo-engineering/hyper-parameter-optimization-algorithms-2fe447525903

[48] J. Brownlee, "How to implement bayesian optimization from scratch in python," 2019, last accessed: 20.12.2021. [Online]. Available: https://machinelearningmastery.com/what-is-bayesian-optimization/

[49] A. Agnihotri and N. Batra, "Exploring bayesian optimization," 2020, last accessed: 20.12.2021. [Online]. Available: https://distill.pub/2020/bayesian-optimization/

[50] J. Prost, "Hands on hyperparameter tuning with keras tuner," 2020, last accessed: 21.12.2021. [Online]. Available: https://www.sicara.ai/blog/hyperparameter-tuning-keras-tuner

[51] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, "Malware detection by eating a whole exe," 2018, last accessed: 03.05.2021.

[52] A. Souri and R. Hosseini, "A state-of-the-art survey of malware detection approaches using data mining techniques," *Human-centric Computing and Information Sciences*, vol. 8, pp. 1–22, 12 2018.

[53] D. Ucci, L. Aniello, and R. Baldoni, "Survey on the usage of machine learning techniques for malware analysis," *CoRR*, vol. abs/1710.08189, 2017. [Online]. Available: http://arxiv.org/abs/1710.08189

[54] W. Zhong and F. Gu, "A multi-level deep learning system for malware detection," *Expert Systems with Applications*, vol. 133, pp. 151–162, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417419303008

[55] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, and S. Venkatraman, "Robust intelligent malware detection using deep learning," *IEEE Access*, vol. 7, pp. 46 717–46 738, 2019.

[56] H. Rathore, S. Agarwal, S. K. Sahay, and M. Sewak, "Malware detection using machine learning and deep learning," *CoRR*, vol. abs/1904.02441, 2019. [Online]. Available: http://arxiv.org/abs/1904.02441

[57] N. A. Azeez, O. E. Odufuwa, S. Misra, J. Oluranti, and R. Damasevicius, "Windows pe malware detection using ensemble learning," *Informatics*, vol. 8, no. 1, 2021. [Online]. Available: https://www.mdpi.com/2227-9709/8/1/10

[58] K. Raman, "Selecting features to classify malware," *InfoSec Southwest*, 2012. [Online]. Available: https://2012.infosecsouthwest.com/files/speaker_materials/ISSW2012_Selecting_Features_to_Classify_Malware.pdf

[59] M. Jureček, O. Jurečková, and R. Lórencz, "Improving classification of malware families using learning a distance metric," *ICISSP 2021 - Proceedings of the 7th International Conference on Information Systems Security and Privacy*, pp. 643–652, 2021.

[60] S. Kim, S. Yeom, H. Oh, D. Shin, and D. Shin, "Automatic malicious code classification system through static analysis using machine learning," *Symmetry*, vol. 13, no. 1, 2021. [Online]. Available: https://www.mdpi.com/2073-8994/13/1/35

[61] M. Al-Kasassbeh, S. Mohammed, M. Alauthman, and A. Almomani, "Feature selection using a machine learning to classify a malware," in *Handbook of Computer Networks and Cyber Security*, 2020.

[62] A. Mahindru and Sangal, "Mldroid framework for android malware detection using machine learning techniques," *Neural Computing and Applications*, vol. 33, pp. 1–58, 05 2021.

[63] X. Wang and C. Li, "Android malware detection through machine learning on kernel task structures," *Neurocomputing*, vol. 435, pp. 126–150, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231220320002

[64] H. Gao, S. Cheng, and W. Zhang, "Gdroid: Android malware detection and classification with graph convolutional network," *Computers and Security*, vol. 106, p. 102264, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404821000882

[65] Mauricio, "Benign and malicious pe files," May 2018, last accessed: 10.05.2021. [Online]. Available: https://www.kaggle.com/amauricio/pe-files-malwares

[66] J.-M. Roberts, last accessed: 05.05.2021. [Online]. Available: https://virusshare.com/

[67] O. O. D. Science, "Transforming skewed data for machine learning," Jul 2019, last accessed: 13.05.2021. [Online]. Available: https://medium.com/@ODSC/transforming-skewed-data-for-machine-learning-90e6cc364b0

[68] m. Mark K Cowan, "Neural network," 2013, last accessed: 13.05.2021. [Online]. Available: https://github.com/battlesnake/neural

[69] J. Brownlee, "Gentle introduction to the adam optimization algorithm for deep learning," 2017, last accessed: 13.05.2021. [Online]. Available: https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/

[70] P. Probst, M. N. Wright, and A. Boulesteix, "Hyperparameters and tuning strategies for random forest," *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 3, Jan 2019. [Online]. Available: http://dx.doi.org/10.1002/widm.1301

[71] W. Koehrsen, "Hyperparameter tuning the random forest in python," 2018, last accessed: 03.12.2021. [Online]. Available: https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74

[72] S. Bhandari, "How to tune hyperparameter in k nearest neighbors classifier?" 2020, last accessed: 03.12.2021. [Online]. Available: https://sijanb.com.np/posts/how-to-tune-hyperparameter-in-k-nearest-neighbors-classifier/

[73] M. S. K. Inan, "What is hyper parameter tuning in machine learning?" 2020, last accessed: 03.12.2021. [Online]. Available: https://dev.to/techlearners/what-is-hyper-parameter-tuning-in-machine-learning-32kg

[74] C. Liu, "Svm hyperparameter tuning using gridsearchcv," 2020, last accessed: 03.12.2021. [Online]. Available: https://www.vebuso.com/2020/03/svm-hyperparameter-tuning-using-gridsearchcv/

[75] Rendyk, "Tuning the hyperparameters and layers of neural network deep learning," 2021, last accessed: 03.12.2021. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/05/tuning-the-hyperparameters-and-layers-of-neural-network-deep-learning/

[76] A. Mishra, "Metrics to evaluate your machine learning algorithm," 2018, last accessed: 03.12.2021. [Online]. Available: https://towardsdatascience.com/metrics-to-evaluate-your-machine-learning-algorithm-f10ba6e38234

[77] G. Developers, "Classification: Roc curve and auc," 2020, last accessed: 03.12.2021. [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc

[78] H. Graf, E. Cosatto, L. Bottou, I. Dourdanovic, and V. Vapnik, "Parallel support vector machines: The cascade svm," in *Advances in Neural Information Processing Systems*, L. Saul, Y. Weiss, and L. Bottou, Eds., vol. 17. MIT Press, 2005. [Online]. Available: https://proceedings.neurips.cc/paper/2004/file/d756d3d2b9dac72449a6a6926534558a-Paper.pdf

[79] R. Damasevicius, A. Venckauskas, J. Toldinas, and S. Grigaliunas, "Ensemble-based classification using neural networks and machine learning models for windows pe malware detection," *Electronics*, vol. 10, no. 4, 2021. [Online]. Available: https://www.mdpi.com/2079-9292/10/4/485

[80] A. Azmee, P. P. Choudhury, M. Alam, O. Dutta, and M. I. Hossain, "Performance analysis of machine learning classifiers for detecting pe malware," *International Journal of Advanced Computer Science and Applications*, vol. 11, 01 2020.

[81] S. Hou, A. Saas, L. Chen, Y. Ye, and T. Bourlai, "Deep neural networks for automatic android malware detection," in *Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2017*, ser. ASONAM '17. New York, NY, USA: Association for Computing Machinery, 2017, pp. 803–810. [Online]. Available: https://doi.org/10.1145/3110025.3116211

[82] E. B. Karbab, M. Debbabi, A. Derhab, and D. Mouheb, "Android malware detection using deep learning on API method sequences," *CoRR*, vol. abs/1712.08996, 2017. [Online]. Available: http://arxiv.org/abs/1712.08996

[83] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[84] B. Akkaya and N. Çolakoğlu, "Comparison of multi-class classification algorithms on early diagnosis of heart diseases," in *y-BIS 2019 Conference: ISBIS Young Business and Industrial Statisticians Workshop on Recent Advances in Data Science and Business Analytics*, 09 2019, p. 165.

[85] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 2623–2631. [Online]. Available: https://doi.org/10.1145/3292500.3330701

[86] T. Y. Takuya Akiba, Shotaro Sano, "Optuna," 2019, last accessed: 15.05.2021. [Online]. Available: https://optuna.org/#key_features

[87] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, "Tune: A research platform for distributed model selection and training," *arXiv preprint arXiv:1807.05118*, 2018.

[88] J. Brownlee, "When to use mlp, cnn, and rnn neural networks," 2018, last accessed: 15.05.2021. [Online]. Available: https://machinelearningmastery.com/when-to-use-mlp-cnn-and-rnn-neural-networks/

[89] ——, "6 dimensionality reduction algorithms with python," 2020, last accessed: 15.05.2021. [Online]. Available: https://machinelearningmastery.com/dimensionality-reduction-algorithms-with-python/

[90] V. Kouliaridis, G. Kambourakis, and T. Peng, "Feature importance in android malware detection," in *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2020, pp. 1449–1454.

Appendices

A. Features of the Kaggle Dataset

Features	
Name	e_magic
e_cblp	e_cp
e_crlc	e_cparhdr
e_maxalloc	e_ss
e_sp	e_csum
e_ip	e_cs
e_lfarlc	e_ovno
e_oemid	e_oeminfo
e_lfanew	TimeDateStamp
e_minalloc	Machine
PointerToSymbolTable	NumberOfSymbols
SizeOfOptionalHeader	Characteristics
Magic	MajorLinkerVersion
MinorLinkerVersion	SizeOfCode
SizeOfInitializedData	SizeOfUnitializedData
AddressOfEntryPoint	BaseOfCode
ImageBase	SectionAlignment
FileAlignment	MajorOperatingSystemVersion
MinorOperatingSystemVersion	MajorImageVersion
MinorImageVersion	MajorSubsystemVersion
MinorSubsystemVersion	SizeOfHeaders
CheckSum	SizeOfImage
Subsystem	DllCharacteristics
SizeOfStackReserve	SizeOfStackCommit
SizeOfHeapReserve	SizeOfHeapCommit
LoaderFlags	NumberOfRvaAndSizes
Malware	SuspiciousImportFunctions
SuspiciousNameSection	SectionsLength
SectionMinEntropy	SectionMaxEntropy
SectionMinRawsize	SectionMaxRawsize
SectionMinVirtualsize	SectionMaxVirtualsize
SectionMaxPhysical	SectionMinPhysical
SectionMaxVirtual	SectionMinVirtual
SectionMaxPointerData	SectionMinPointerData
SectionMaxChar	SectionMainChar
DirectoryEntryImport	DirectoryEntryImportSize
DirectoryEntryExport	ImageDirectoryEntryExport
ImageDirectoryEntryImport	ImageDirectoryEntryResource
ImageDirectoryEntryException	ImageDirectoryEntrySecurity
NumberOfSections	

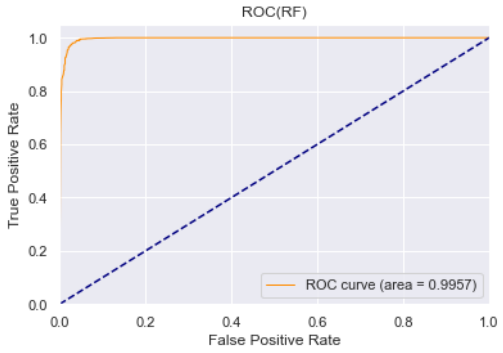
Table A.1.: Features of the Kaggle dataset

B. Features of the Virusshare Dataset

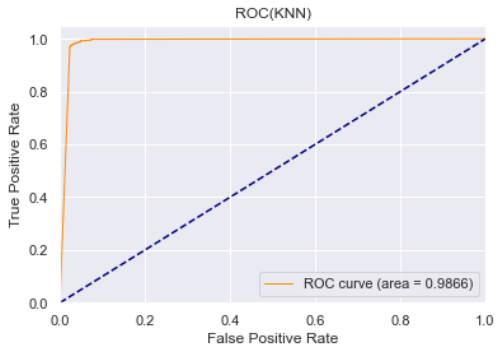
Features	
Name	md5
Machine	legitimate
SizeOfOptionalHeader	Characteristics
BaseOfData	MajorLinkerVersion
MinorLinkerVersion	SizeOfCode
SizeOfInitializedData	SizeOfUnitializedData
AddressOfEntryPoint	BaseOfCode
ImageBase	SectionAlignment
FileAlignment	MajorOperatingSystemVersion
MinorOperatingSystemVersion	MajorImageVersion
MinorImageVersion	MajorSubsystemVersion
MinorSubsystemVersion	SizeOfHeaders
CheckSum	SizeOfImage
Subsystem	DllCharacteristics
SizeOfStackReserve	SizeOfStackCommit
SizeOfHeapReserve	SizeOfHeapCommit
LoaderFlags	NumberOfRvaAndSizes
SectionsNb	SectionsMeanEntropy
SectionsMinEntropy	SectionsMaxEntropy
SectionsMinRawsize	SectionsMaxRawsize
SectionssMinVirtualsize	SectionMaxVirtualsize
SectionMeanVirtualsize	SectionsMeanRawsize
ImportsNbDLL	ImportsNb
ImportsNbDLL	ImportsNbOrdinal
ExportsNb	ResourcesNb
ResourcesMeanEntropy	ResourcesMinEntropy
ResourcesMeanSize	ResourcesMinSize
ResourcesMaxSize	LoadConfigurationSize
VersionInformationSize	

Table B.1.: Features of the Virusshare dataset

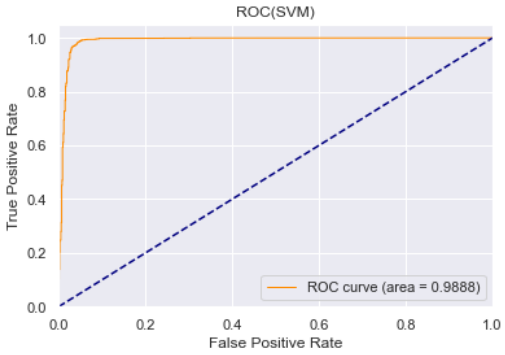
C. Classification Results of the Kaggle Dataset



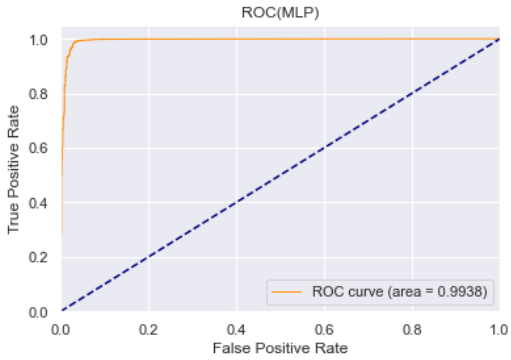
(a) ROC and AUC of Random Forest



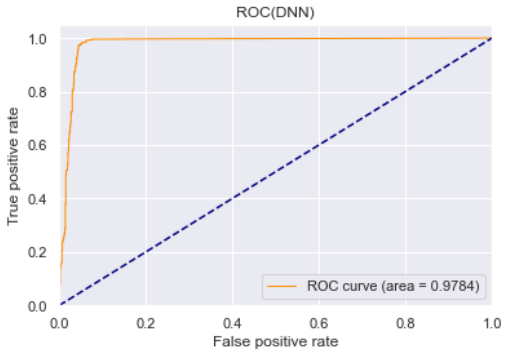
(b) ROC and AUC of KNN



(c) ROC and AUC of SVM



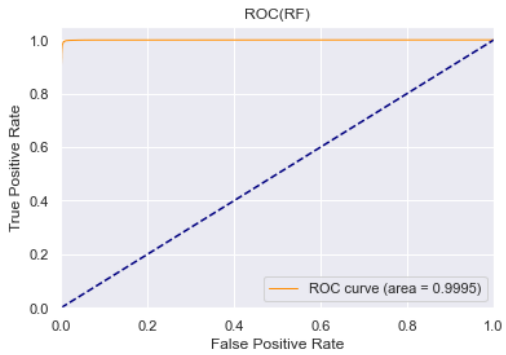
(d) ROC and AUC of MLP



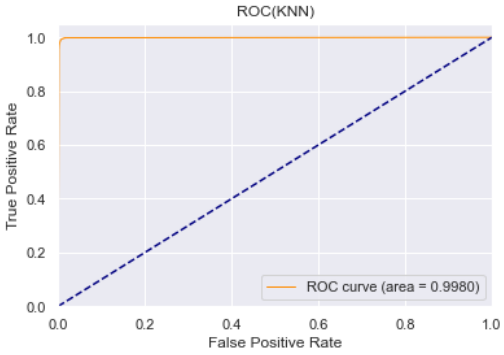
(e) ROC and AUC of DNN

Figure C.1.: ROC and AUC of the classifiers with Keras Tuner

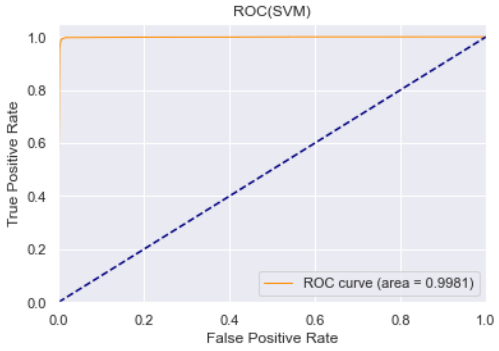
D. Classification Results of the Virusshare Dataset



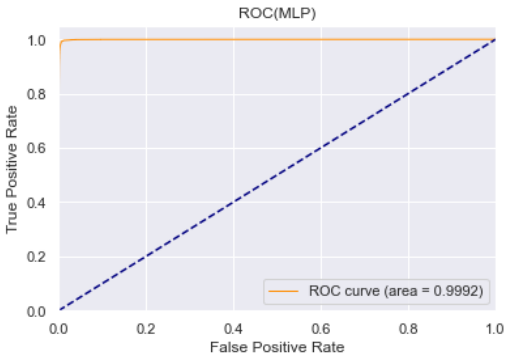
(a) ROC and AUC of Random Forest



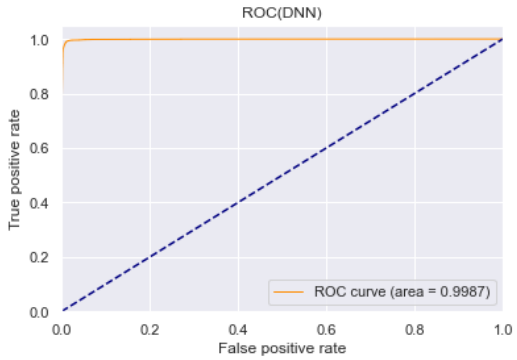
(b) ROC and AUC of KNN



(c) ROC and AUC of SVM

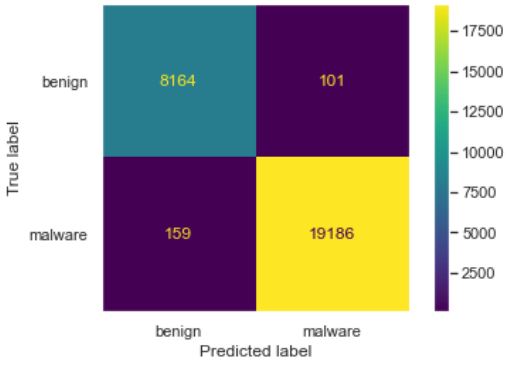


(d) ROC and AUC of MLP

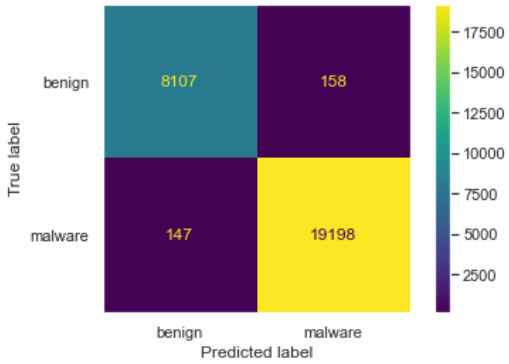


(e) ROC and AUC of DNN

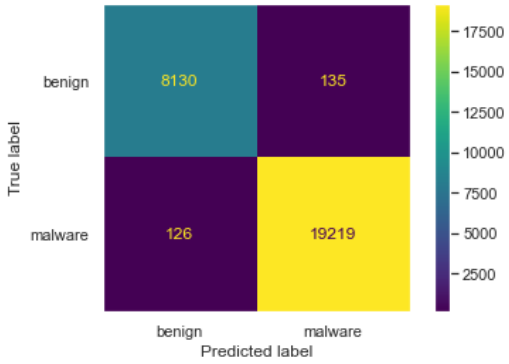
Figure D.1.: ROC and AUC of the classifiers with Keras Tuner



(a) Confusion matrix of KNN



(b) Confusion matrix of SVM



(c) Confusion matrix of MLP

Figure D.2.: Confusion matrices of the classifiers without PCA

E. Feature Importance of the Kaggle Dataset

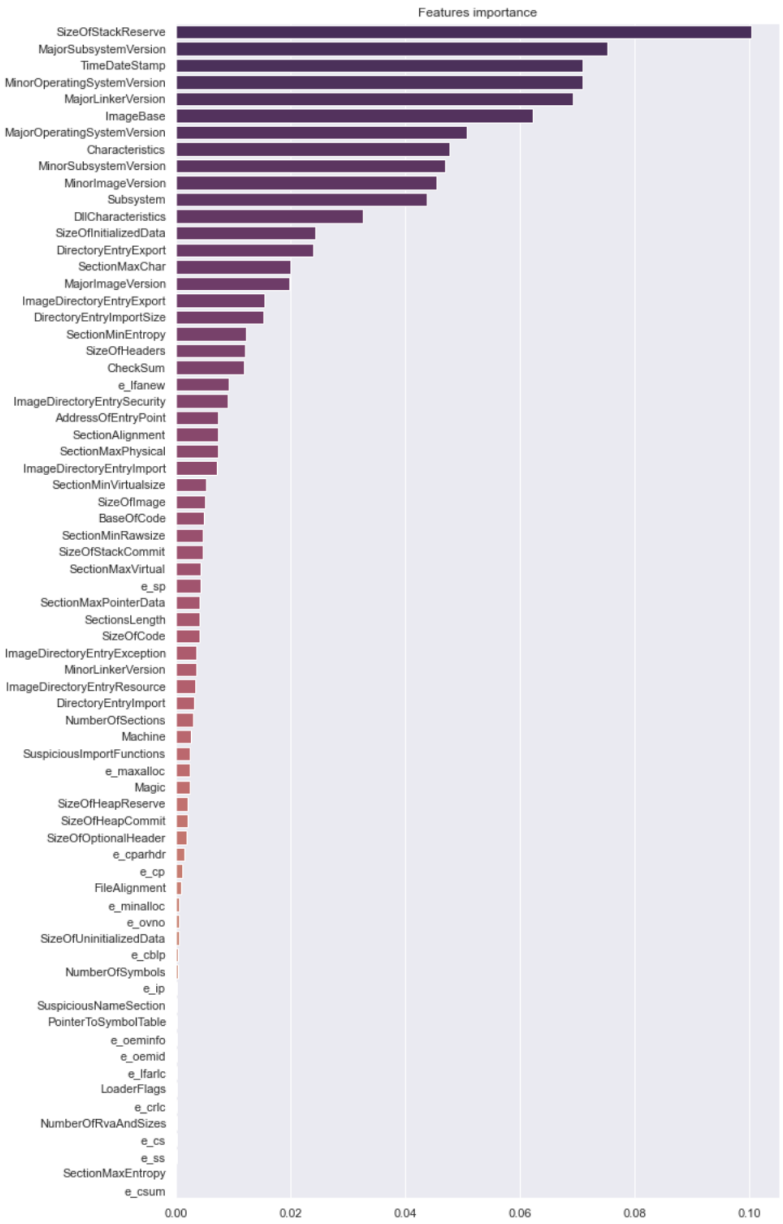


Figure E.1.: Feature importance of the Kaggle dataset

F. Feature Importance of the Virusshare Dataset

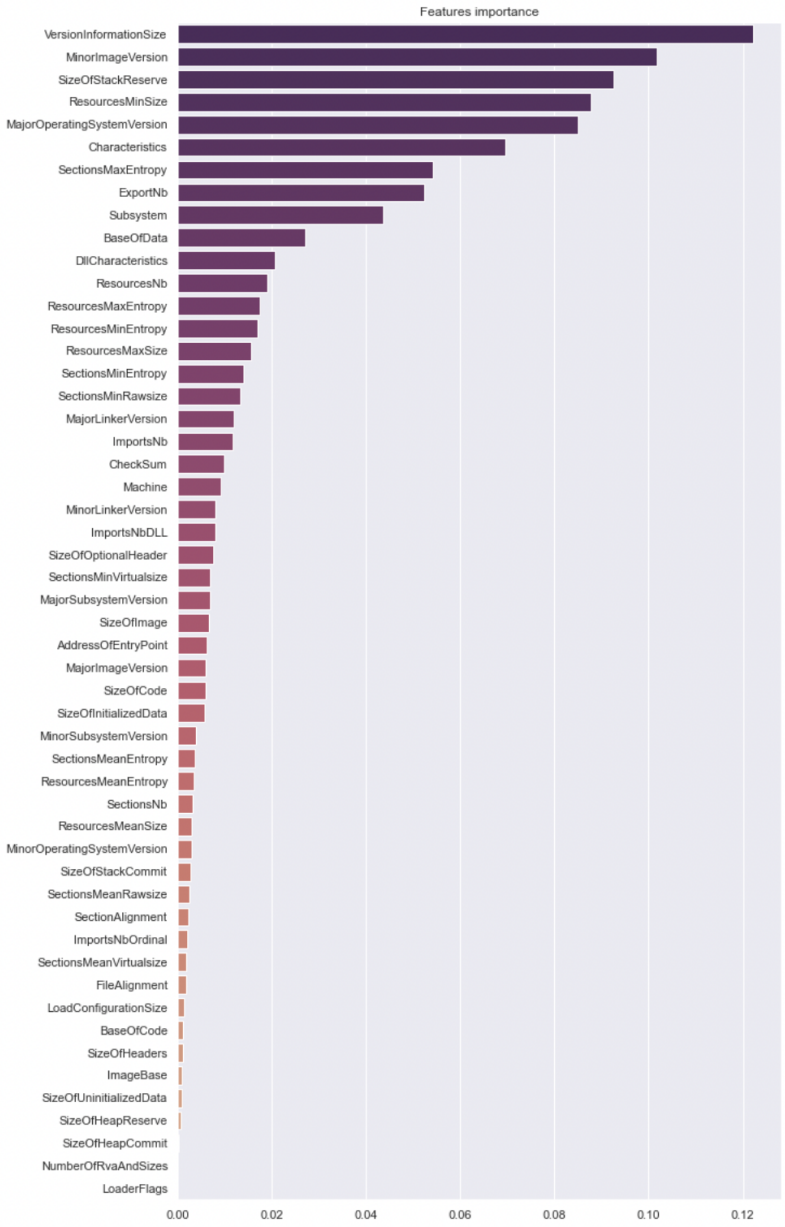


Figure F.1.: Feature importance of the Virusshare dataset